

第2章 线性表

提纲

CONTENTS



2.1 线性表及其逻辑结构

2.2 线性表的顺序存储结构

2.3 线性表的链式存储结构

2.4 线性表的应用

2.5 有序表

2.2 线性表的顺序存储结构

2.2.1 线性表的顺序存储—顺序表

线性表的顺序存储结构:

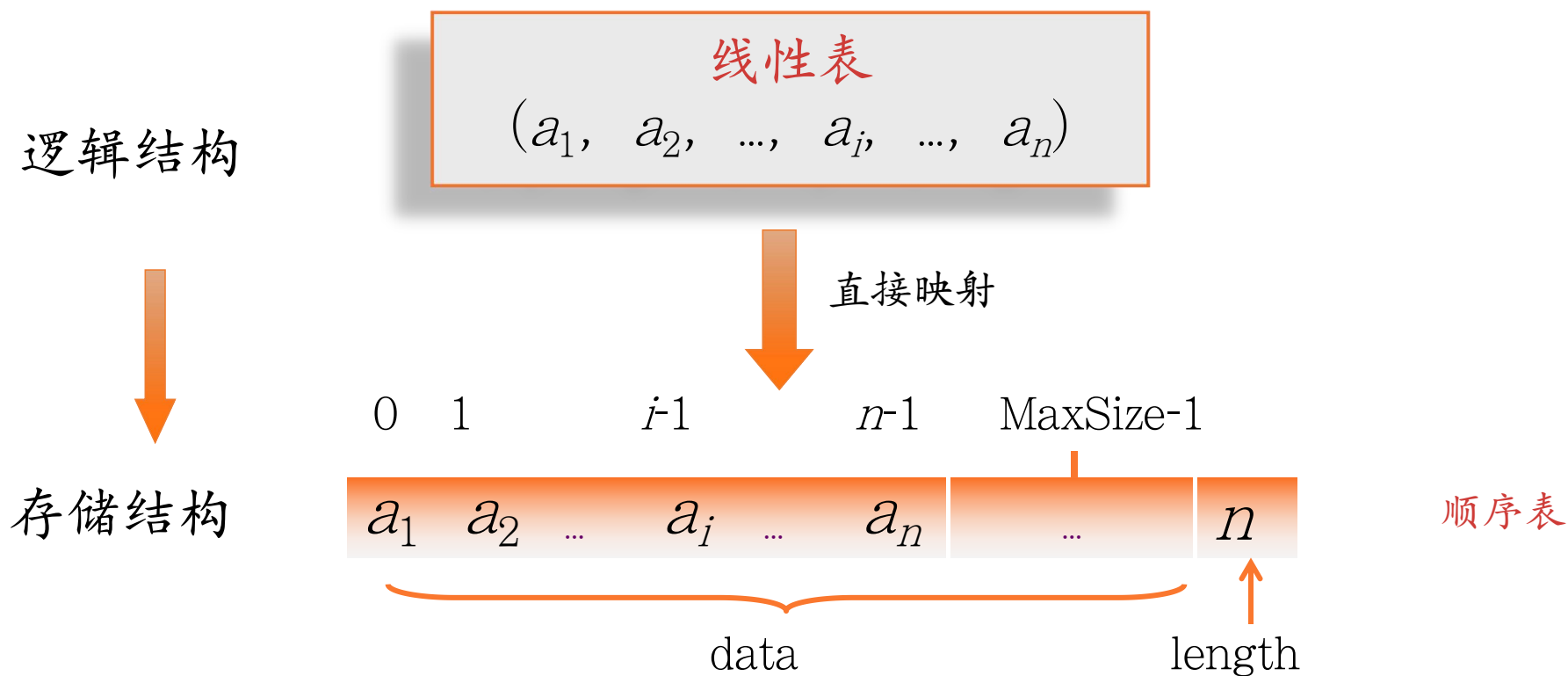
把线性表中的所有元素按照顺序存储方法进行存储。



按逻辑顺序依次存储到存储器中一片连续的存储空间中。

2.2 线性表的顺序存储结构

2.2.1 线性表的顺序存储—顺序表



说明：注意逻辑位序和物理位序相差1。

按逻辑顺序依次存储到存储器中一片连续的存储空间中。

2.2 线性表的顺序存储结构

2.2.1 线性表的顺序存储—顺序表

➤ 顺序表类型定义

这里，假设ElemType为char、int等类型

```
typedef struct  
{ ElemType data[MaxSize];  
  int length;  
} SqList;           //顺序表类型
```

其中data成员存放元素，length成员存放线性表的实际长度。

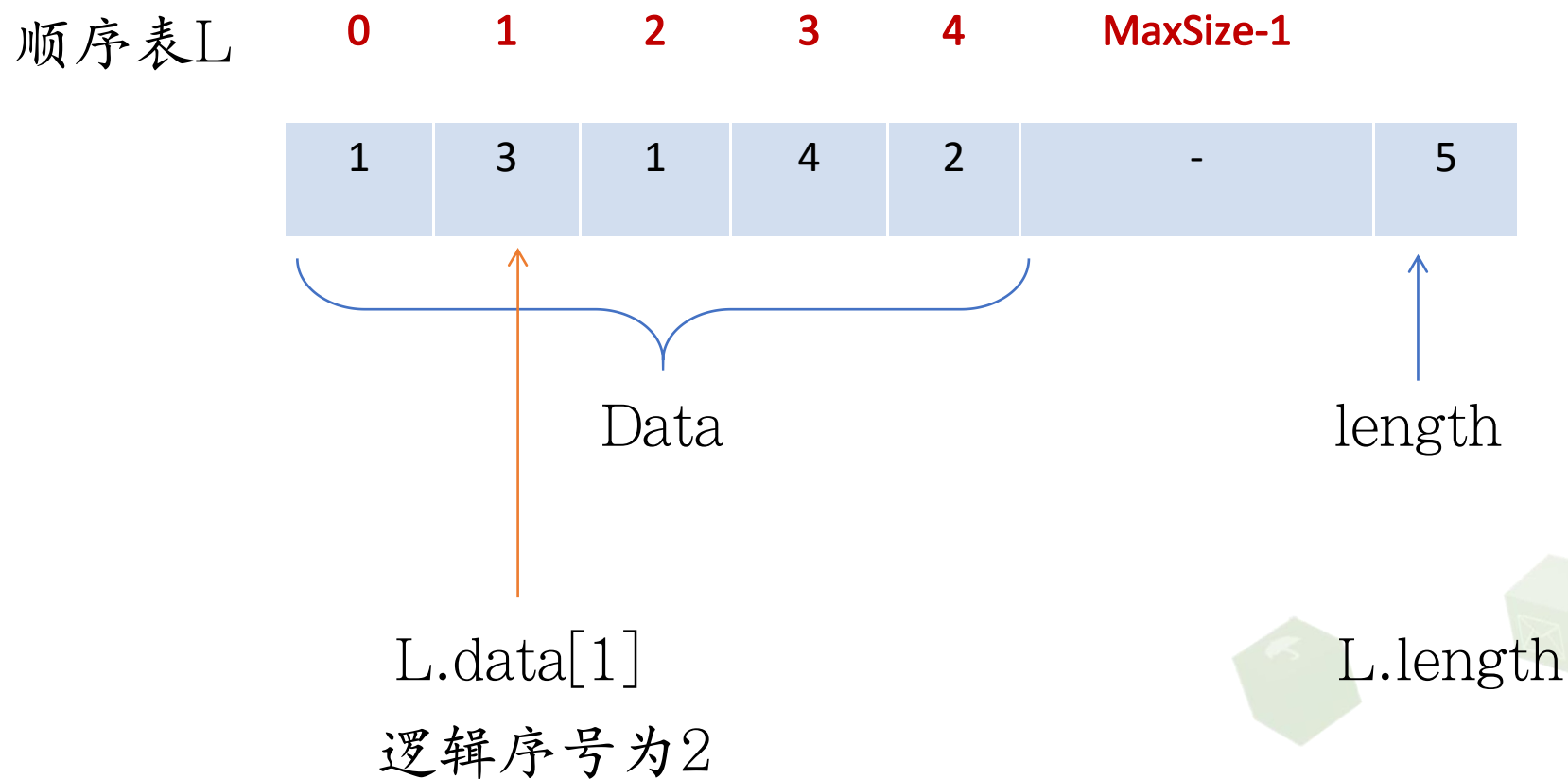
说明：注意逻辑位序和物理位序相差1。



2.2 线性表的顺序存储结构

2.2.1 线性表的顺序存储—顺序表

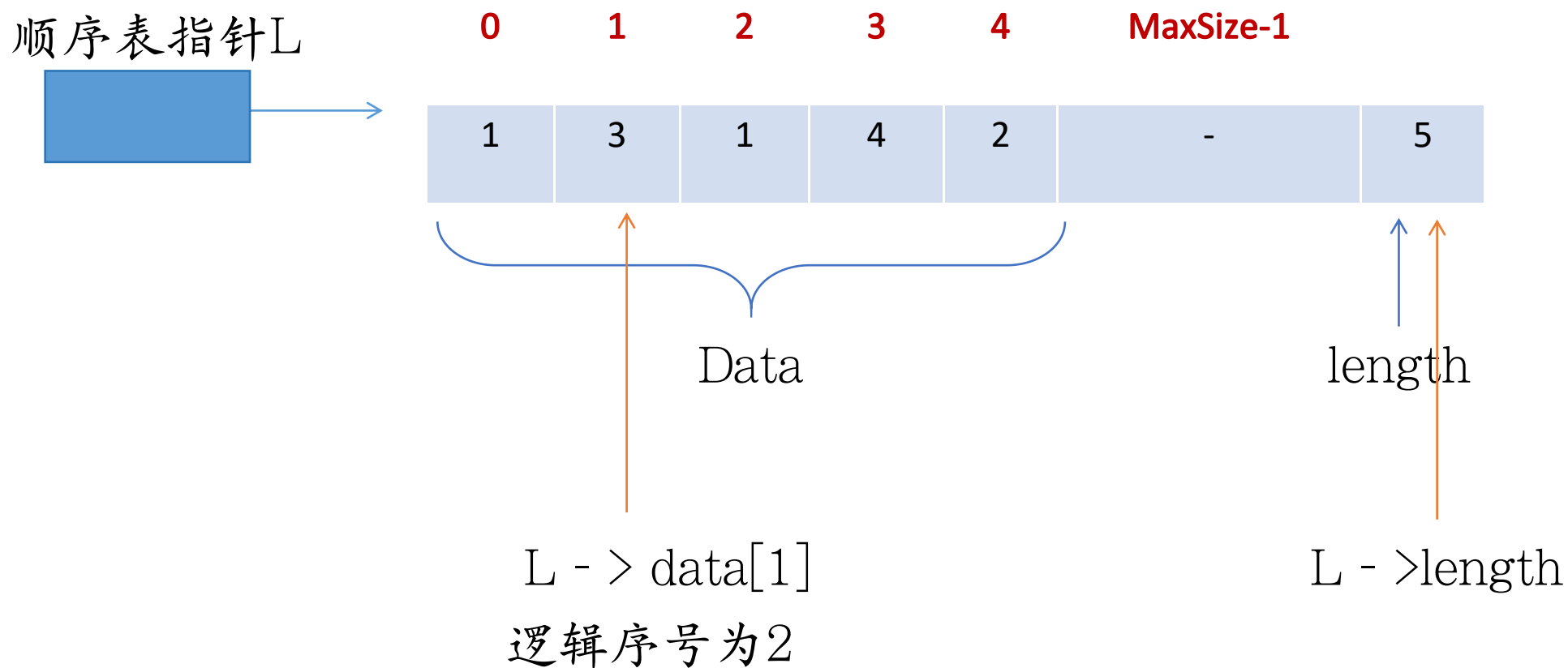
示例： 一个整数型线性表 $L = (1, 3, 1, 4, 2)$



2.2 线性表的顺序存储结构

2.2.1 线性表的顺序存储—顺序表

示例： 一个整数型线性表 $L = (1, 3, 1, 4, 2)$



2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

1、建立顺序表

$a[0..n-1] \Rightarrow$ 顺序表L — 整体创建顺序表。

```
void CreateList(SqList * &L, ElemType a[], int n)
{
    int i=0, k=0;
    L=(SqList *)malloc(sizeof(SqList));
    while (i<n)           //i扫描a中元素
    {
        L->data[k]=a[i];
        k++; i++;         //k记录插入到L中的元素个数
    }
    L->length=k;
}
```

→ 传递顺序表指针



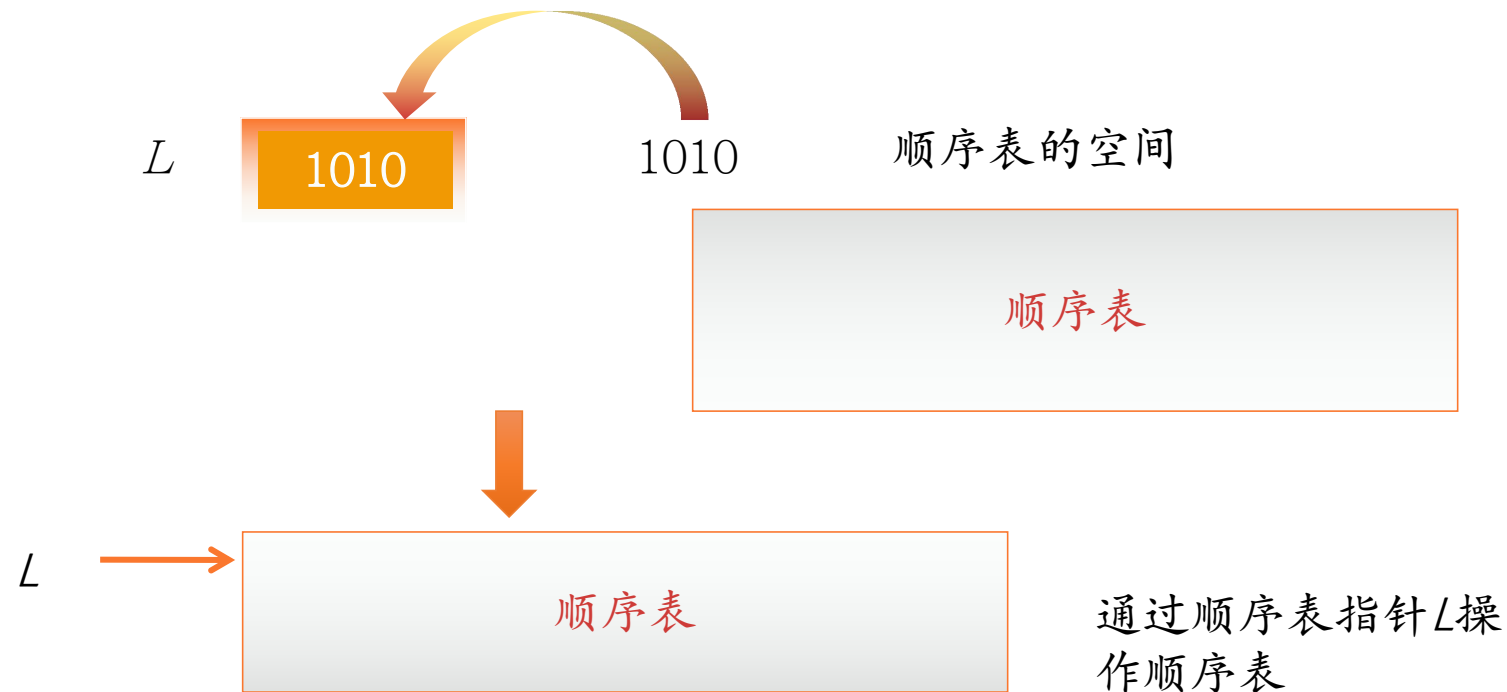
2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

算法参数说明

① 顺序表指针的含义

```
SqList *L;  
L=(SqList *)malloc(sizeof(SqList));
```



2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

② 顺序表指针引用

```
void CreateList(SqList *&L, ElemType a[], int n)
```



引用参数：将执行结果回传给实参

- 引用符号“&”放在形参L的前面。
- 输出型参数均为使用“&”。

引用参数SqList *&L  将顺序表地址L回传给对应的实参

2.2 线性表的顺序存储结构

示例：如果使用引用

```
void main()
{
    SqList *h;
    int n = 5;
    ElemType a[] = {1,3,1,4,2};
    CreateList(h, a, n);
    // 对顺序表h进行其他操作
}
```

```
void CreateList(SqList *&L, ElemType a[ ], int n)
{
    int i = 0, k = 0;
    L = (SqList *)malloc(sizeof(SqList));
    while (i < n)    // i扫描a中元素
    {
        L->data[k] = a[i];
        k++;
        i++;        // k记录插入到L中的元素个数
    }
    L->length = k;
}
```

h

???

L

1 3 1 4 2 5

h指向有效顺序表，可以操作

2.2 线性表的顺序存储结构

示例：如果不使用引用

```
void main()
{
    SqList *h;
    int n = 5;
    ElemType a[] = {1,3,1,4,2};
    CreateList(h, a, n);
    // 对顺序表h进行其他操作
}
```

```
void CreateList(SqList *L, ElemType a[ ], int n)
{
    int i = 0, k = 0;
    L = (SqList *)malloc(sizeof(SqList));
    while (i < n)    // i扫描a中元素
    {
        L->data[k] = a[i];
        k++;
        i++;        // k记录插入到L中的元素个数
    }
    L->length = k;
}
```

h

???

h依然是垃圾值，操作错误

L

1 3 1 4 2 5

2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

2、顺序表基本运算算法

(1) 初始化线性表InitList(L)

构造一个空的线性表L。实际上只需将length成员设置为0即可。

```
void InitList(SqList *&L)
{
    L=(SqList *)malloc(sizeof(SqList));
    //分配存放线性表的顺序表空间
    L->length=0;
}
```


2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

(2) 销毁线性表DestroyList(L)

释放线性表L占用的内存空间。

```
void DestroyList(SqList *&L)
{
    free(L);
}
```

L



顺序表

free(L)释放L所指向的空间

顺序表采用指针传递，有两个优点：

- 更清楚看到顺序表创建和销毁过程（malloc/free）。
- 在算法的函数之间传递更加节省空间（在函数体内不必创建值形参即整个顺序表的副本）。

2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

(3) 判定是否为空表ListEmpty(L)

回一个值表示L是否为空表。若L为空表，则返回true，否则返回false。

```
bool ListEmpty(SqList *L)
{
    return(L->length==0);
}
```



2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

(4) 求线性表的长度ListLength(L)

返回顺序表L的长度。实际上只需返回length成员的值即可。

```
int ListLength(SqList *L)
{
    return(L->length);
}
```



2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

(5) 输出线性表DispList(L)

当线性表L不为空时，顺序显示L中各元素的值。

```
void DispList(SqList *L)
{
    int i;
    if (ListEmpty(L)) return;
    for (i=0; i<L->length; i++)
        printf("%c", L->data[i]);
    printf("\n");
}
```



2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

(6) 求某个数据元素值 `GetElem(L, i, e)`

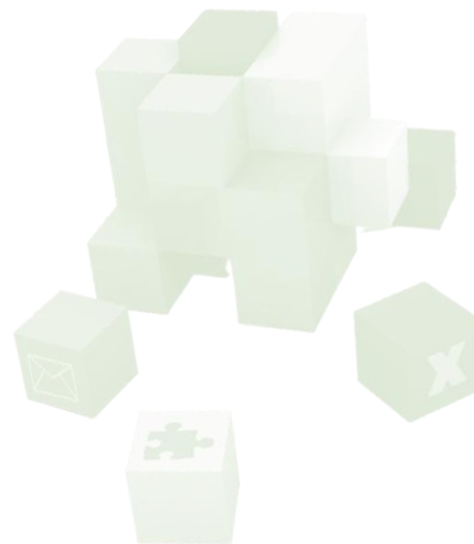
返回L中第 i ($1 \leq i \leq \text{ListLength}(L)$) 个元素的值, 存放在 e 中。

```
bool GetElem(SqList *L, int i, ElemType &e)
{
    if (i < 1 || i > L->length) return false;
    e = L->data[i-1];
    return true;
}
```

本算法的时间复杂度为 $O(1)$ 。



体现顺序表的**随机存取特性**



2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

(7) 按元素值查找LocateElem(L, e)

顺序查找第一个值域与 e 相等的元素的逻辑位序。若这样的元素不存在，则返回值为0。

```
int LocateElem(SqList *L, ElemType e)
{
    int i=0;
    while (i<L->length && L->data[i]!=e)
        i++;
    if (i>=L->length) return 0;
    else return i+1;
}
```



2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

(8) 插入数据元素 $\text{ListInsert}(L, i, e)$

在顺序表 L 的第 i ($1 \leq i \leq \text{ListLength}(L)+1$) 个位置上插入新的元素 e 。



插入完成

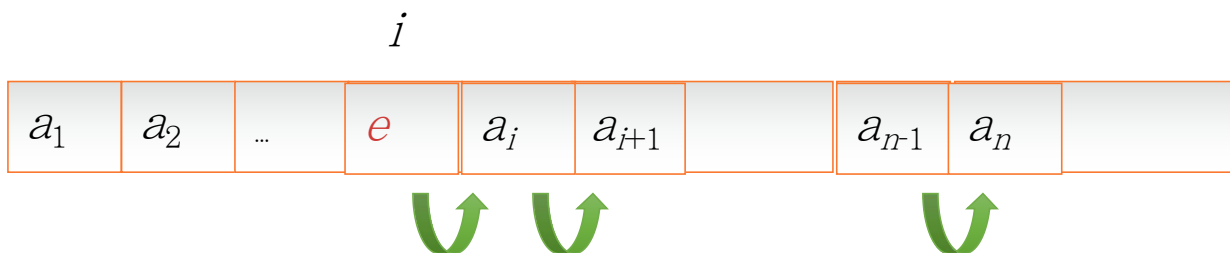


2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

插入算法如下:

```
bool ListInsert(SqList *&L, int i, ElemType e)
{
    int j;
    if (i < 1 || i > L->length + 1)
        return false;           //参数错误时返回false
    i--;                          //将顺序表逻辑序号转化为物理序号
    for (j = L->length; j > i; j--) //将data[i..n]元素后移一个位置
        L->data[j] = L->data[j-1];
    L->data[i] = e;               //插入元素e
    L->length++;                  //顺序表长度增1
    return true;                 //成功插入返回true
}
```



2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

对于本算法来说，元素移动的次数不仅与表长 $L \rightarrow \text{length} = n$ 有关，而且与插入位置 i 有关：

- 当 $i = n + 1$ 时，移动次数为 0；
- 当 $i = 1$ 时，移动次数为 n ，达到最大值。

算法最好时间复杂度为 $O(1)$

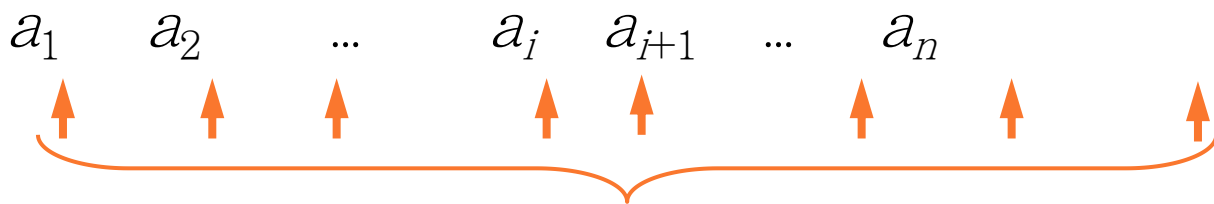
算法最坏时间复杂度为 $O(n)$



2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

平均情况分析：



在线性表L中共有 $n+1$ 个可以插入元素的地方

在插入元素 a_i 时，若为等概率情况，则 $p_i = \frac{1}{n+1}$

此时需要将 $a_i \sim a_n$ 的元素均后移一个位置，共移动 $n-i+1$ 个元素。

在长度为 n 的线性表中插入一个元素时所需移动元素的平均次数为：

$$\sum_{i=1}^{n+1} p_i (n - i + 1) = \sum_{i=1}^{n+1} \frac{1}{n+1} (n - i + 1) = \frac{n}{2}$$

因此插入算法的平均时间复杂度为 $O(n)$ 。



2.2 线性表的顺序存储结构

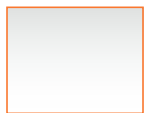
2.2.2 顺序表运算的实现

(9) 删除数据元素 $\text{ListDelete}(L, i, e)$

删除顺序表 L 的第 i ($1 \leq i \leq \text{ListLength}(L)$) 个元素。



e



删除完成

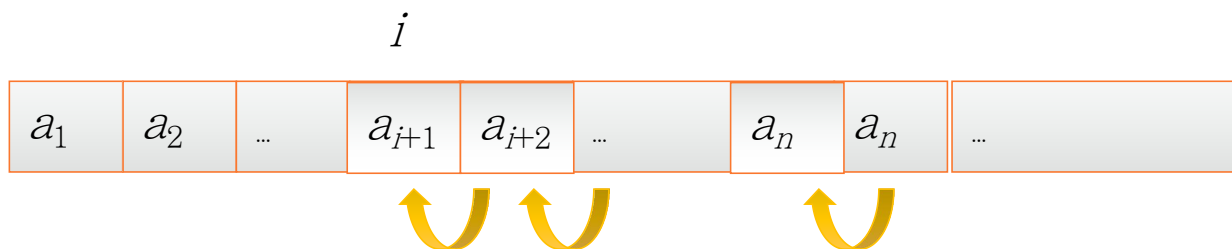


2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

删除算法如下：

```
bool ListDelete(SqList *&L, int i, ElemType &e)
{
    int j;
    if (i < 1 || i > L->length)           //参数错误时返回false
        return false;
    i--;                                   //将顺序表逻辑序号转化为物理序号
    e = L->data[i];
    for (j = i; j < L->length - 1; j++)    //将data[i..n-1]元素前移
        L->data[j] = L->data[j + 1];
    L->length--;                           //顺序表长度减1
    return true;                           //成功删除返回true
}
```



2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

对于本算法来说，元素移动的次数也与表长 n 和删除元素的位置 i 有关：

- 当 $i=n$ 时，移动次数为0；
- 当 $i=1$ 时，移动次数为 $n-1$ 。

删除算法最好时间复杂度为 $O(1)$

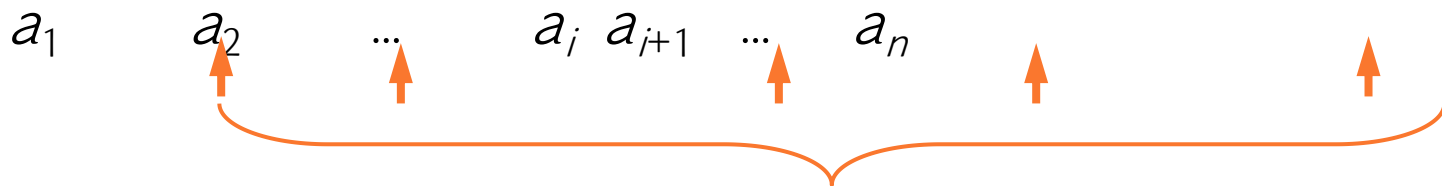
删除算法最坏时间复杂度为 $O(n)$



2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

平均情况分析:



在线性表L中共有 n 个可以删除元素的地方

在删除元素 a_i 时, 若为等概率情况, 则 $p_i = \frac{1}{n}$

此时需要将 $a_{i+1} \sim a_n$ 的元素均前移一个位置, 共移动 $n-(i+1)+1 = n-i$ 个元素。

所以在长度为 n 的线性表中删除一个元素时所需移动元素的平均次数为:

$$\sum_{i=1}^n p_i (n-i) = \sum_{i=1}^n \frac{1}{n} (n-i) = \frac{n-1}{2}$$

因此删除算法的平均时间复杂度为 $O(n)$ 。

2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

思考题

❶ 假如有一个学生表，每个学生包含学号、姓名和分数。你如何设计相应的学生顺序表？

❷ 如果需要对该学生表进行插入、修改和删除运算，你如何实现相关算法？



2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

3、顺序表的应用示例

顺序表应用算法设计：

数据采用顺序表存储，利用顺序表的基本操作来完成求解任务。

2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

顺序表应用算法设计： 数据采用顺序表存储，利用顺序表的基本操作来完成任务

- 基于整体建表的算法设计
- 基于顺序表基本操作的算法设计

2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

- 基于整体建表的算法设计

这类算法设计中需要根据条件产生新的结果顺序表



两者可以相同，也可以不同

2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

【例2.3】 已知长度为 n 的线性表 L 采用顺序存储结构。设计一个时间复杂度为 $O(n)$ 、空间复杂度为 $O(1)$ 的算法，该算法删除线性表中所有值为 x 的数据元素。

解

2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

解法一（整体建表法）

设删除 L 中所有值等于 x 元素后的顺序表为 $L1$ ，显然 $L1$ 包含在 L 中，为此 $L1$ 重用 L 的空间。

思路：扫描顺序表 L ，重建 L 只包含不等于 x 的元素。



2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

删除顺序表中所有值为 x 的元素（方法1） 演示

删除所有 $x=2$ 的元素（ k 记录保留的元素个数，初值=0）：



k $k=3$

$k=3$, $L \rightarrow \text{length} = k = 3$

删除完成

2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

对应的算法如下：

```
void delnode1(SqList *&L, ElemType x)
{ int k=0, i;           //k记录值不等于x的元素个数
  for (i=0;i<L->length;i++)
    if (L->data[i]!=x)   //若当前元素不为x, 将其插入A中
    { L->data[k]=L->data[i];
      k++;               //不等于x的元素增1
    }
  L->length=k;          //顺序表L的长度等于k
}
```

2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

解法二（前移）

用 k 记录顺序表 L 中等于 x 的元素个数，一边扫描 L 一边统计 k 值。

思路：将不为 x 的元素前移 k 个位置，最后修改 L 的长度。



2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

删除顺序表中所有值为 x 的元素（方法2） 演示

删除所有 $x=2$ 的元素（ k 记录删除的元素个数，初值=0）

0	1	2	3	4	5	length
1	2	1	2	3	2	3

$k=3$

顺序表长度= $6-k=3$

删除完成

2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

对应的算法如下：

```
void delnode2(SqList *&L, ElemType x)
{ int k=0, i=0;           //k记录值等于x的元素个数
  while (i<L->length)
  { if (L->data[i]==x)     //当前元素值为x时k增1
    k++;
    else                  //当前元素不为x时将其前移k个位置
      L->data[i-k] = L->data[i];
    i++;
  }
  L->length-=k;           //顺序表L的长度递减k
}
```

2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

解法三（区间划分法）

$\text{data}[0 \dots i]$

不为 x 的区间 为 x 的区间



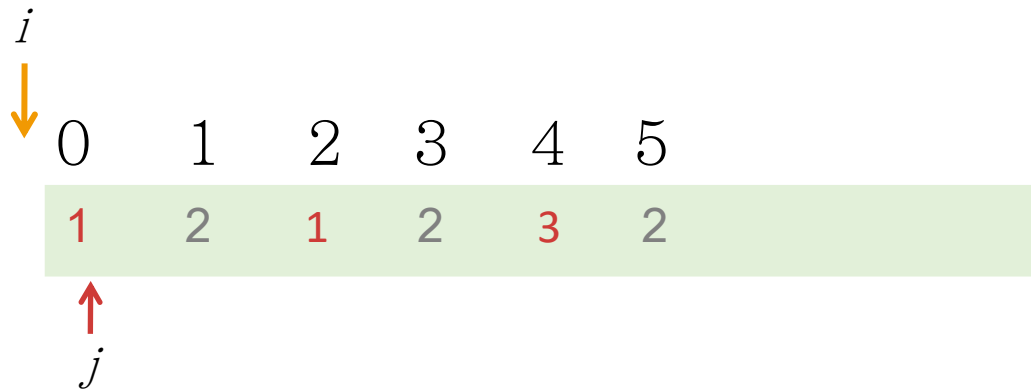
- $\text{data}[j]=x$: $j++$
- $\text{data}[j] \neq x$, i 增加1扩大“不为 x 的区间”，将 $\text{data}[i]$ 与 $\text{data}[j]$ 交换



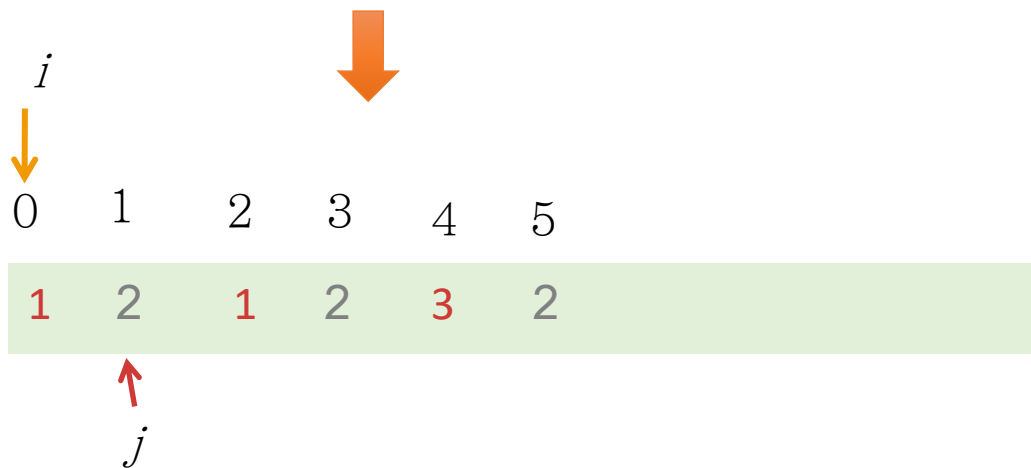
2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

删除所有 $x=2$ 的元素，初始时: $i=-1$, $j=0$

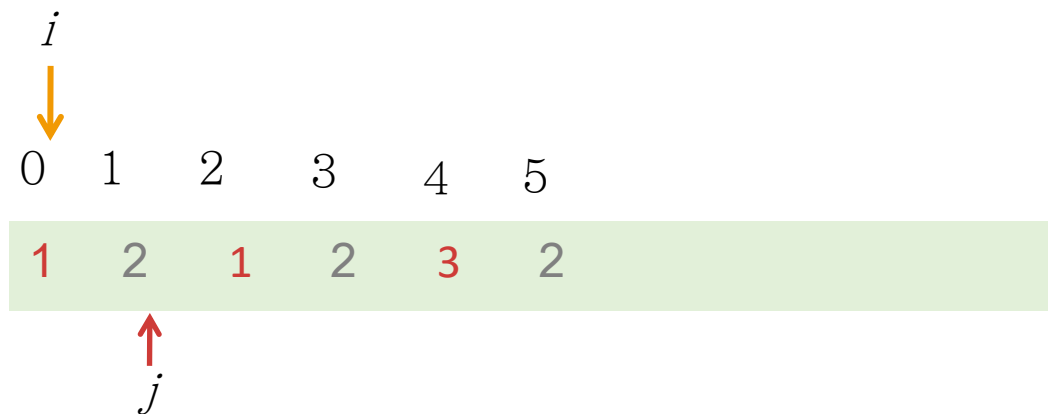


$j=0, \text{data}[0] \neq x$: $i++ \Rightarrow i=0$, $j++ \Rightarrow j=1$

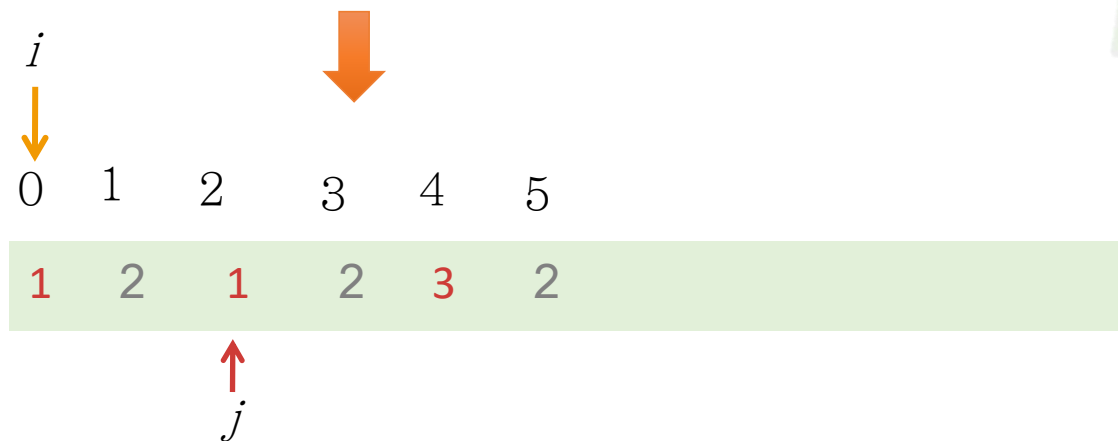


2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

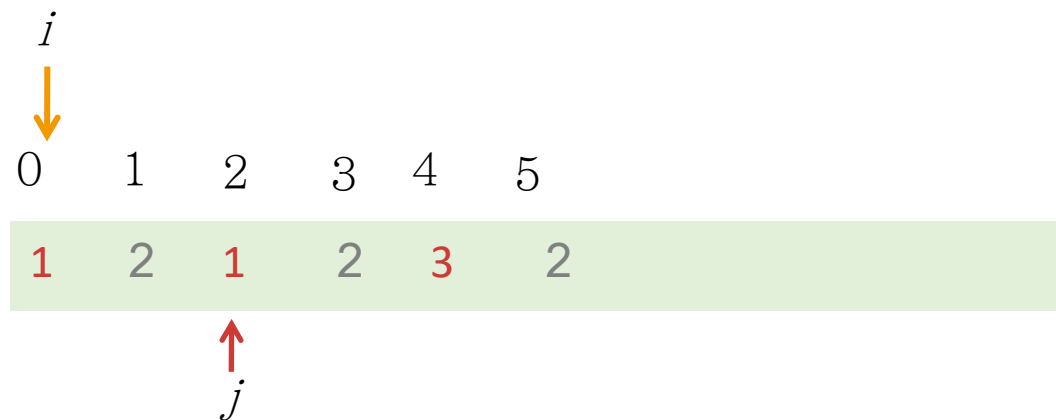


$j=1, \text{data}[1]=x: j++ \Rightarrow j=2$

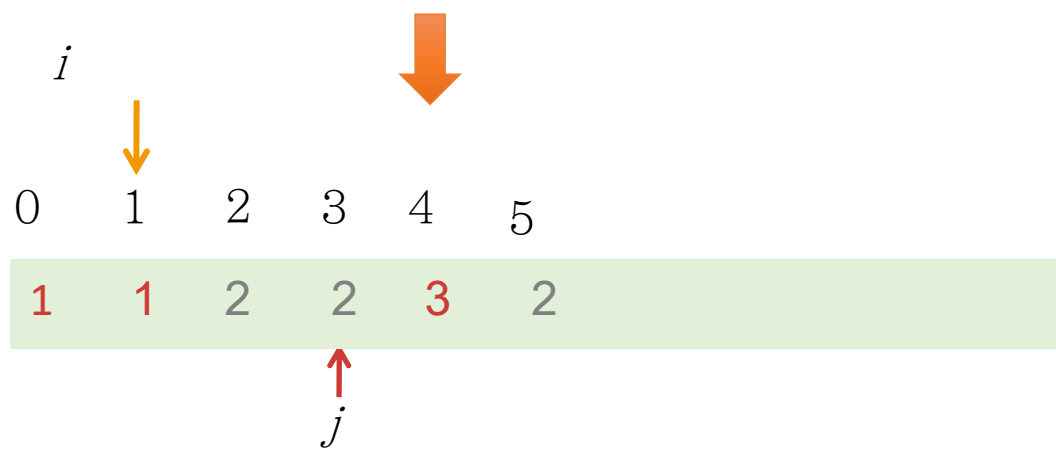


2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

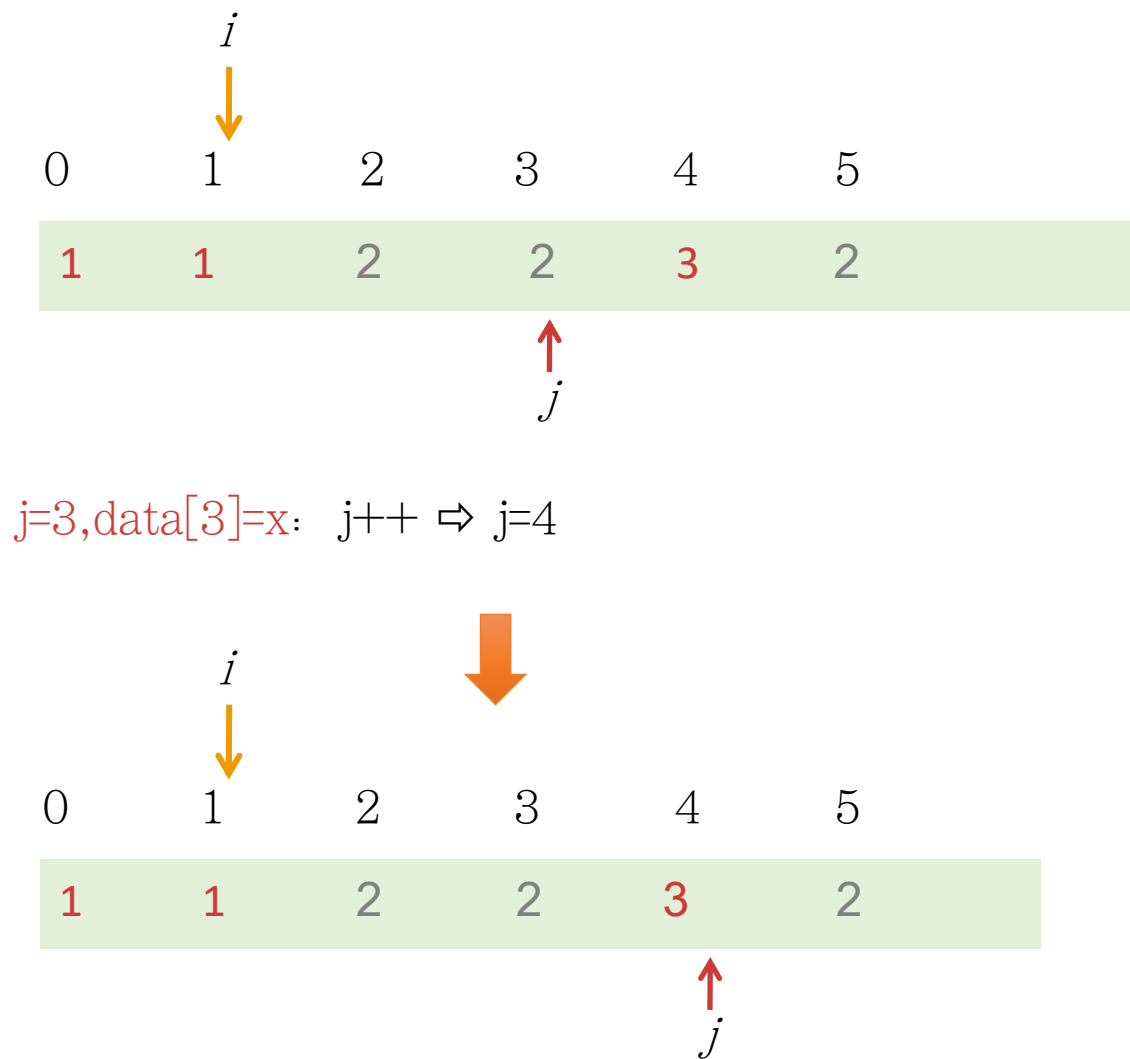


$j=2, \text{data}[2] \neq x$: $i++ \Rightarrow i=1$, $\text{swap}(\text{data}[2], \text{data}[1])$, $j++ \Rightarrow j=3$



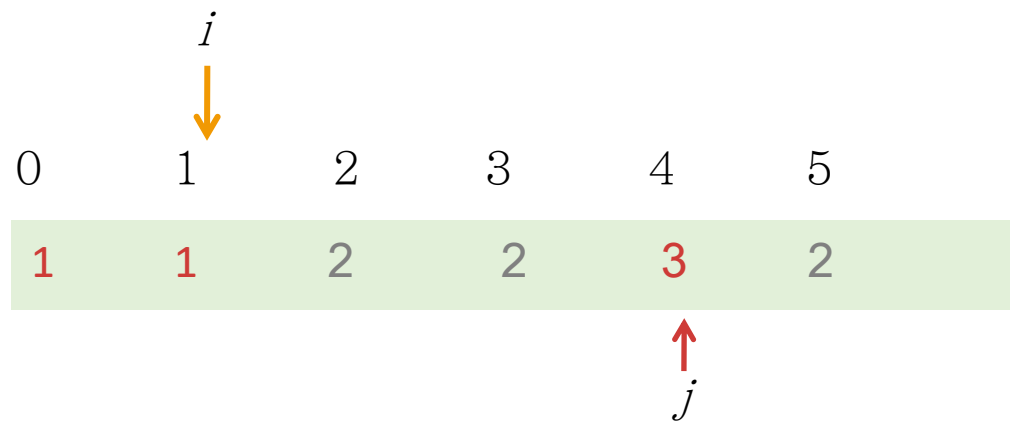
2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

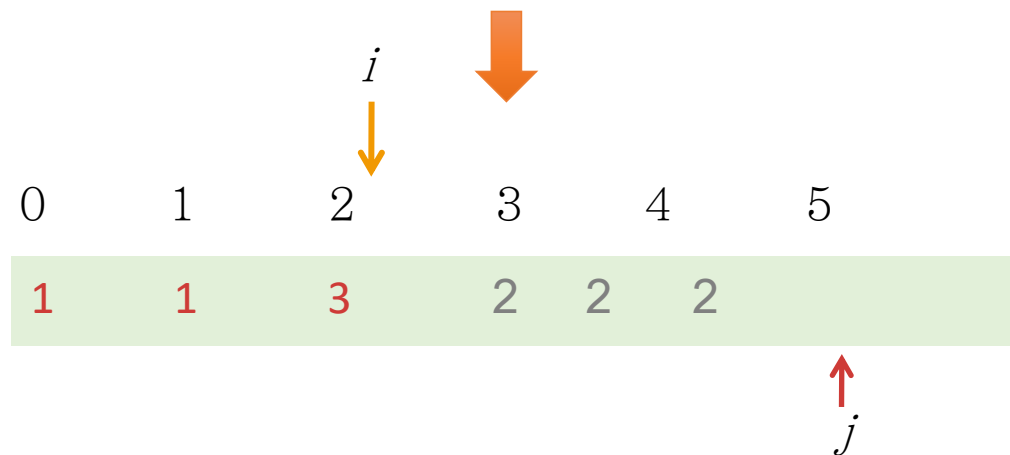


2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

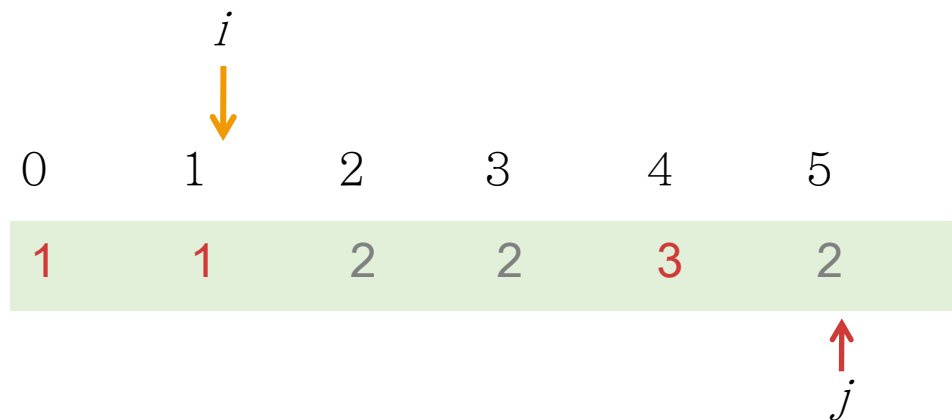


$j=4, \text{data}[4] \neq x$: $i++ \Rightarrow i=2$, $\text{swap}(\text{data}[2], \text{data}[4])$, $j++ \Rightarrow j=5$

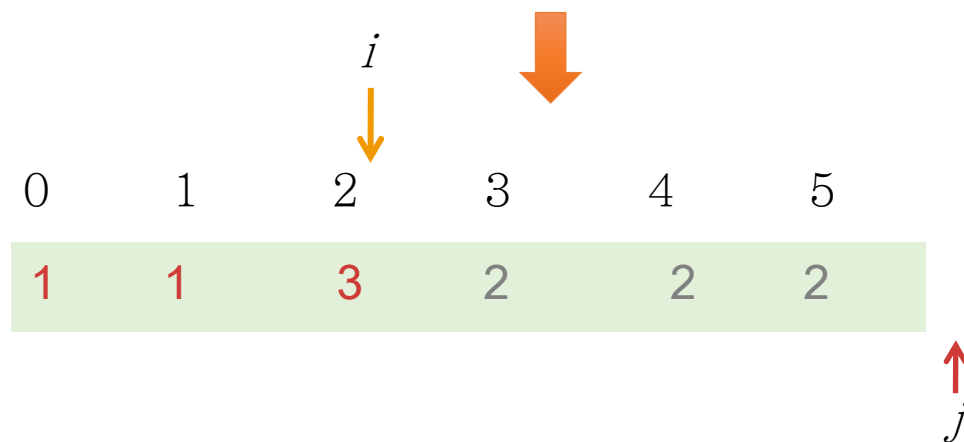


2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现



$j=5, \text{data}[5]=x: j++ \Rightarrow j=6$



删除完成

$\text{length}=i+1=3$

2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

```
void delnode3(SqList *&L, ElemType x)
{
    int i=-1, j=0;
    while (j<L->length)    //j扫描所有元素
    {
        if (L->data[j]!=x)    //找到不为x的元素
        {
            i++;                //扩大不为x的区间
            if (i!=j)
                swap(L->data[i], L->data[j]);
        }
        j++;                //继续扫描
    }
    L->length=i+1;            //设置L中实际元素个数
}
```

2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

【例2.3】 已知长度为 n 的线性表 L 采用顺序存储结构。设计一个时间复杂度为 $O(n)$ 、空间复杂度为 $O(1)$ 的算法，该算法删除线性表中所有值为 x 的数据元素。

解

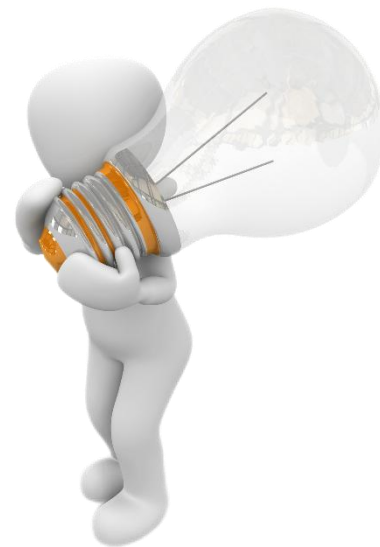
以下两种方法都不满足要求：

- 如果每删除一个值为 x 的元素都进行移动，其时间复杂度为 $O(n^2)$ ，空间复杂度为 $O(1)$ 。
- 如果借助一个新的顺序表，存放将 A 中所有不为 x 的元素，其时间复杂度为 $O(n)$ ，空间复杂度为 $O(n)$ 。

2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

【例2.4】 设顺序表L有10个整数。设计一个算法，以第一个元素为分界线（基准），将所有小于等于它的元素移到该元素的前面，将所有大于它的元素移到该元素的后面。



2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

解

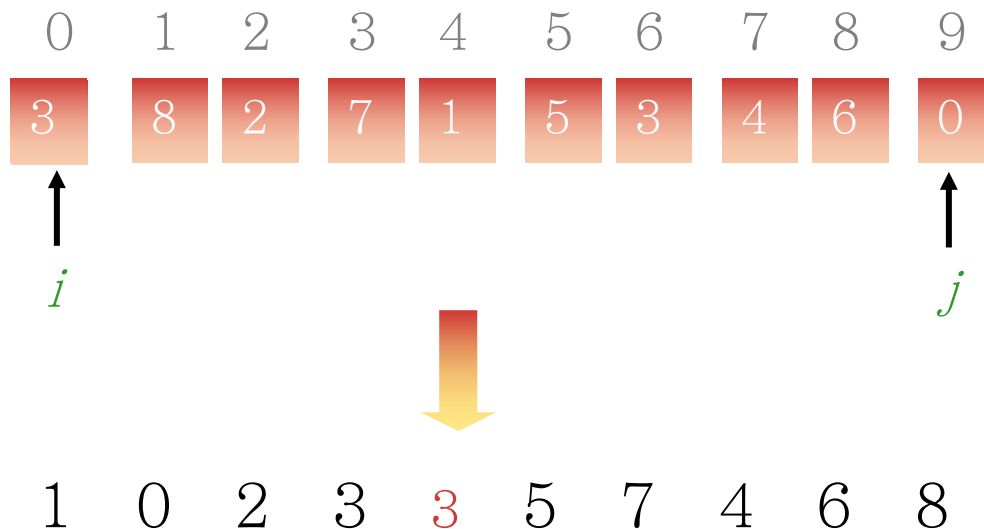
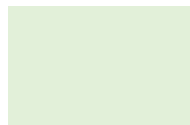
解法一（前后交换法）

- $base = L \rightarrow data[0]$ （基准）
- j 从后向前找 $\leq base$ 的元素
- i 从前向后找 $> base$ 的元素



两者交换

base



2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

```
void move1(SqList *&L)
{ int i=0, j=L->length-1;
  ElemType base=L->data[0]; //以data[0]为基准
  while (i<j)
  { while (i<j && L->data[j]>base)
      j--; //从后向前扫描, 找一个 $\leq$ base的元素
    while (i<j && L->data[i] $\leq$ base)
      i++; //从前向后扫描, 找一个 $>$ base的元素
    if (i<j)
      swap(L->data[i],L->data[j]);
  }
  swap(L->data[0],L->data[i]);
}
```

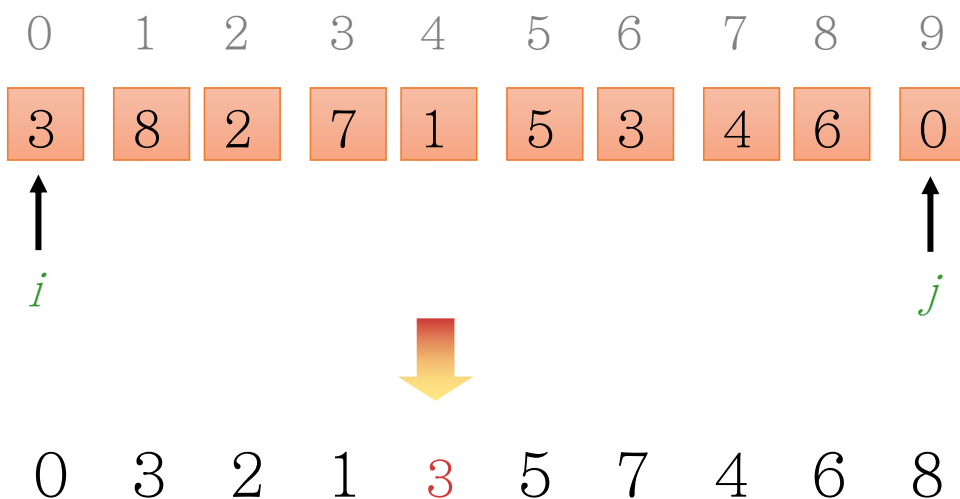
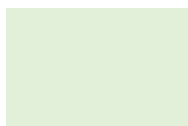
2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

解法二（前后移动法）

- $\text{base} = \text{L} \rightarrow \text{data}[0]$ (基准)
- j 从后向前找小于等于 base 的元素：前移
- i 从前向后找大于 base 的元素：后移

base (基准)



算法时间复杂度为 $O(n)$ 。

2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

```
void move2(SqList *&L)
{ int i=0, j=L->length-1;
  ElemType base=L->data[0];    //以data[0]为基准
  while (i<j)
  { while (j>i && L->data[j]>base)
      j--;                      //从右向左, 找 $\leq$ base的data[j]
    L->data[i]=L->data[j];       //将其放入data[i]处
    while (i<j && L->data[i]<=base)
      i++;                      //从左向右, 找 $>$ base的记录data[i]
    L->data[j]=L->data[i];       //将其放入data[j]处
  }
  L->data[i]=base;              //放置基准
}
```

2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

【例2.5】有一个顺序表 L ，假设元素类型

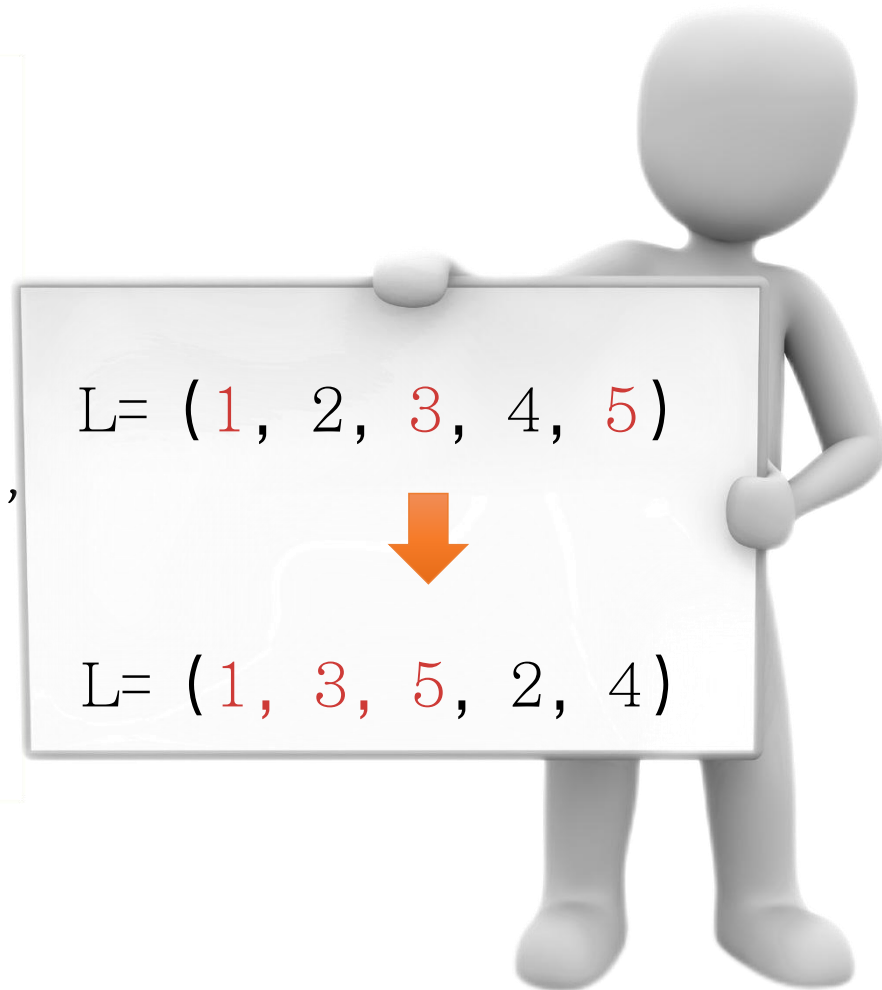
ElemType为整型。设计一个尽可能高效的算法，

将所有奇数移动到偶数的前面。

$L = (1, 2, 3, 4, 5)$



$L = (1, 3, 5, 2, 4)$



2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

解

解法一（前后交换法）

- i （初值为0）从左向右扫描， j （初值为 $L \rightarrow \text{length}-1$ ）从右向左扫描。
- 当 $i \neq j$ 时循环： j 从右向左找一个奇数元素 $\text{data}[j]$ ， i 从左向右找一个偶数 $\text{data}[i]$ ，然后在 $i < j$ 时将两者进行交换。

2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

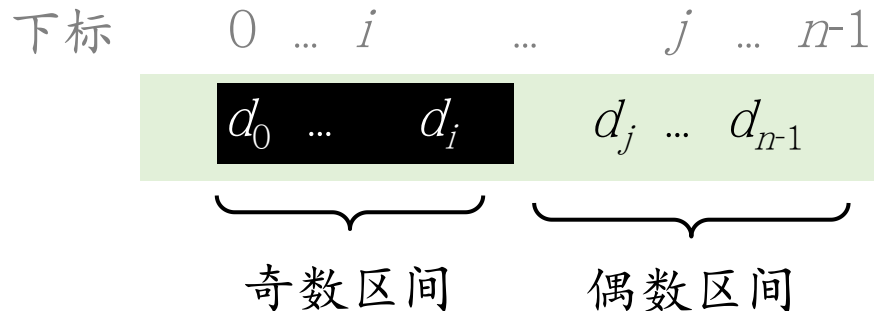
```
void move1(SqList *&L)
{ int i=0,j=L->length-1;
  while (i<j)
  { while (i<j && L->data[j]%2==0)
      j--; //从右向左,找一个奇数元素
    while (i<j && L->data[i]%2==1)
      i++; //从左向右,找一个偶数元素
    if (i<j) //若i<j, L->data[i]和L->data[j]交换
      swap(L->data[i],L->data[j]);
  }
}
```

2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

解法二（区间划分法）

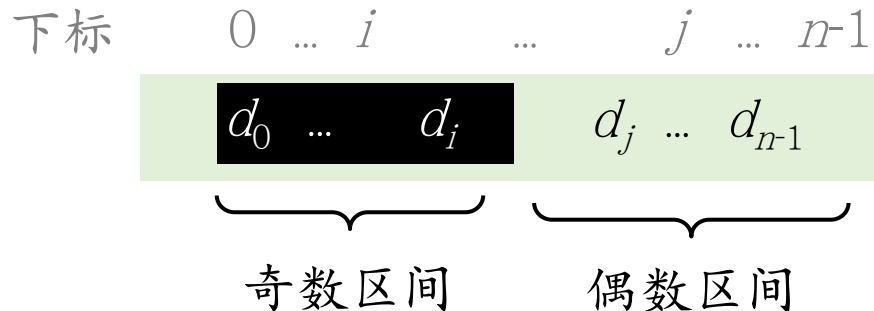
- 用 $L \rightarrow \text{data}[0..i]$ 表示存放奇数的奇数区间（ i 指向奇数区间中的最后元素），初始时 i 为 -1 表示奇数区间为空。
- j 从左向右扫描所有元素，如果 j 指向的元素是奇数，让 i 增 1 表示奇数区间多了一个奇数，然后将 $L \rightarrow \text{data}[j]$ 和 $L \rightarrow \text{data}[i]$ 交换， j 继续扫描。



2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

```
void move2(SqList *&L)
{
    int i=-1,j;
    for (j=0;j<=L->length-1;j++)
        if (L->data[j]%2==1)           //j指向奇数时
        {
            i++;                         //奇数区间个数增1
            if (i!=j)                   //若i不为j, L->data[i]和L->data[j]交换
                swap(L->data[i],L->data[j]);
        }
}
```



2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

【例2.3】 已知长度为 n 的线性表 L 采用顺序存储结构。设计一个时间复杂度为 $O(n)$ 、空间复杂度为 $O(1)$ 的算法，该算法删除线性表中所有值为 x 的数据元素。

解

2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

解法三（区间划分法）

$\text{data}[0 \dots i]$

不为 x 的区间 为 x 的区间



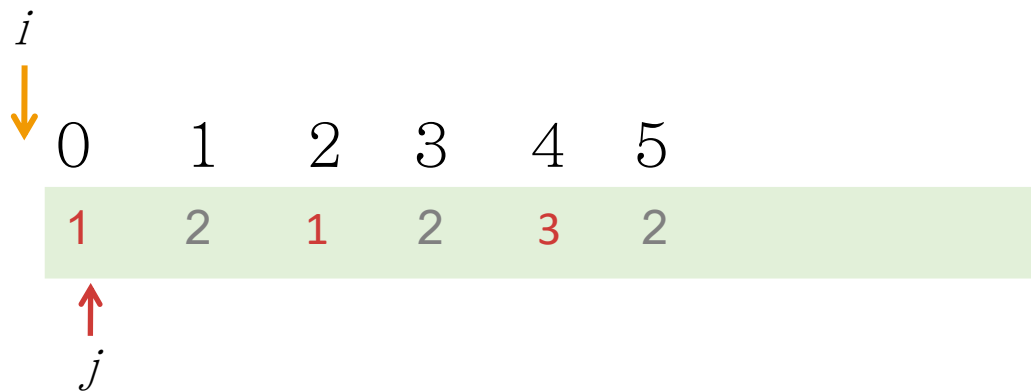
- $\text{data}[j] = x$: $j++$
- $\text{data}[j] \neq x$, i 增加 1 扩大“不为 x 的区间”，将 $\text{data}[i]$ 与 $\text{data}[j]$ 交换



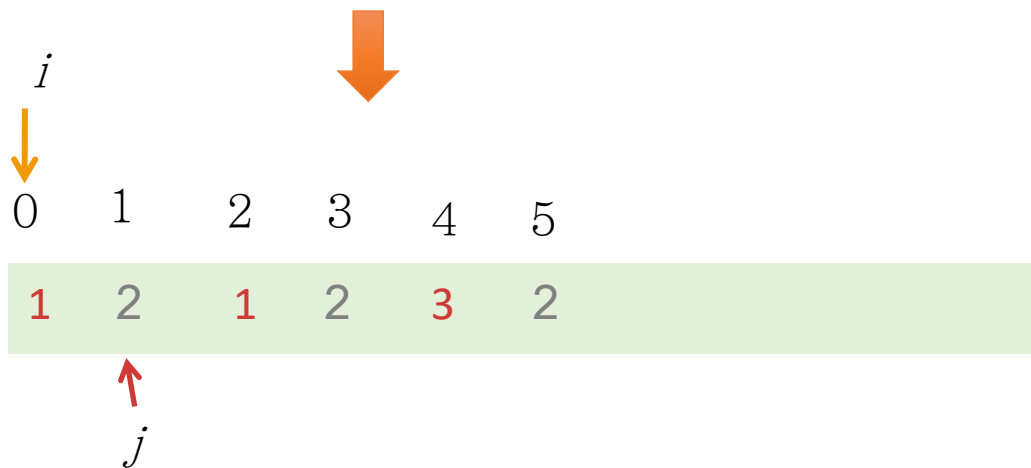
2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

删除所有 $x=2$ 的元素，初始时: $i=-1$, $j=0$

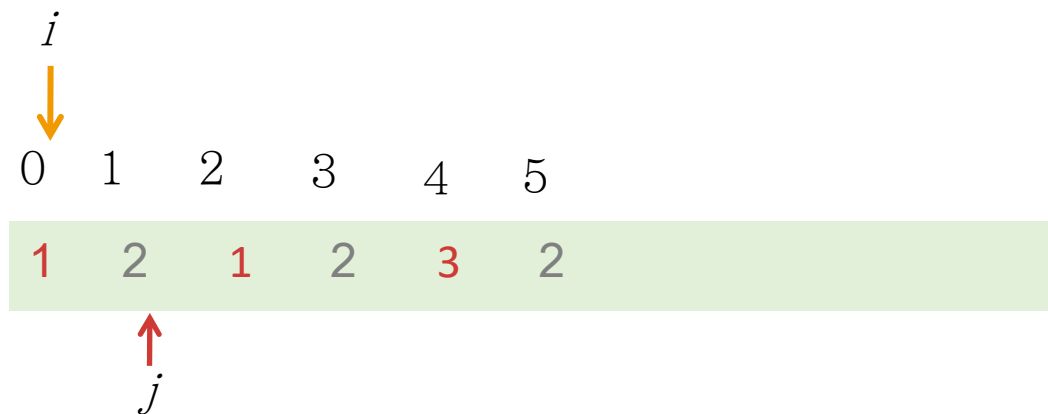


$j=0, \text{data}[0] \neq x$: $i++ \Rightarrow i=0$, $j++ \Rightarrow j=1$

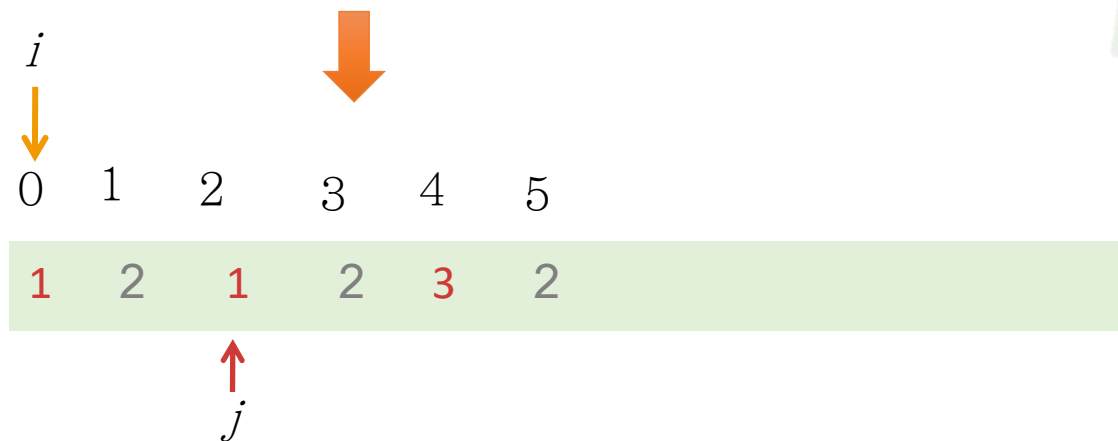


2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

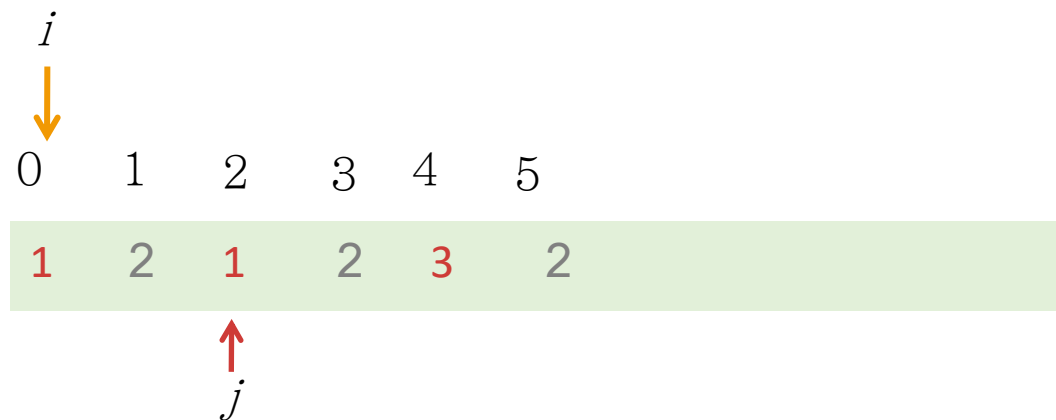


$j=1, \text{data}[1]=x: j++ \Rightarrow j=2$

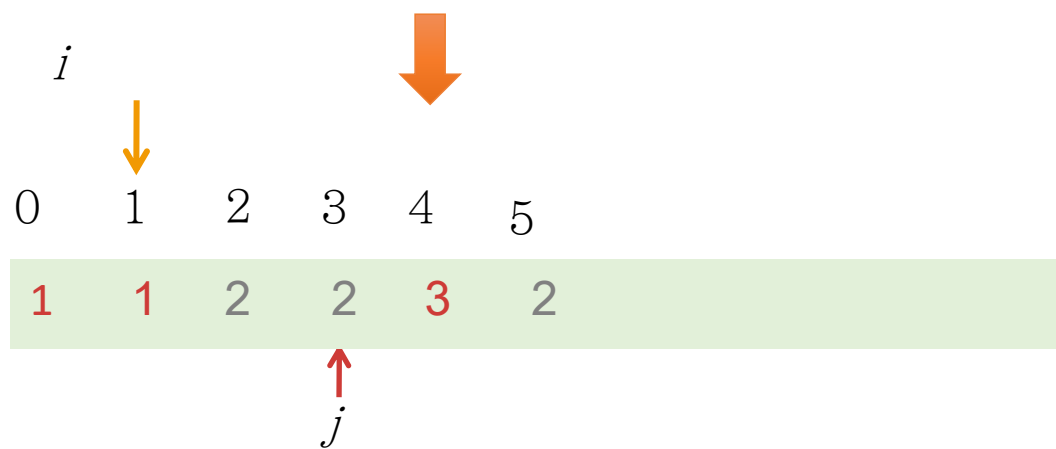


2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

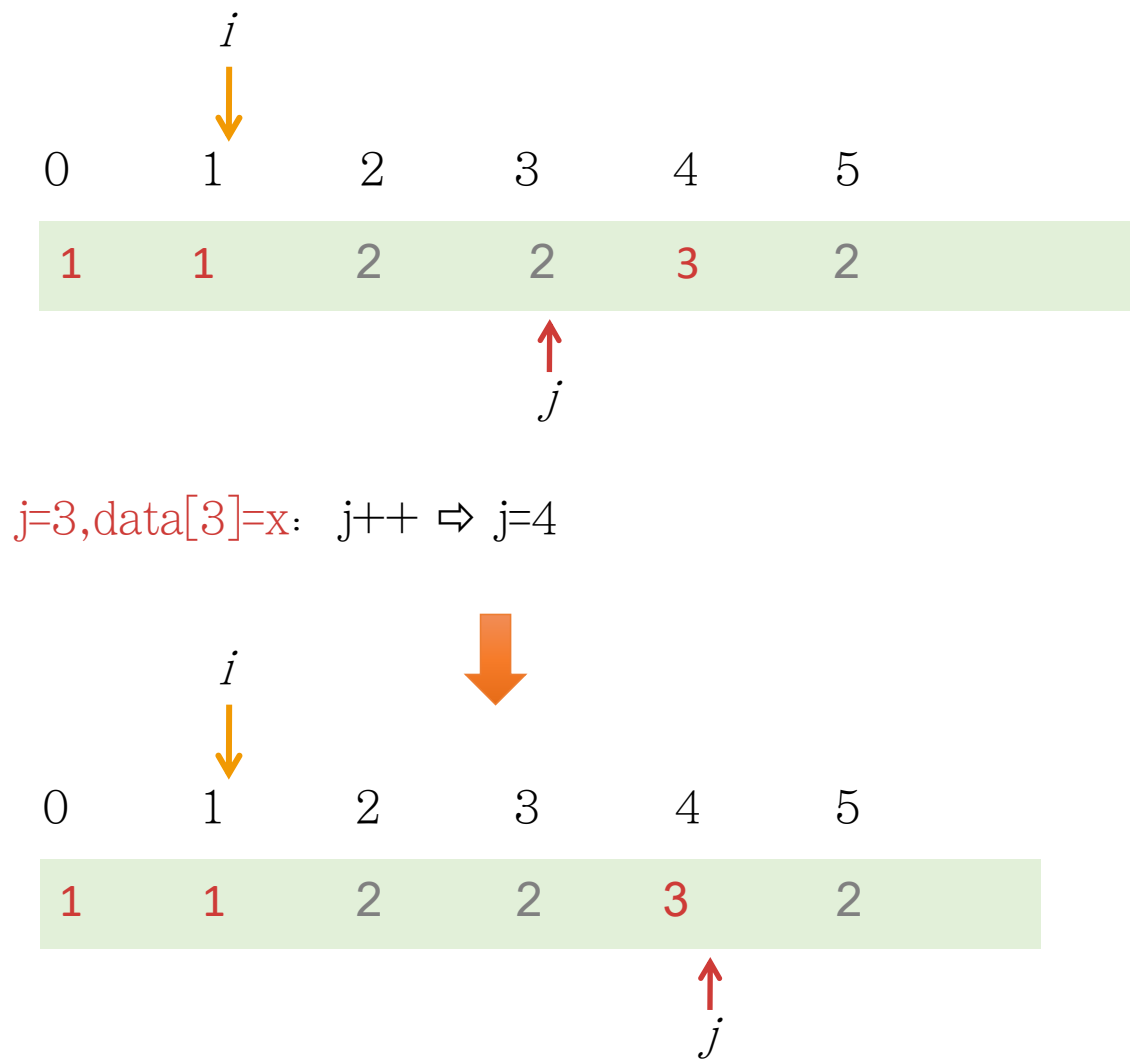


$j=2, \text{data}[2] \neq x$: $i++ \Rightarrow i=1$, $\text{swap}(\text{data}[2], \text{data}[1])$, $j++ \Rightarrow j=3$



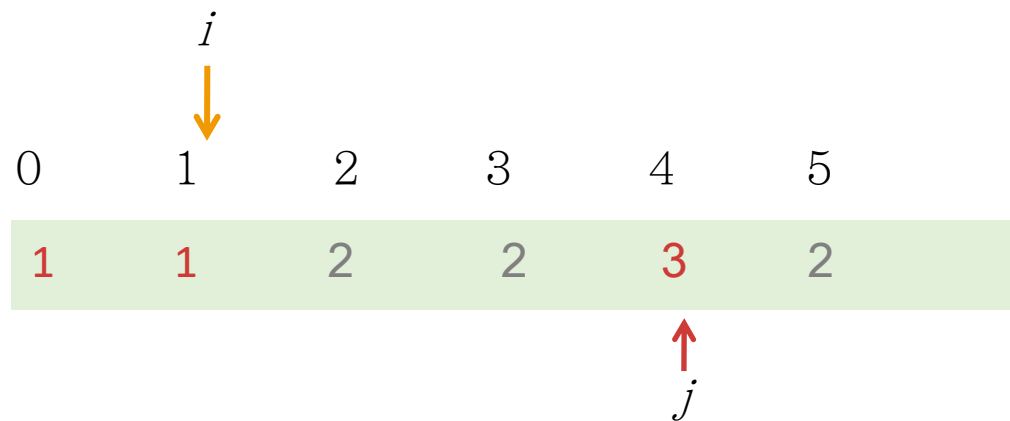
2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

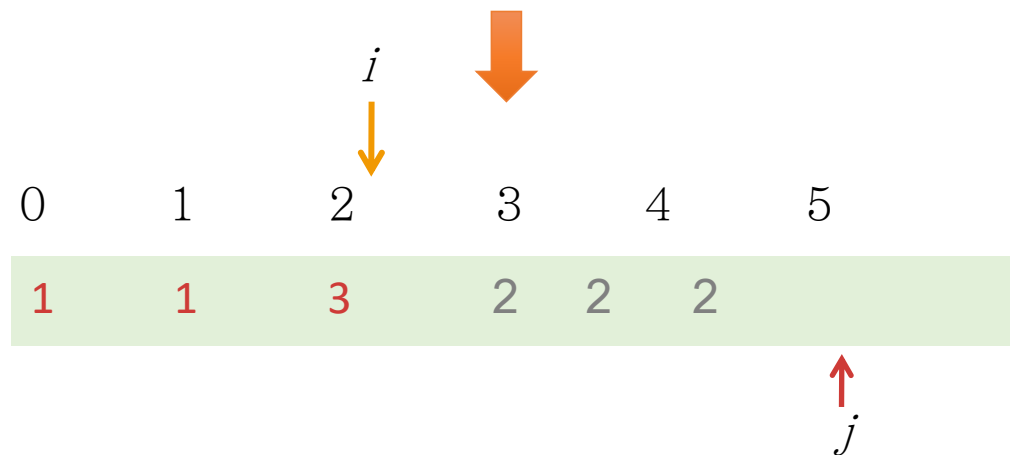


2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

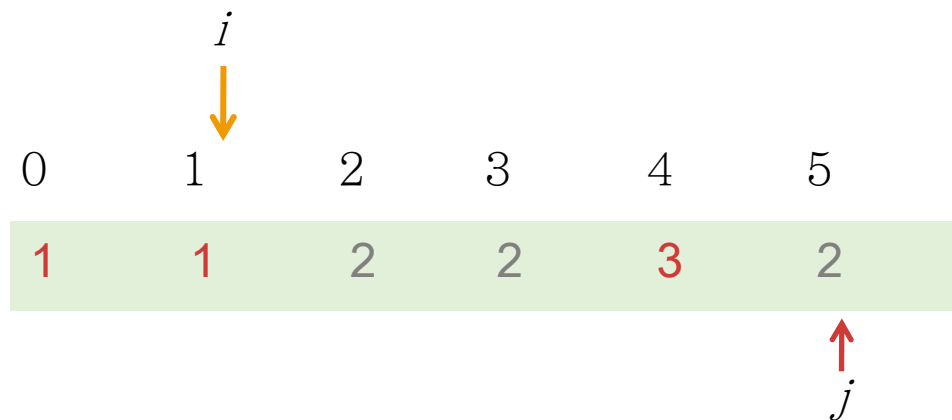


$j=4, \text{data}[4] \neq x$: $i++ \Rightarrow i=2$, $\text{swap}(\text{data}[2], \text{data}[4])$, $j++ \Rightarrow j=5$

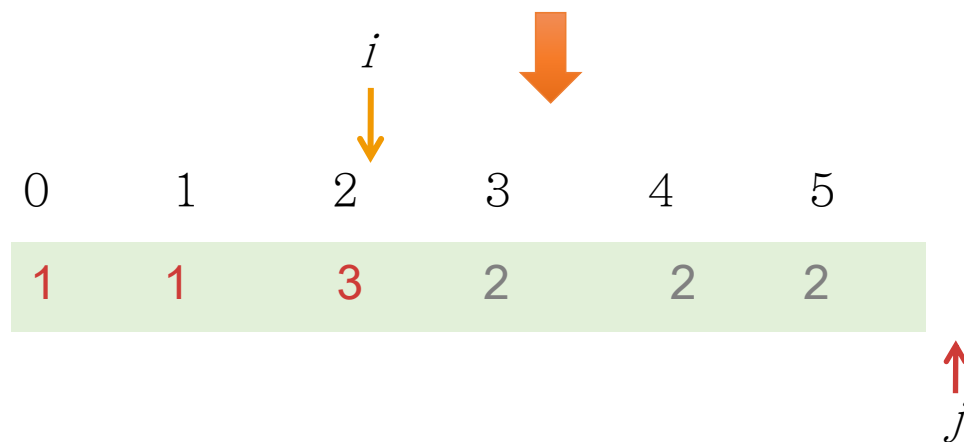


2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现



$j=5, \text{data}[5]=x: j++ \Rightarrow j=6$



删除完成

$\text{length}=i+1=3$

2.2 线性表的顺序存储结构

2.2.2 顺序表运算的实现

```
void delnode3(SqList *&L, ElemType x)
{
    int i=-1, j=0;
    while (j<L->length)    //j扫描所有元素
    {
        if (L->data[j]!=x)    //找到不为x的元素
        {
            i++;                //扩大不为x的区间
            if (i!=j)
                swap(L->data[i], L->data[j]);
        }
        j++;                //继续扫描
    }
    L->length=i+1;            //设置L中实际元素个数
}
```

第2章 线性表

提纲

CONTENTS

2.1 线性表及其逻辑结构

2.2 线性表的顺序存储结构



2.3 线性表的链式存储结构

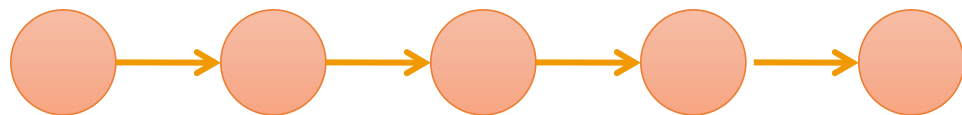
2.4 线性表的应用

2.5 有序表

2.3 线性表的链式存储结构

2.3.1 线性表的链式存储—链表

线性表中每个结点有**唯一**的前驱结点和前驱结点。

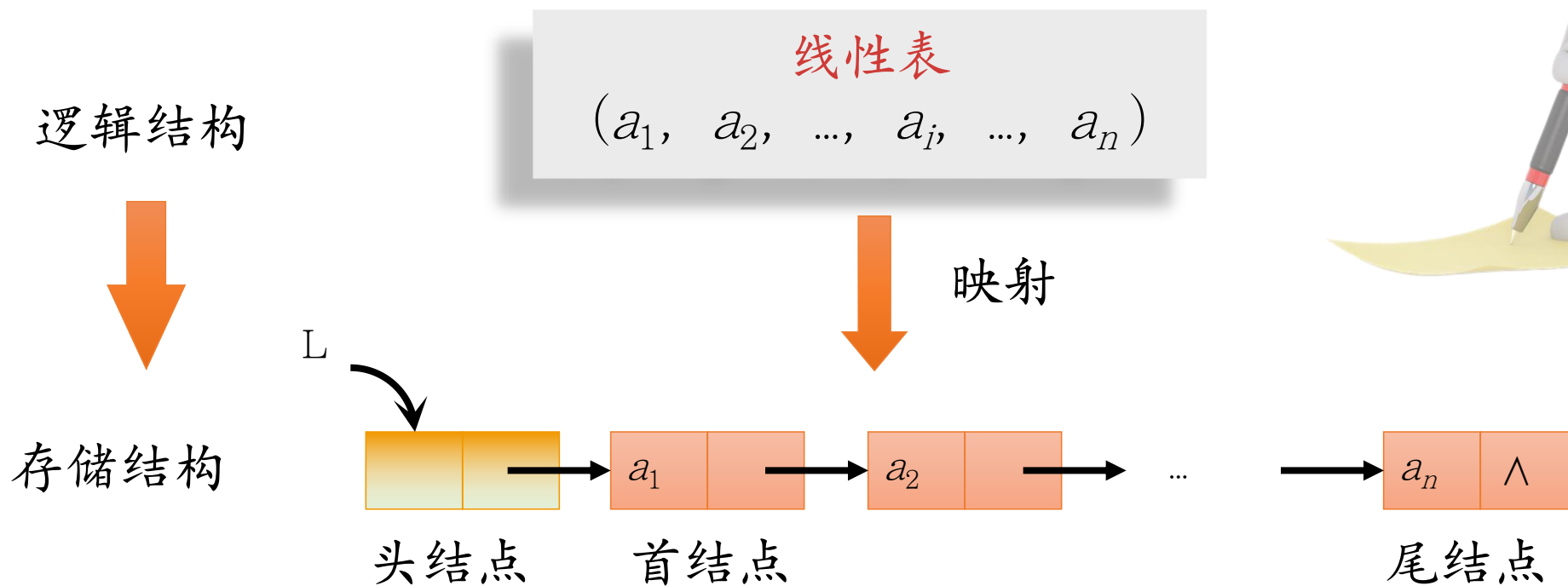


设计链式存储结构时，每个逻辑结点单独存储，为了表示逻辑关系，增加**指针域**。

- 每个物理结点增加一个指向后继结点的指针域 ⇨ **单链表**。
- 每个物理结点增加一个指向后继结点的指针域和一个指向前驱结点的指针域 ⇨ **双链表**。

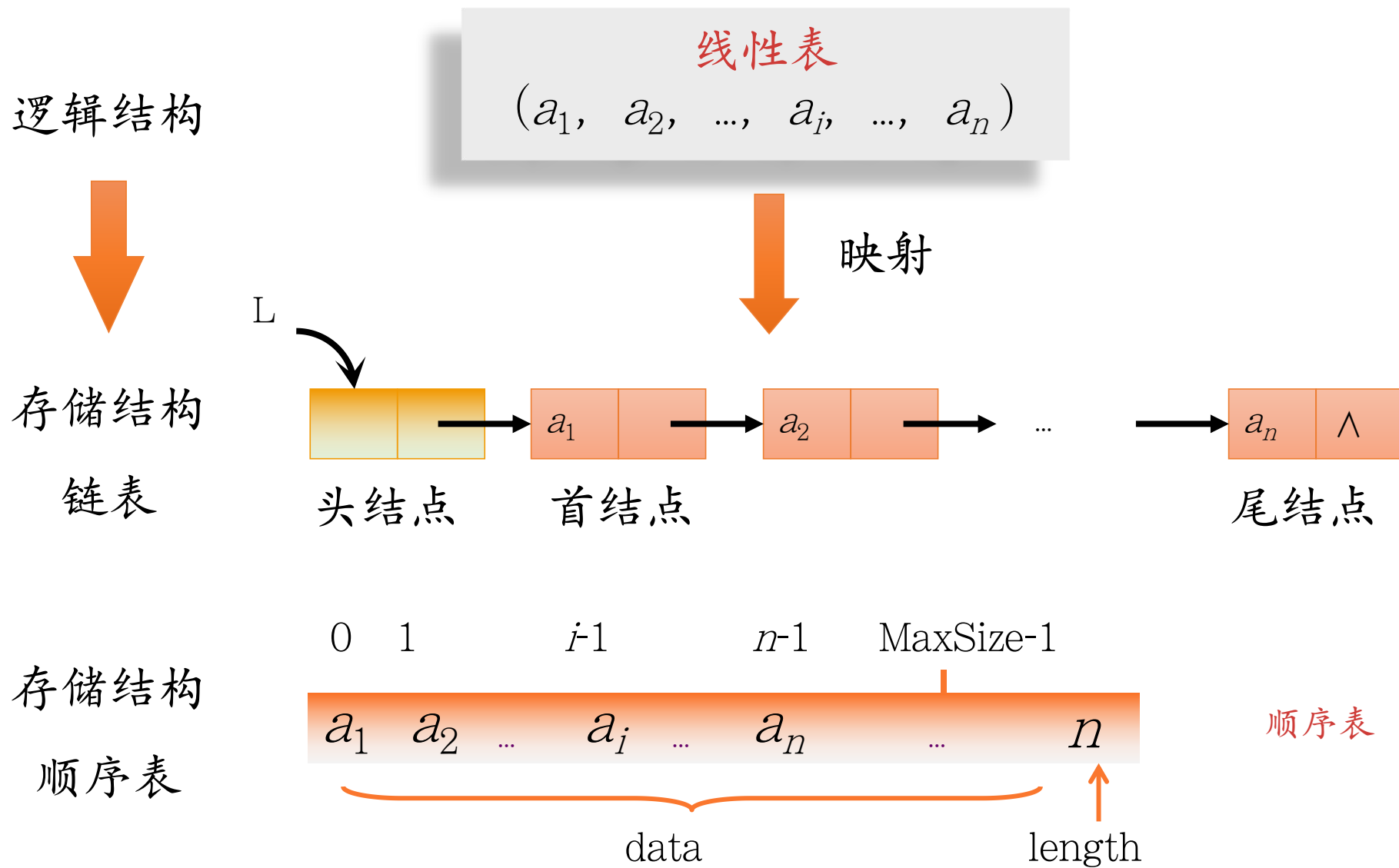
2.3 线性表的链式存储结构

2.3.1 线性表的链式存储—链表



带头结点单链表示意图

2.3 线性表的链式存储结构



2.3 线性表的链式存储结构

1. 数据采用链式存储结构时，要求 () A

A. 每个结点占用一片连续的存储区域

B. 所有结点占用一片连续的存储区域 ← 顺序表

C. 结点的最后一个域必须是指针域

D. 每个结点有多少后继结点，就必须设多少个指针域

一个结点设多少个指针域，取决于存储结构类型。

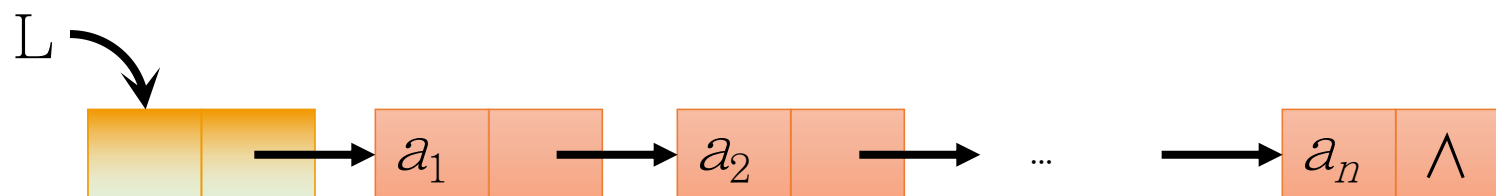
比如：

- 单链表：1个指针域
- 双链表：2个指针域
- 树、图的链式存储也有不同设计

2.3 线性表的链式存储结构

2.3.1 线性表的链式存储—链表

带头结点单链表



单链表增加一个头结点的优点如下：

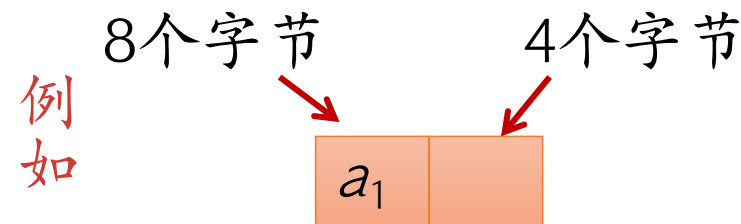
- 首结点的操作和表中其他结点的操作相一致，无需进行特殊处理；
- 无论链表是否为空，都有一个头结点，因此空表和非空表的处理也就统一了。

2.3 线性表的链式存储结构

2.3.1 线性表的链式存储—链表

存储密度是指结点数据本身所占的存储量和整个结点结构中所占的存储量之比，即：

$$\text{存储密度} = \frac{\text{结点数据本身占用的空间}}{\text{结点占用的空间总量}}$$



$$\text{存储密度} = 8/12 = 67\%$$

一般地，存储密度越大，存储空间的利用率就越高。显然，顺序表的存储密度为1（100%），而链表的存储密度小于1。

2.3 线性表的链式存储结构

2.3.2 单链表

单链表中结点类型LinkNode的定义如下:

```
typedef struct LNode    //定义单链表结点类型
{
    ElemType data;      //存储元素的数据域
    struct LNode *next; //指向后继结点
} LinkNode;
```



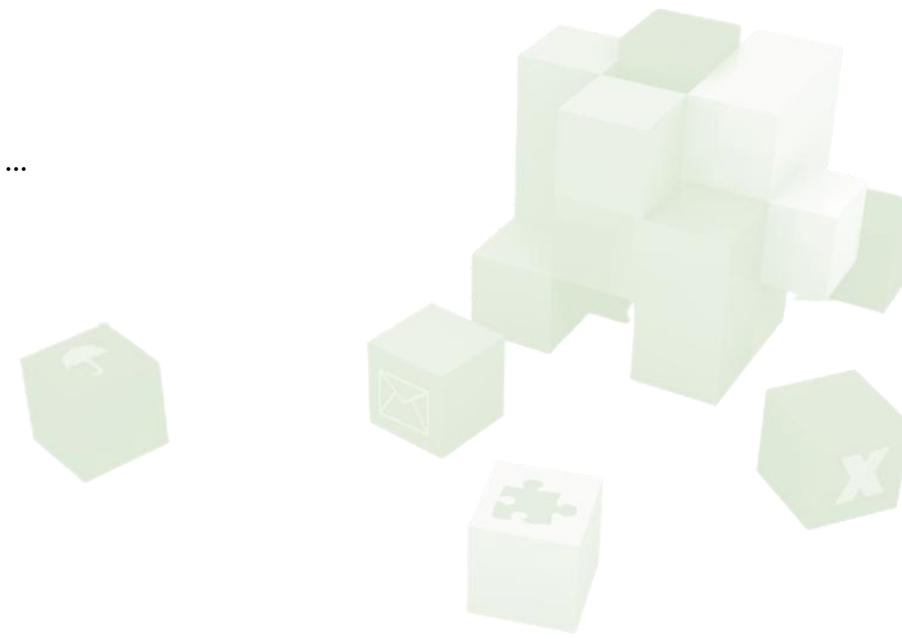
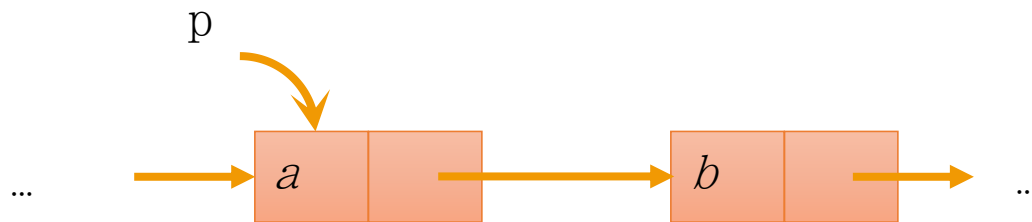
- 链表为每个结点都定义一个类型
- 头结点的类型与元素结点类型可以不一致
- 元素结点类型需要一致 (一致性)

2.3 线性表的链式存储结构

2.3.2 单链表

➤ 单链表的特点

当访问过一个结点后，只能接着访问它的后继结点，而无法访问它的前驱结点。



2.3 线性表的链式存储结构

2.3.2 单链表

1、插入结点和删除结点操作

(1) 插入结点

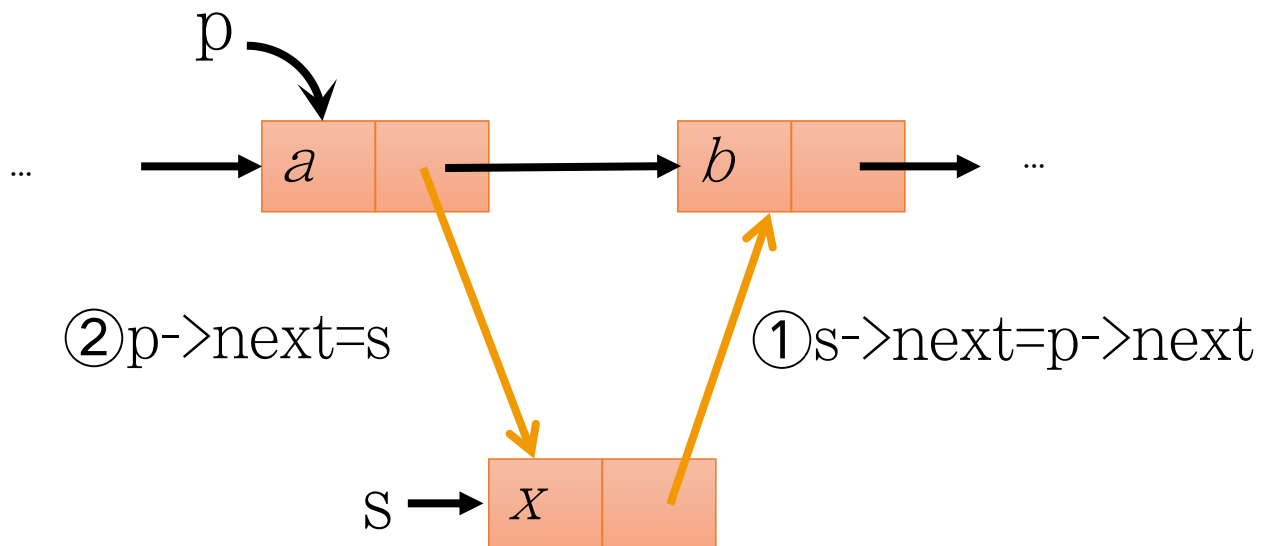
插入操作： 将值为 x 的新结点 s 插入到 p 结点之后。

特点： 只需修改相关结点的指针域，不需要移动结点。

2.3 线性表的链式存储结构

2.3.2 单链表

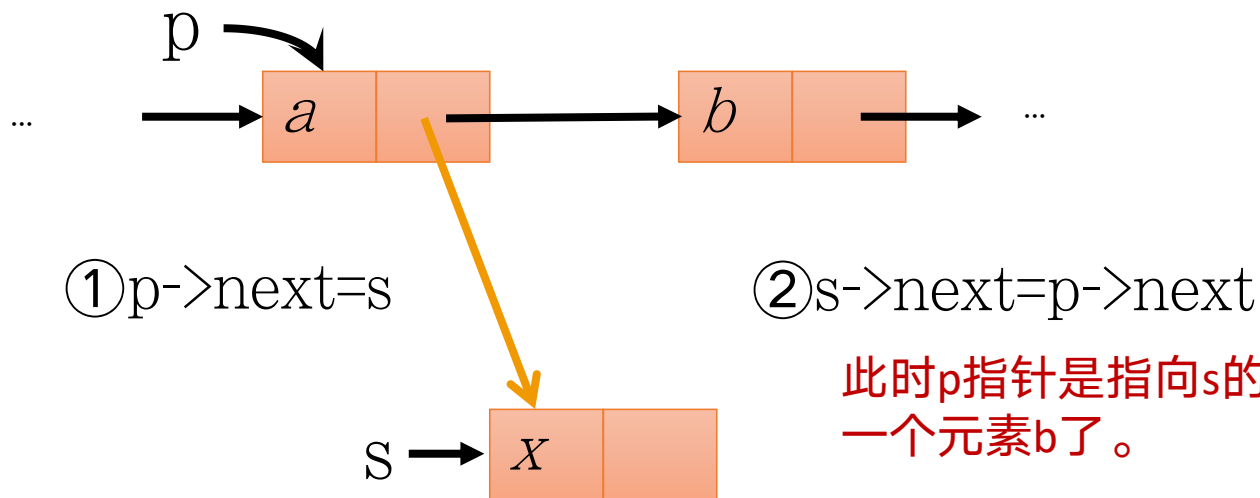
单链表插入结点演示



插入操作语句描述如下:

```
s->next = p->next;
```

```
p->next = s;
```



思考：如果先执行了2再执行会怎么样？

此时p指针是指向s的，所以s找不到下一个元素b了。

2.3 线性表的链式存储结构

2.3.2 单链表

1、插入结点和删除结点操作

(2) 删除结点

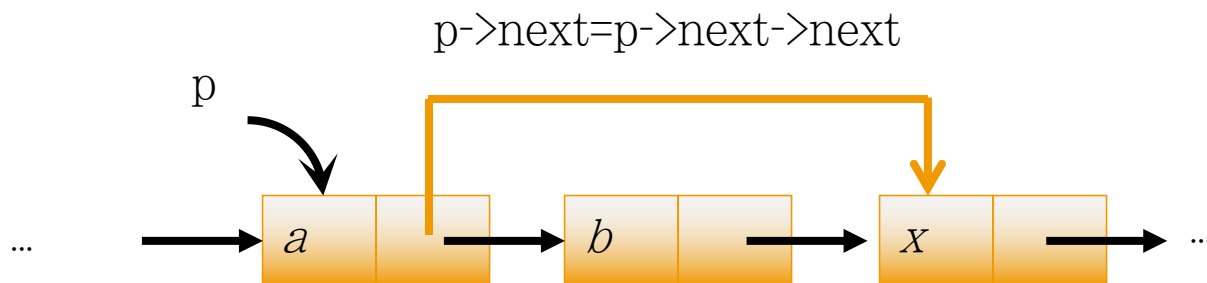
删除操作： 删除p结点之后的一个结点。

特点： 只需修改相关结点的指针域，不需要移动结点。

2.3 线性表的链式存储结构

2.3.2 单链表

单链表删除结点演示



释放被删除的节点:

```
q=p->next;
```

```
p -> next = q->next;
```

```
free(q)
```

删除操作语句描述如下:

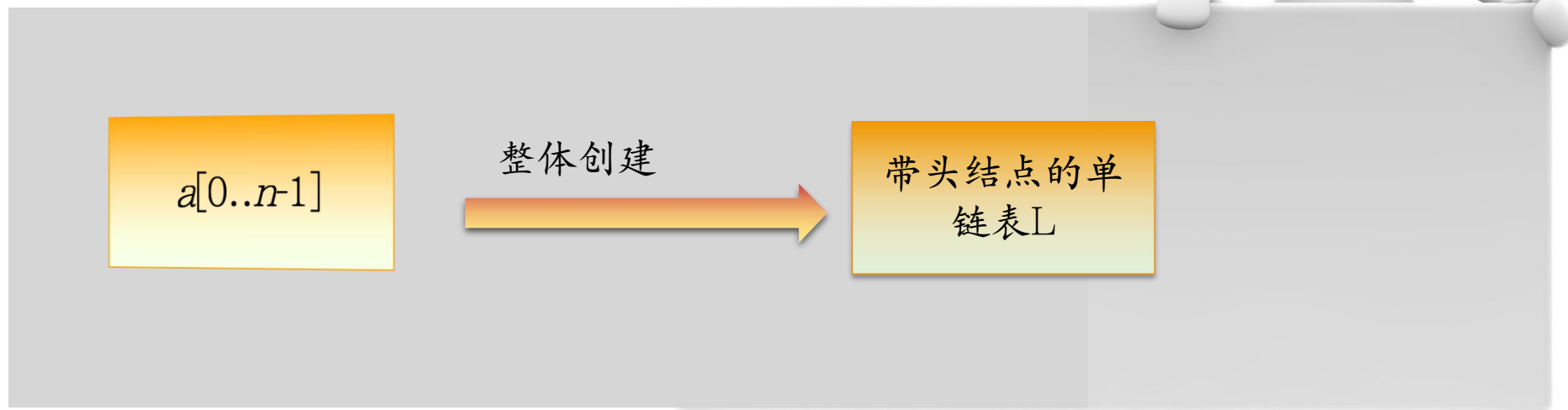
```
p->next = p->next->next;
```

2.3 线性表的链式存储结构

2.3.2 单链表

2、建立单链表

先考虑如何整体建立单链表。



建立单链表的常用方法有两种。

2.3 线性表的链式存储结构

2.3.2 单链表

2、建立单链表

(1) 头插法建表

- 从一个空表开始，创建一个头结点。
- 依次读取字符数组 a 中的元素，生成新结点
- 将新结点插入到当前链表的表头上，直到结束为止。

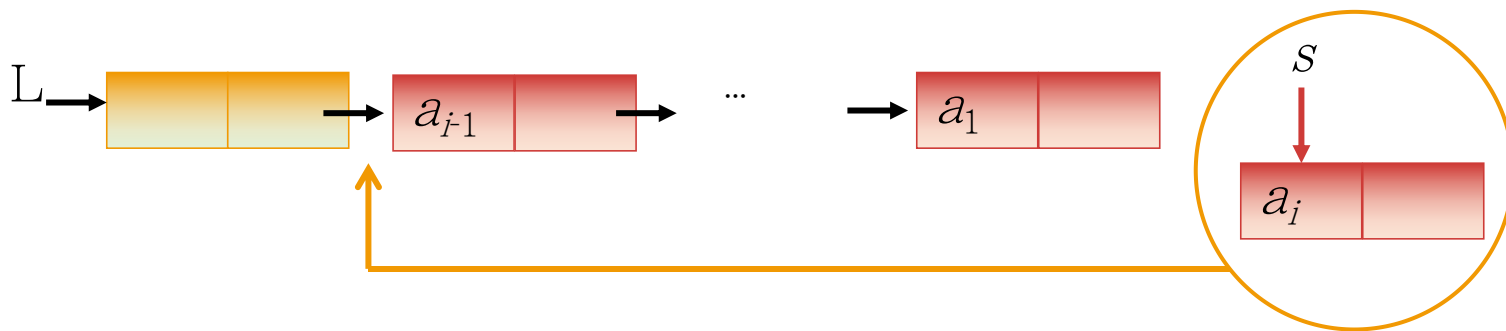
线性表：(a_1, a_2, a_3, a_4)

插入 a_1 : $L \rightarrow a_1$

插入 a_2 : $L \rightarrow a_2 \rightarrow a_1$

插入 a_3 : $L \rightarrow a_3 \rightarrow a_2 \rightarrow a_1$

插入 a_4 : $L \rightarrow a_4 \rightarrow a_3 \rightarrow a_2 \rightarrow a_1$



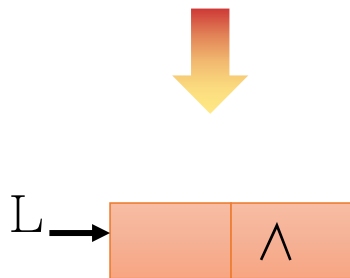
注意：链表的结点顺序与逻辑次序相反。

2.3 线性表的链式存储结构

2.3.2 单链表

头插法建表算法如下:

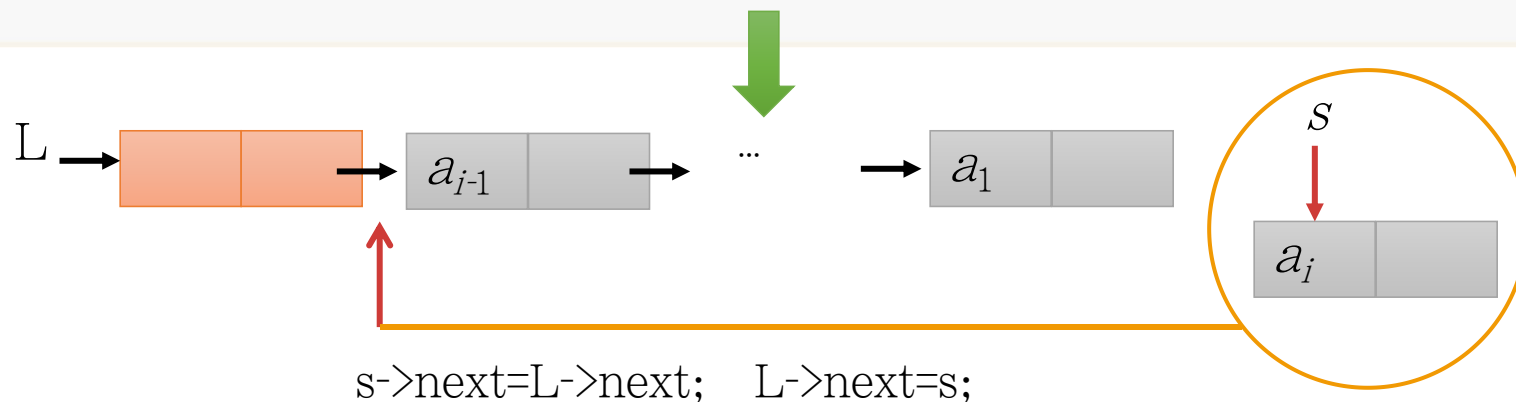
```
void CreateListF(LinkNode *&L, ElemType a[], int n)
{ LinkNode *s;
  int i;
  L=(LinkNode *)malloc(sizeof(LinkNode));
  L->next=NULL;           //创建头结点, 其next域置为NULL
```



2.3 线性表的链式存储结构

2.3.2 单链表

```
for (i=0;i<n;i++)           //循环建立数据结点
{
    s=(LinkNode *)malloc(sizeof(LinkNode));
    s->data=a[i];           //创建数据结点s
    s->next=L->next;        //将s插在原开始结点之前，头结点之后
    L->next=s;
}
```



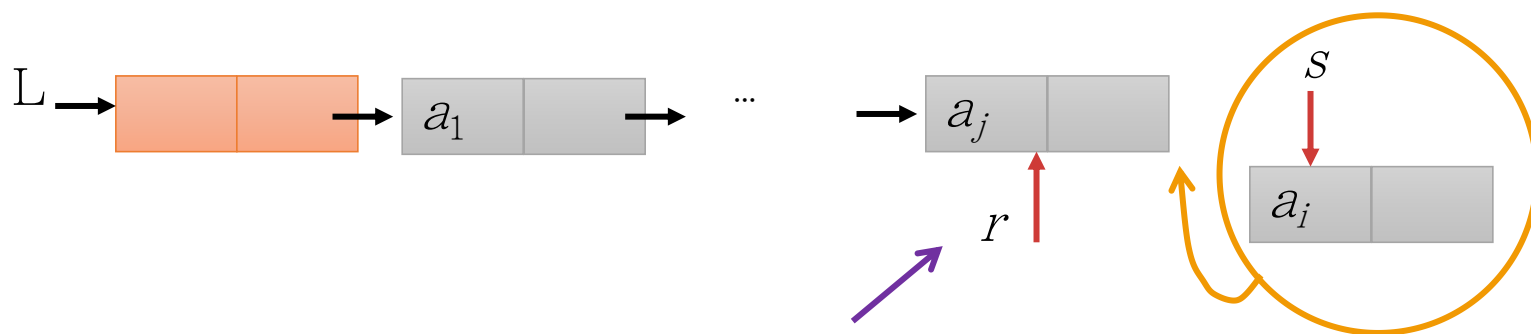
2.3 线性表的链式存储结构

2.3.2 单链表

2、建立单链表

(2) 尾插法建表

- 从一个空表开始，创建一个头结点。
- 依次读取字符数组 a 中的元素，生成新结点
- 将新结点插入到当前链表的表尾上，直到结束为止。



增加一个尾指针 r ，使其始终指向当前链表的尾结点

线性表：(a_1, a_2, a_3, a_4)

创建头结点 L

插入 a_1 : $L \rightarrow a_1$

插入 a_2 : $L \rightarrow a_1 \rightarrow a_2$

插入 a_3 : $L \rightarrow a_1 \rightarrow a_2 \rightarrow a_3$

插入 a_4 : $L \rightarrow a_1 \rightarrow a_2 \rightarrow a_3 \rightarrow a_4$

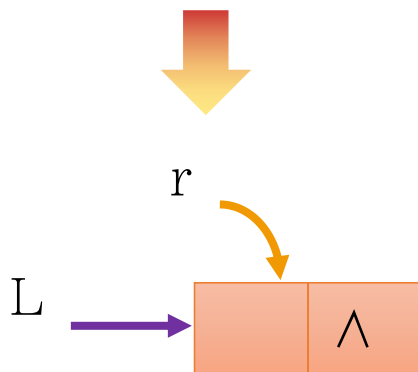
注意：链表的结点顺序与逻辑次序相同。

2.3 线性表的链式存储结构

2.3.2 单链表

尾插法建表算法如下:

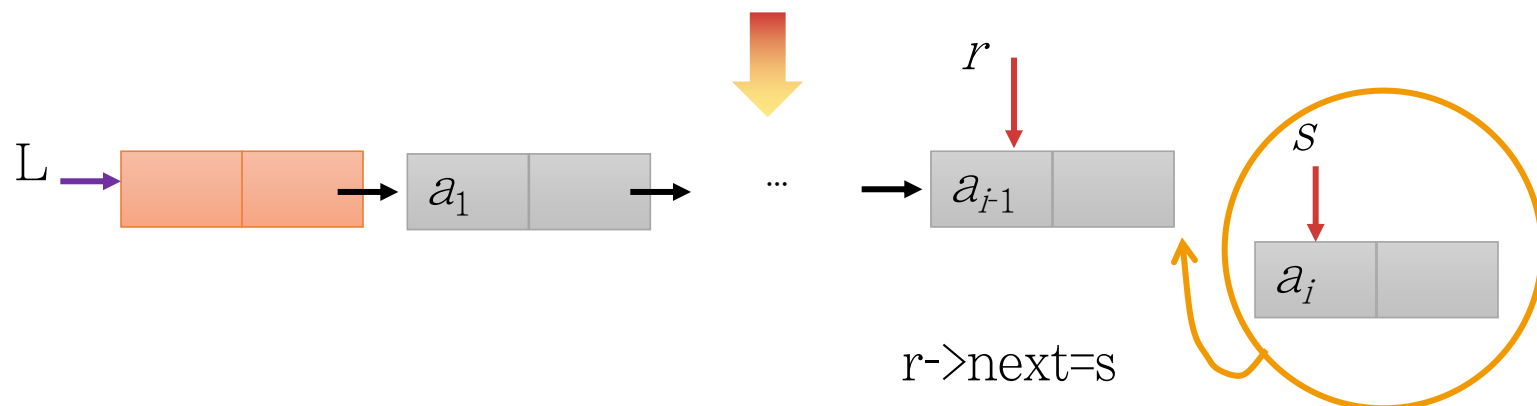
```
void CreateListR(LinkNode *&L, ElemType a[], int n)
{ LinkNode *s, *r;
  int i;
  L=(LinkNode *)malloc(sizeof(LinkNode)); //创建头结点
  r=L;                                     //r始终指向尾结点, 开始时指向头结点
```



2.3 线性表的链式存储结构

2.3.2 单链表

```
for (i=0;i<n;i++)           //循环建立数据结点
{
    s=(LinkNode *)malloc(sizeof(LinkNode));
    s->data=a[i];           //创建数据结点s
    r->next=s;              //将s插入r之后
    r=s;
}
r->next=NULL;              //尾结点next域置为NULL
}
```



2.3 线性表的链式存储结构

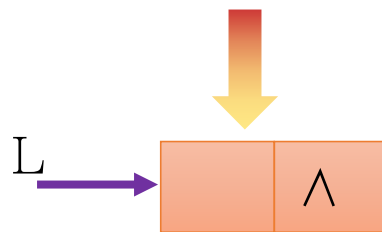
2.3.2 单链表

3、线性表基本运算在单链表上的实现

(1) 初始化线性表 InitList(L)

建立一个空的单链表，即创建一个头结点。

```
void InitList(LinkNode *&L)
{
    L=(LinkNode *)malloc(sizeof(LinkNode));    //创建头结点
    L->next=NULL;
}
```



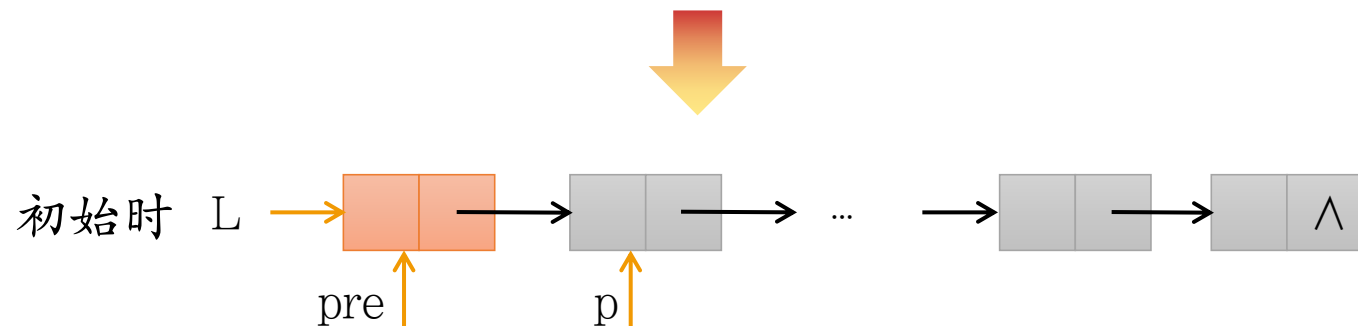
2.3 线性表的链式存储结构

2.3.2 单链表

(2) 销毁线性表DestroyList(L)

释放单链表L占用的内存空间。即逐一释放全部结点的空间。

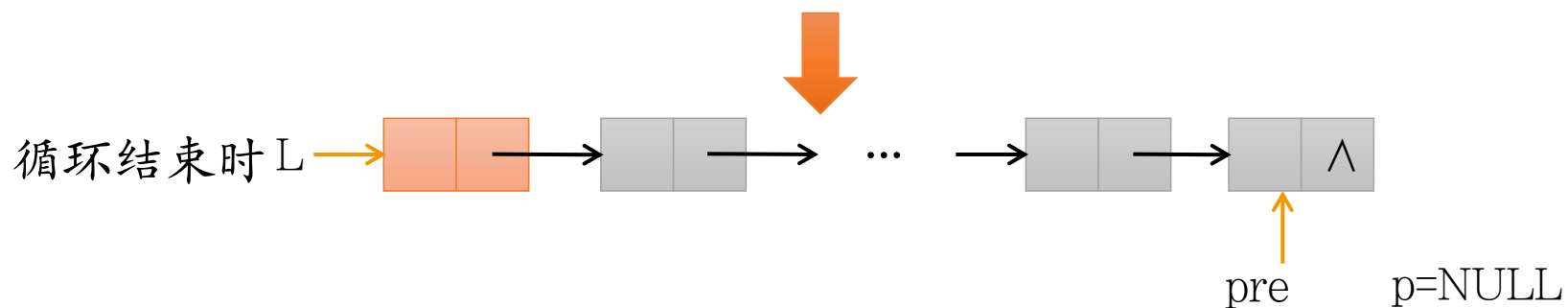
```
void DestroyList(LinkNode *&L)
{
    LinkNode *pre=L, *p=L->next;    //pre指向p的前驱结点
```



2.3 线性表的链式存储结构

2.3.2 单链表

```
while (p!=NULL) //扫描单链表L
{ free(pre);    //释放pre结点
  pre=p;        //pre、p同步后移一个结点
  p=pre->next;
}
free(pre);      //循环结束时，p为NULL，pre指向尾结点，释放它
}
```



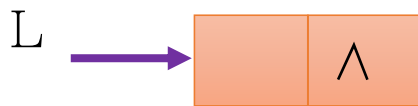
2.3 线性表的链式存储结构

2.3.2 单链表

(3) 判线性表是否为空表 `ListEmpty(L)`

若单链表L没有数据结点，则返回真，否则返回假。

```
bool ListEmpty(LinkNode *L)
{
    return(L->next==NULL);
}
```



空表的情况

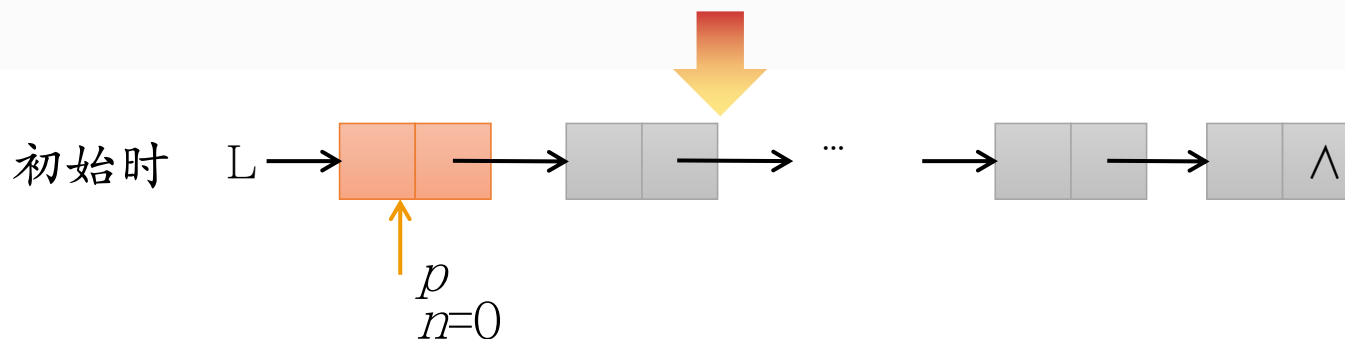
2.3 线性表的链式存储结构

2.3.2 单链表

(4) 求线性表的长度ListLength(L)

返回单链表L中数据结点的个数。

```
int ListLength(LinkNode *L)
{
    int n=0;
    LinkNode *p=L;    //p指向头结点, n置为0 (即头结点的序号为0)
```

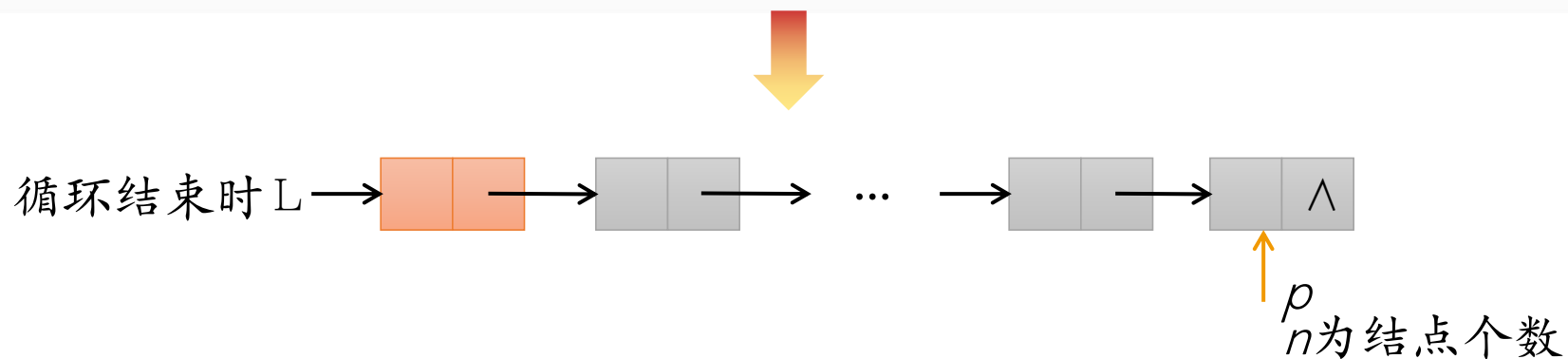


2.3 线性表的链式存储结构

2.3.2 单链表

```
while (p->next!=NULL)
{
    n++;
    p=p->next;
}
return(n);
```

//循环结束，p指向尾结点，其序号n为结点个数



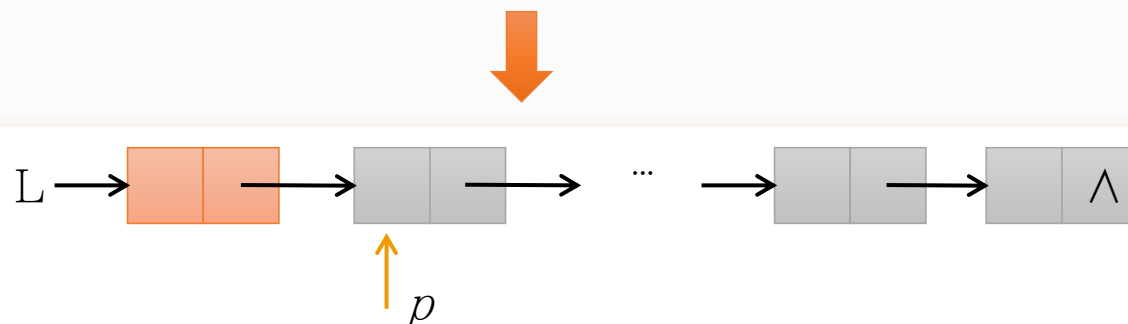
2.3 线性表的链式存储结构

2.3.2 单链表

(5) 输出线性表DispList(L)

逐一扫描单链表L的每个数据结点，并显示各结点的data域值。

```
void DispList(LinkNode *L)
{
    LinkNode *p=L->next;           //p指向开始结点
    while (p!=NULL)                 //p不为NULL，输出p结点的data域
    { printf("%d ", p->data);
      p=p->next;                     //p移向下一个结点
    }
    printf("\n");
}
```



2.3 线性表的链式存储结构

2.3.2 单链表

(6) 求线性表 L 中位置 i 的数据元素GetElem(L , i , $\&e$)

在单链表 L 中从头开始找到第 i 个结点，若存在第 i 个数据结点，则将其data域值赋给变量 e 。



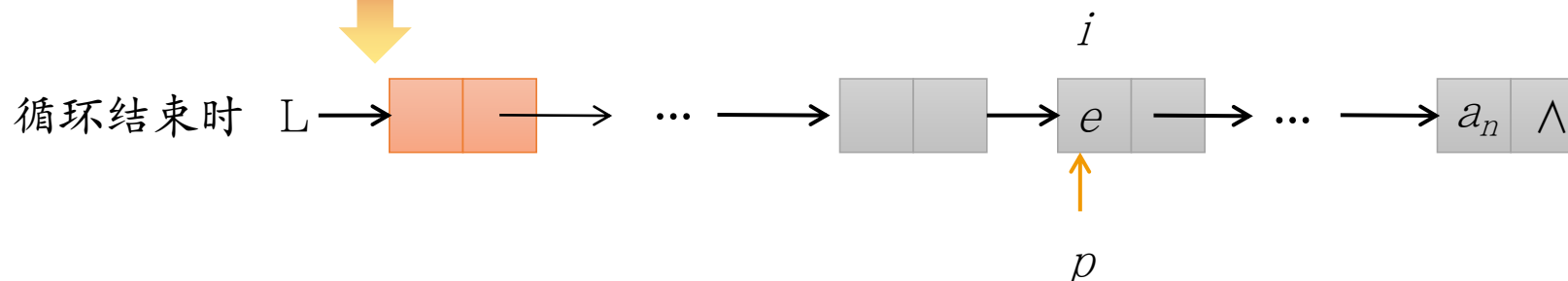
2.3 线性表的链式存储结构

2.3.2 单链表

```
bool GetElem(LinkNode *L, int i, ElemType &e)
{
    int j=0;
    LinkNode *p=L;           //p指向头结点, j置为0 (即头结点的序号为0)

    while (j<i && p!=NULL)
    {
        j++;
        p=p->next;
    }
```

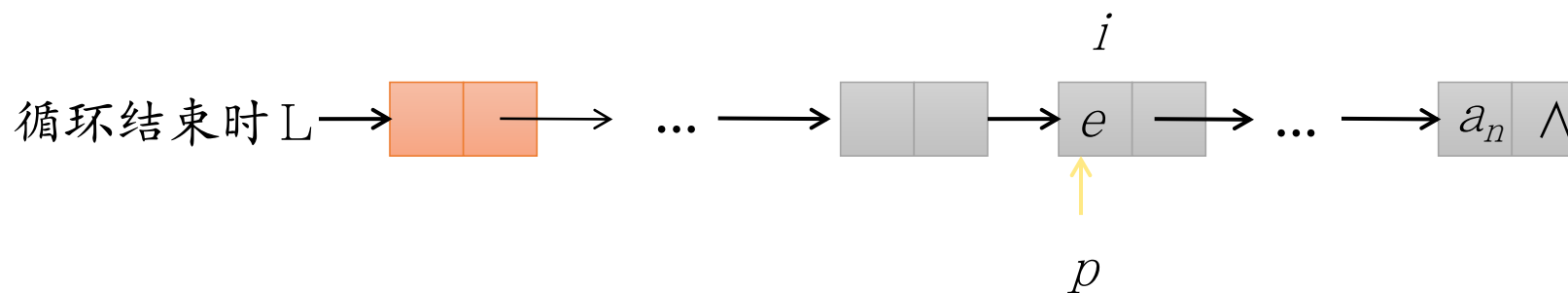
找第 i 个结点 p



2.3 线性表的链式存储结构

2.3.2 单链表

```
if (p==NULL)           //不存在第i个数据结点，返回false
    return false;
else                   //存在第i个数据结点，返回true
{
    e=p->data;
    return true;
}
```



2.3 线性表的链式存储结构

2.3.2 单链表

(7) 按元素值查找LocateElem(L, e)

在单链表L中从头开始找第一个值域与 e 相等的结点，若存在这样的结点，则返回位置，否则返回0。

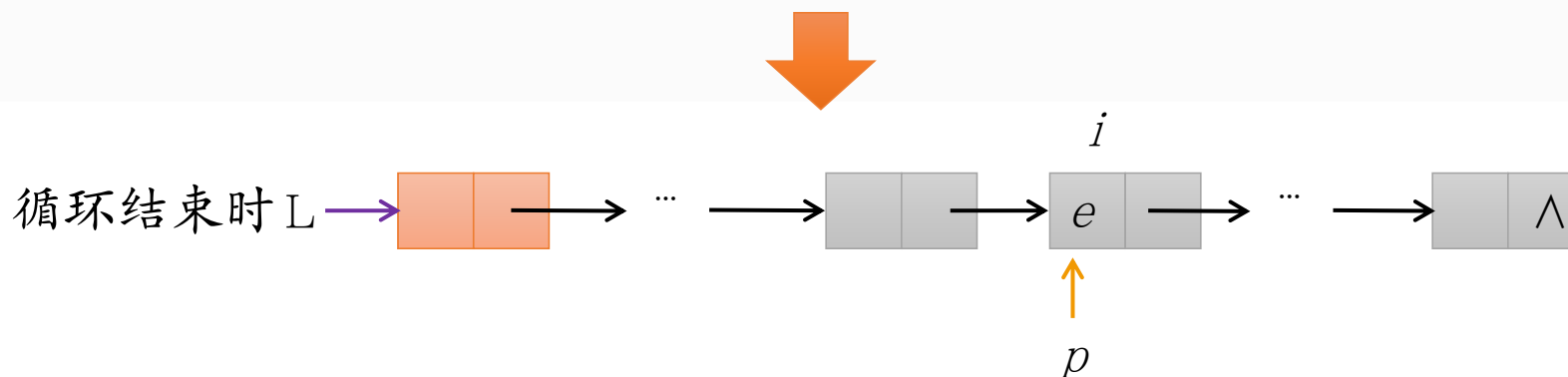


2.3 线性表的链式存储结构

2.3.2 单链表

```
int LocateElem(LinkNode *L, ElemType e)
{
    int i=1;
    LinkNode *p=L->next;    //p指向开始结点, i置为1

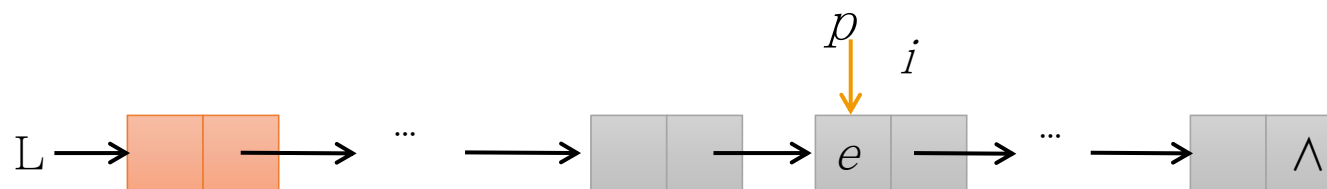
    while (p!=NULL && p->data!=e)
    {
        p=p->next;           //查找data值为e的结点, 其序号为i
        i++;
    }
```



2.3 线性表的链式存储结构

2.3.2 单链表

```
if (p==NULL) //不存在元素值为e的结点, 返回0  
    return 0;  
else  
    //存在元素值为e的结点, 返回其逻辑序号i  
    return i;  
}
```



算法的时间复杂度为 $O(n)$ $\Rightarrow$ 不具有随机存取特性

2.3 线性表的链式存储结构

2.3.2 单链表

(8) 插入数据元素 $\text{ListInsert}(\&L, i, e)$

先在单链表 L 中找到第 $i-1$ 个结点 p ，若存在这样的结点，
将值为 e 的结点 s 插入到其后。



2.3 线性表的链式存储结构

2.3.2 单链表

```
bool ListInsert(LinkNode *&L, int i, ElemType e)
```

```
{ int j=0;
```

```
  LinkNode *p=L, *s;
```

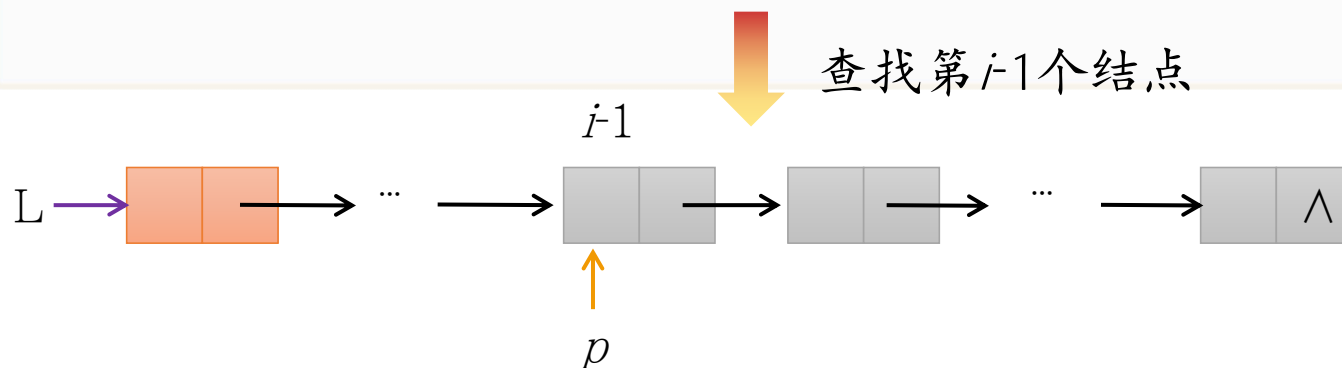
//p指向头结点, j置为0

```
  while (j<i-1 && p!=NULL)
```

```
  {   j++;
```

```
      p=p->next;
```

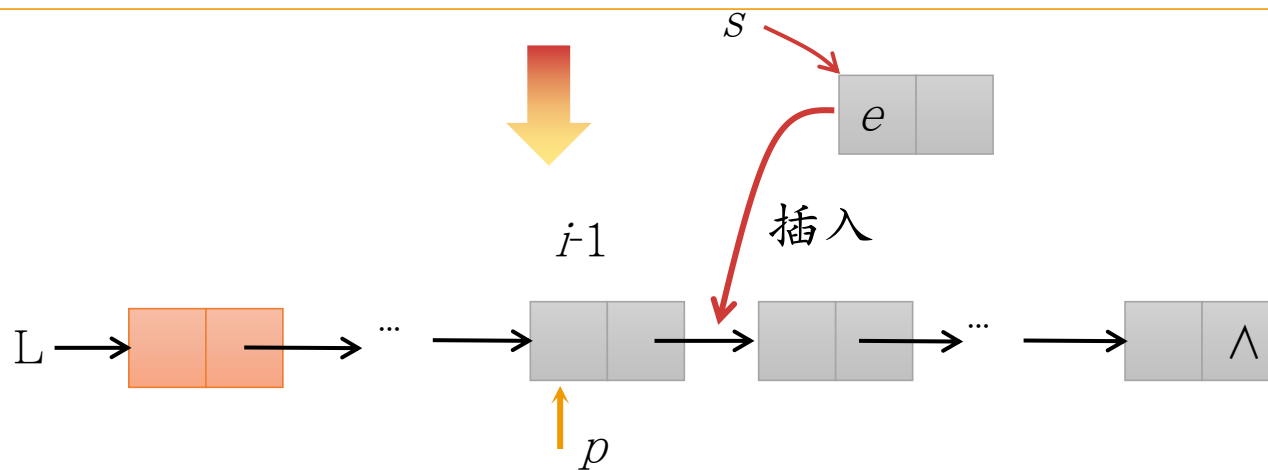
```
  }
```



2.3 线性表的链式存储结构

2.3.2 单链表

```
if (p==NULL)                //未找到第i-1个结点, 返回false
    return false;
else //找到第i-1个结点p, 插入并返回true
{
    s=(LinkNode *)malloc(sizeof(LinkNode));
    s->data=e;                //创建新结点s, 其data域置为e
    s->next=p->next;          //将s插入到p之后
    p->next=s;
    return true;
}
```



2.3 线性表的链式存储结构

2.3.2 单链表

(9) 删除数据元素 `ListDelete(&L, i, &e)`

先在单链表L中找到第 $i-1$ 个结点 p ，若存在这样的结点，
且也存在后继结点，则删除该后继结点。

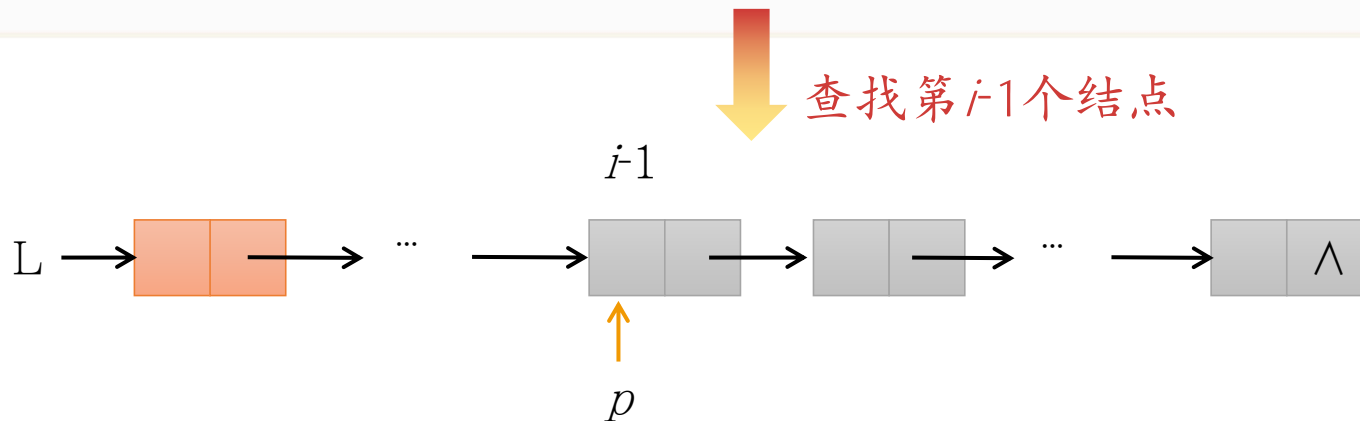


2.3 线性表的链式存储结构

2.3.2 单链表

```
bool ListDelete(LinkNode *&L, int i, ElemType &e)
{
    int j=0;
    LinkNode *p=L, *q;           //p指向头结点, j置为0
```

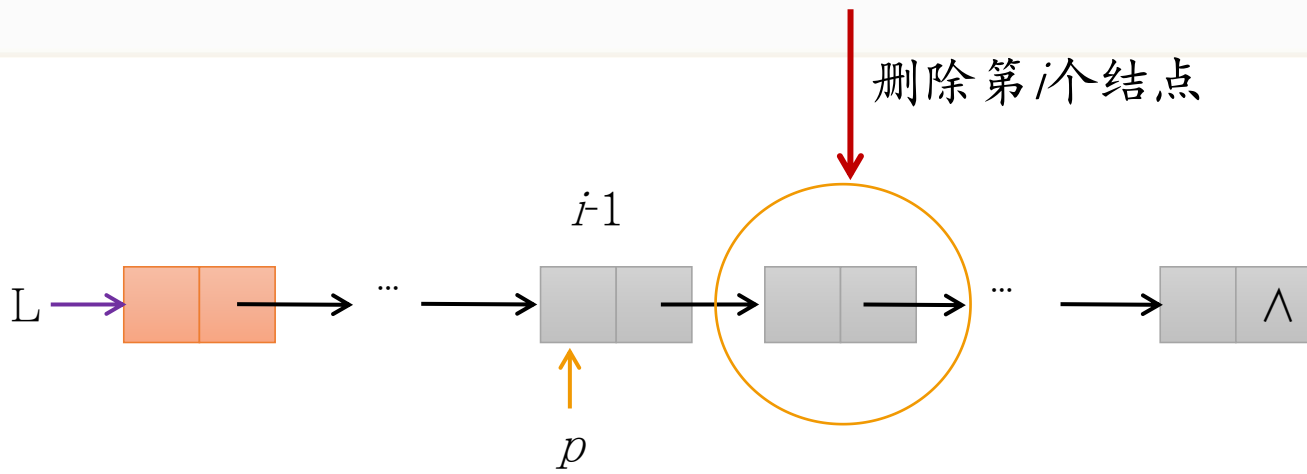
```
    while (j<i-1 && p!=NULL)    //查找第i-1个结点
    {
        j++;
        p=p->next;
    }
```



2.3 线性表的链式存储结构

2.3.2 单链表

```
if (p==NULL)                //未找到第i-1个结点，返回false
    return false;
else                          //找到第i-1个结点p
{
    q=p->next;                //q指向第i个结点
    if (q==NULL)              //若不存在第i个结点，返回false
        return false;
    e=q->data;
    p->next=q->next;           //从单链表中删除q结点
    free(q);                   //释放q结点
    return true;               //返回true表示成功删除第i个结点
}
```



2.3 线性表的链式存储结构

2.3.2 单链表

4、单链表的算法设计方法

单链表的算法设计是线性表链式存储结构算法设计的基础，是需要重点掌握的内容。

这里总结一般的算法设计方法。

① 以建表算法为基础的算法设计

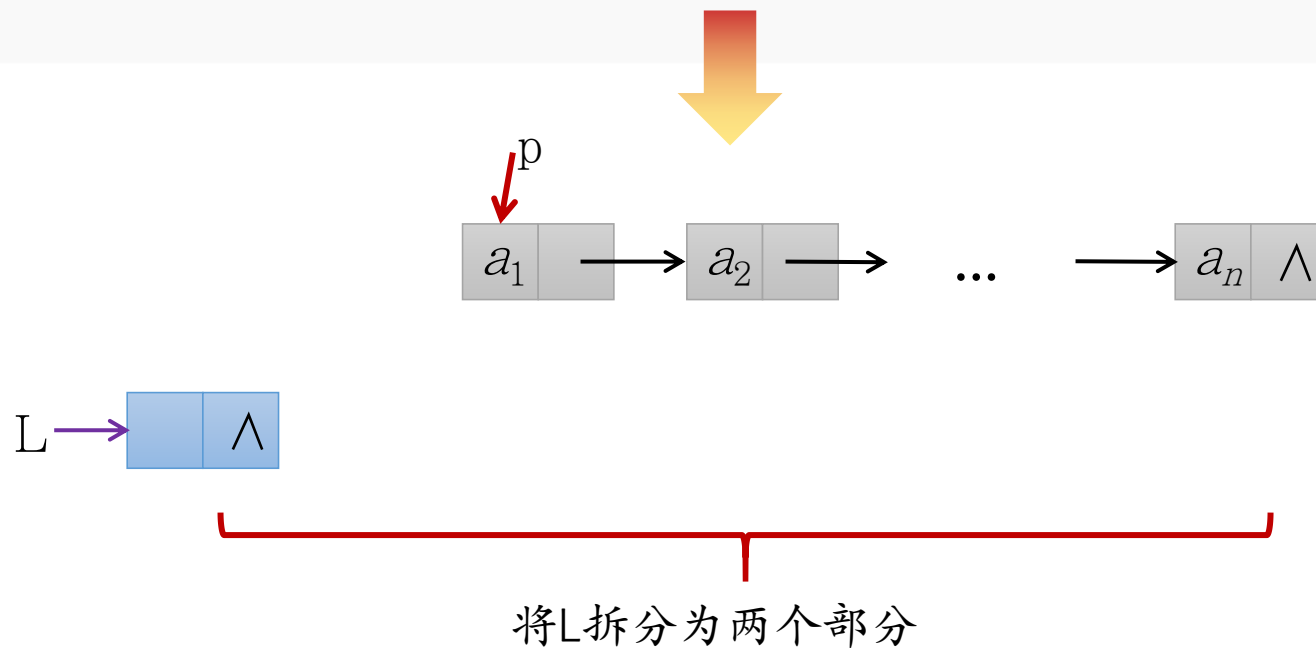
- 单链表有尾插法和头插法两种建表算法。
- 很多算法是以这两个建表算法为基础进行设计的。



2.3 线性表的链式存储结构

2.3.2 单链表

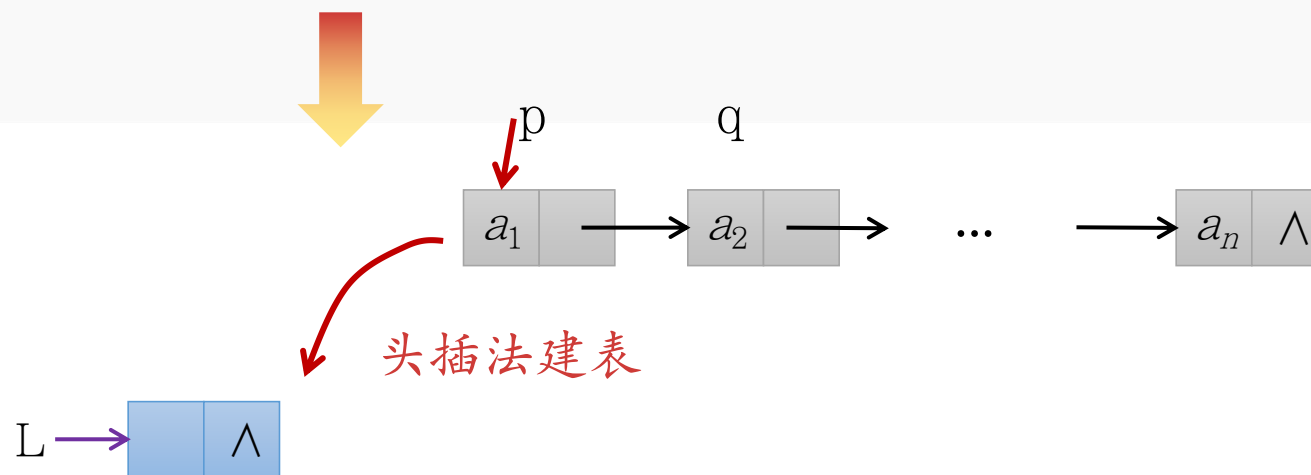
```
void Reverse(LinkNode *&L)
{
    LinkNode *p=L->next, *q;
    L->next=NULL;
```



2.3 线性表的链式存储结构

2.3.2 单链表

```
while (p!=NULL)
{
    q=p->next;           //临时保存p的后继结点
    p->next=L->next;      //将p结点采用头插法连接
    L->next=p;
    p=q;
}
```



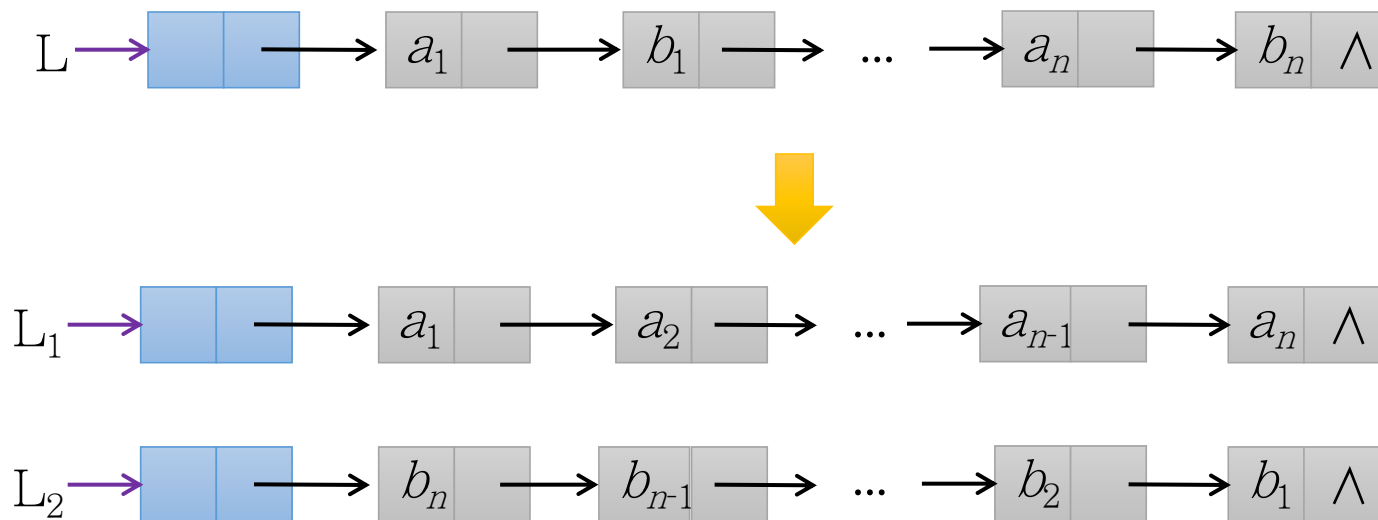
2.3 线性表的链式存储结构

2.3.2 单链表

【例2.6】 假设有一个带头结点的单链表 $L = (a_1, b_1, a_2, b_2, \dots, a_n, b_n)$ 。设计一个算法将其拆分成两个带头结点的单链表 L_1 和 L_2 :

$$L_1 = (a_1, a_2, \dots, a_n), \quad L_2 = (b_n, b_{n-1}, \dots, b_1)$$

要求 L_1 使用 L 的头结点。

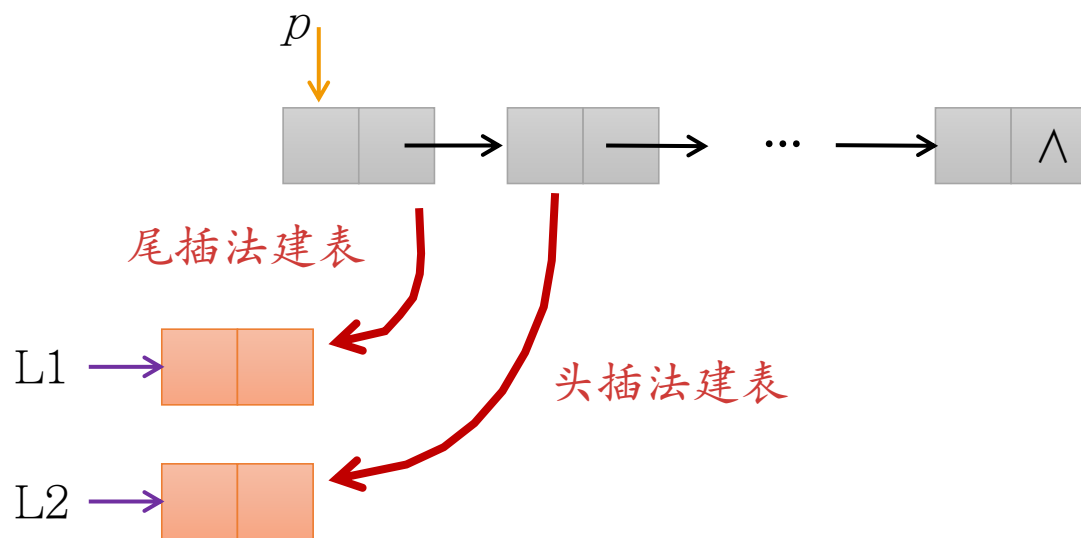


2.3 线性表的链式存储结构

2.3.2 单链表

解

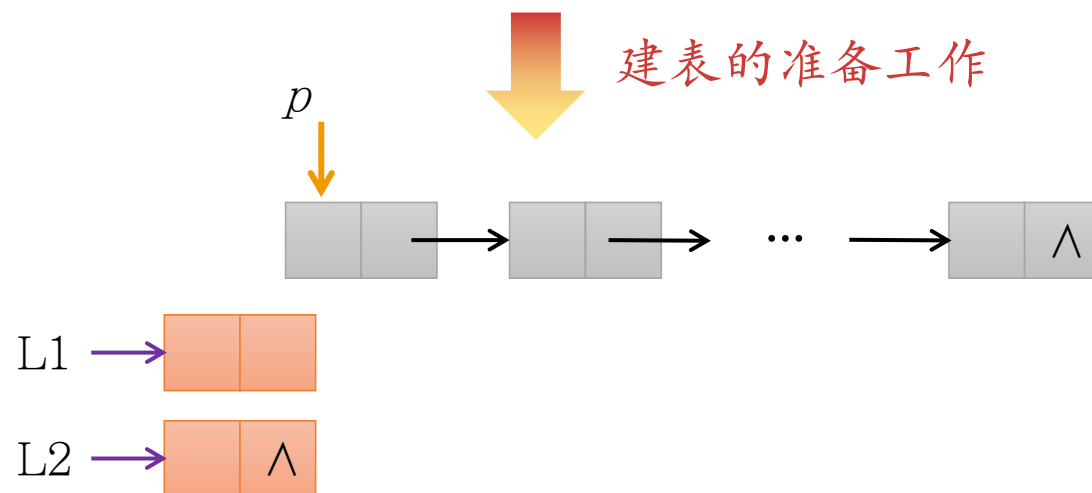
- 利用原单链表L中的所有结点通过改变指针域重组为单链表L1和L2。
- 由于L1中结点的相对顺序与L中的相同，所以采用尾插法建立单链表L1；
- 由于L2中结点的相对顺序与L中的相反，所以采用头插法建立单链表L2。



2.3 线性表的链式存储结构

2.3.2 单链表

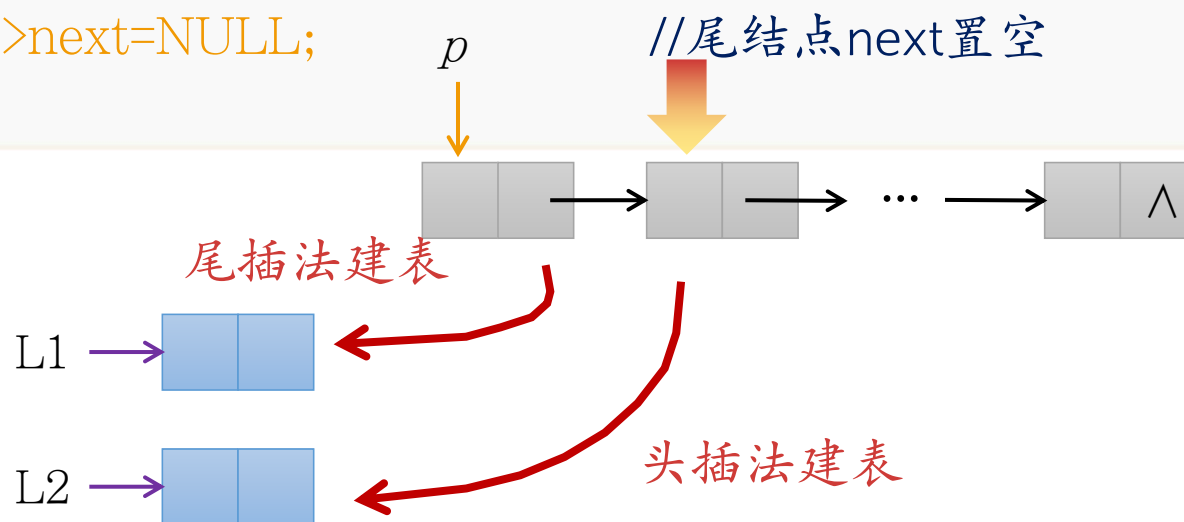
```
void split(LinkNode *&L, LinkNode *&L1, LinkNode *&L2)
{
    LinkNode *p=L->next, *q, *r1;           //p指向第1个数据结点
    L1=L;                                     //L1利用原来L的头结点
    r1=L1;                                   //r1始终指向L1的尾结点
    L2=(LinkNode *)malloc(sizeof(LinkNode)); //创建L2的头结点
    L2->next=NULL;                           //置L2的指针域为NULL
}
```



2.3 线性表的链式存储结构

2.3.2 单链表

```
while (p!=NULL)
{
    r1->next=p;           //采用尾插法将p(data值为ai)插入L1中
    r1=p;
    p=p->next;           //p移向下一个结点(data值为bi)
    q=p->next;           //用q保存p的后继结点
    p->next=L2->next;     //采用头插法将p插入L2中
    L2->next=p;
    p=q;                 //p重新指向下一个结点
}
r1->next=NULL;          //尾结点next置空
}
```



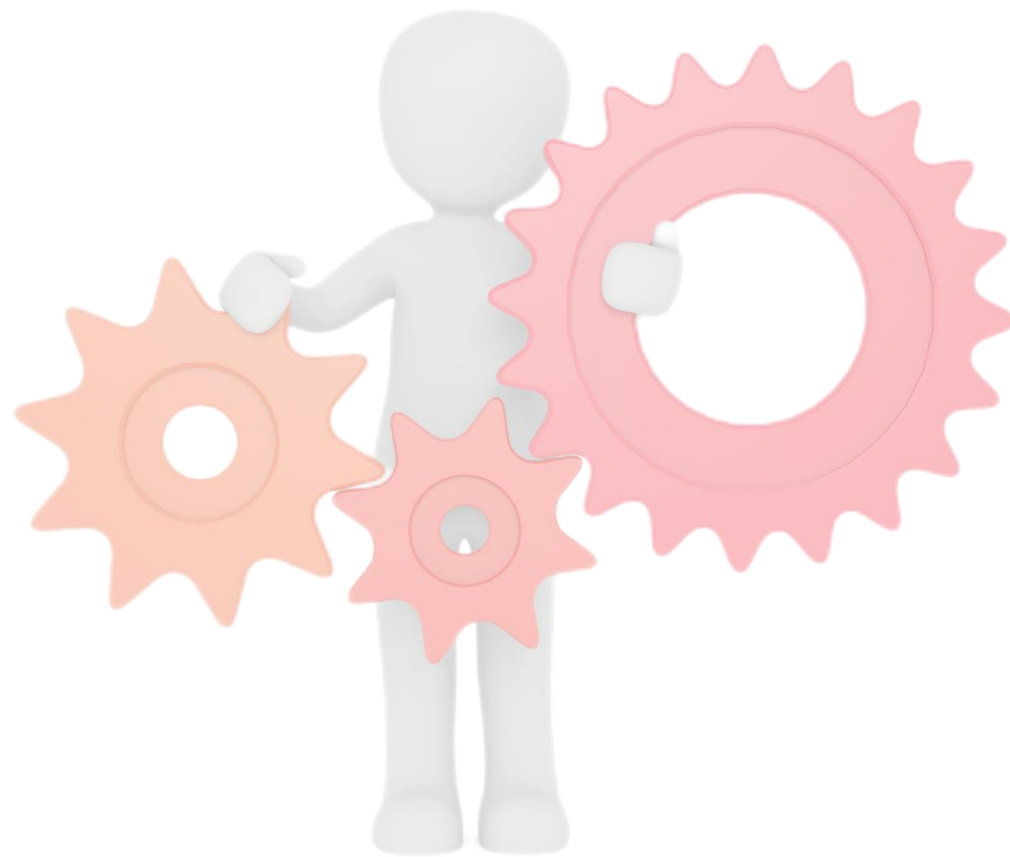
2.3 线性表的链式存储结构

2.3.2 单链表

4、单链表的算法设计方法

② 以查找为基础的算法设计

- 按照条件进行结点查找;
- 进行插入或者删除操作。

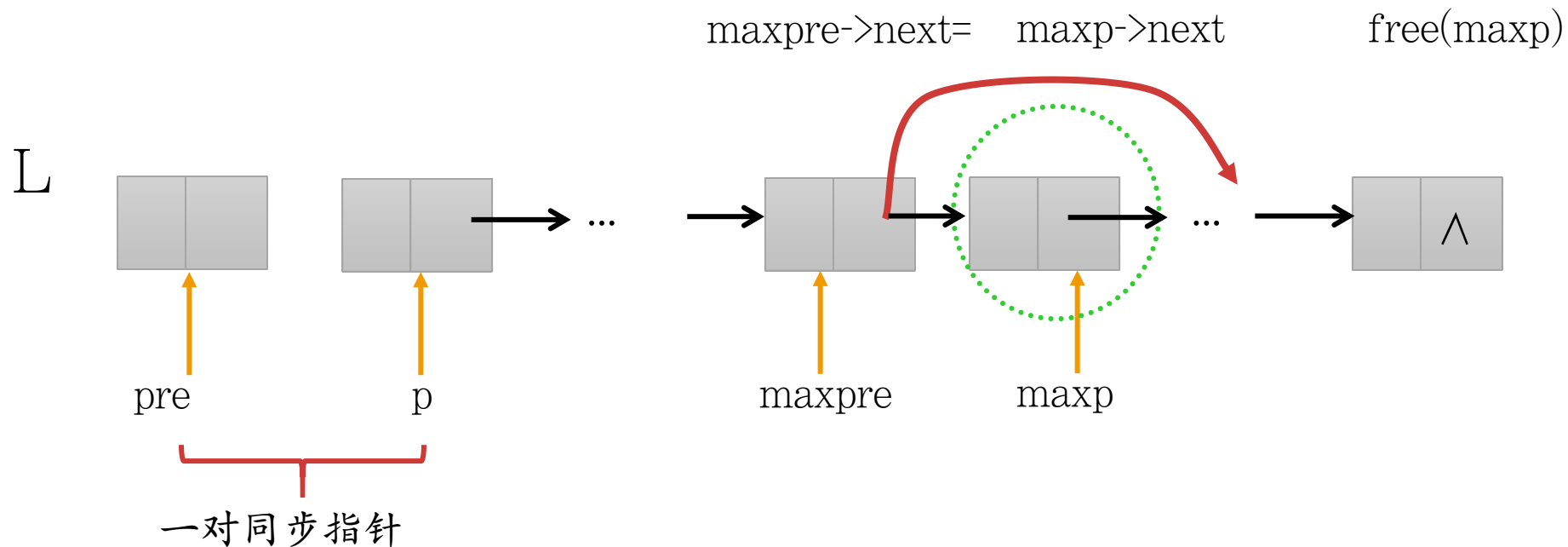


2.3 线性表的链式存储结构

2.3.2 单链表

【例2.7】设计一个算法，删除一个单链表L中元素值最大的结点（假设最大值结点是唯一的）。

解



2.3 线性表的链式存储结构

2.3.2 单链表

```
void delmaxnode(LinkNode *&L)
{
    LinkNode *p=L->next, *pre=L, *maxp=p, *maxpre=pre;

    while (p!=NULL)
    {
        if (maxp->data<p->data)           //若找到一个更大的结点
        {
            maxp=p;                       //更改maxp
            maxpre=pre;                   //更改maxpre
        }
        pre=p;                           //p、pre同步后移一个结点
        p=p->next;
    }

    maxpre->next=maxp->next;              //删除maxp结点
    free(maxp);                          //释放maxp结点
}
```

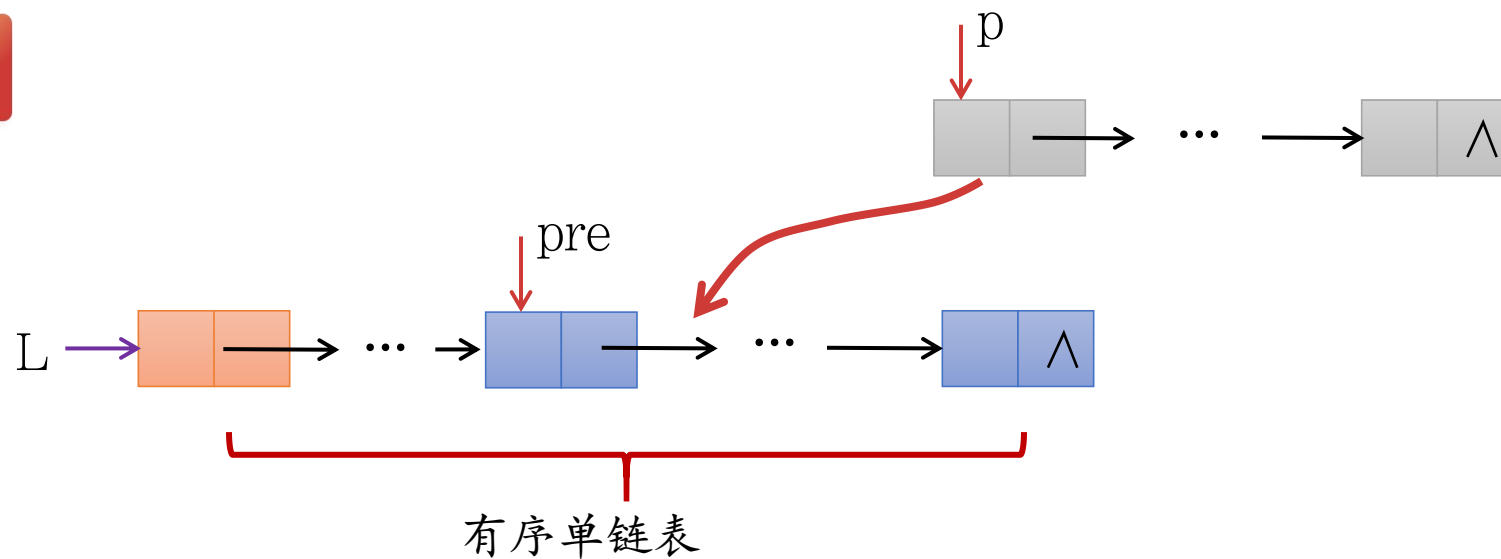
该算法的时间复杂度为 $O(n)$ 。

2.3 线性表的链式存储结构

2.3.2 单链表

【例2.8】有一个带头结点的单链表L（至少有一个数据结点），设计一个算法使其元素递增有序排列。

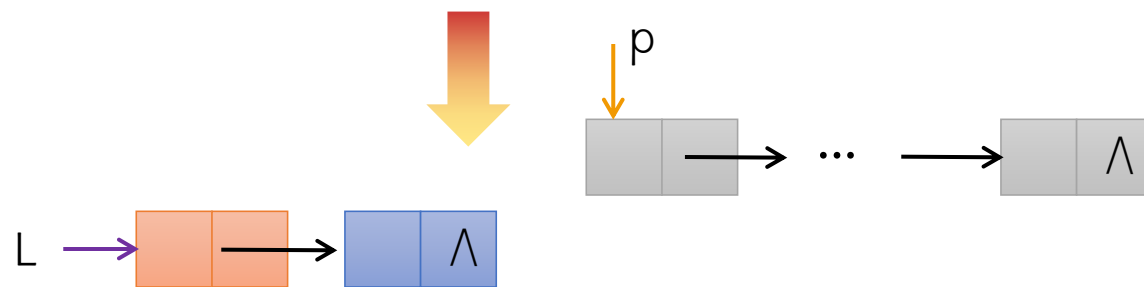
解



2.3 线性表的链式存储结构

2.3.2 单链表

```
void sort(LinkNode *&L)
{
    LinkNode *p, *pre, *q;
    p=L->next->next;           //p指向L的第2个数据结点
    L->next->next=NULL;        //构造只含一个数据结点的有序表
```



含一个数据结点的单
链表是有序单链表

将L拆分为两个部分

2.3 线性表的链式存储结构

2.3.2 单链表

```
while (p!=NULL)
```

```
{    q=p->next;
```

//q保存p结点后继结点的指针

```
    pre=L;    //从有序表开头，pre指向插入p的前驱结点
```

```
    while (pre->next!=NULL && pre->next->data<p->data)
```

```
        pre=pre->next;    //在有序表中找插入p的前驱结点pre
```

```
    p->next=pre->next;
```

```
    pre->next=p;
```

```
    p=q;
```

//扫描原单链表余下的结点

```
}  
}
```

在有序单链表中查找插入结点的前驱结点pre

在pre之后插入p

该算法的时间复杂度为 $O(n^2)$ 。

第2章 线性表

提纲

CONTENTS

2.1 线性表及其逻辑结构

2.2 线性表的顺序存储结构



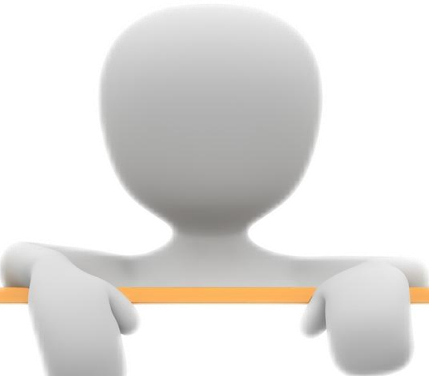
2.3 线性表的链式存储结构

2.4 线性表的应用

2.5 有序表

2.3 线性表的链式存储结构

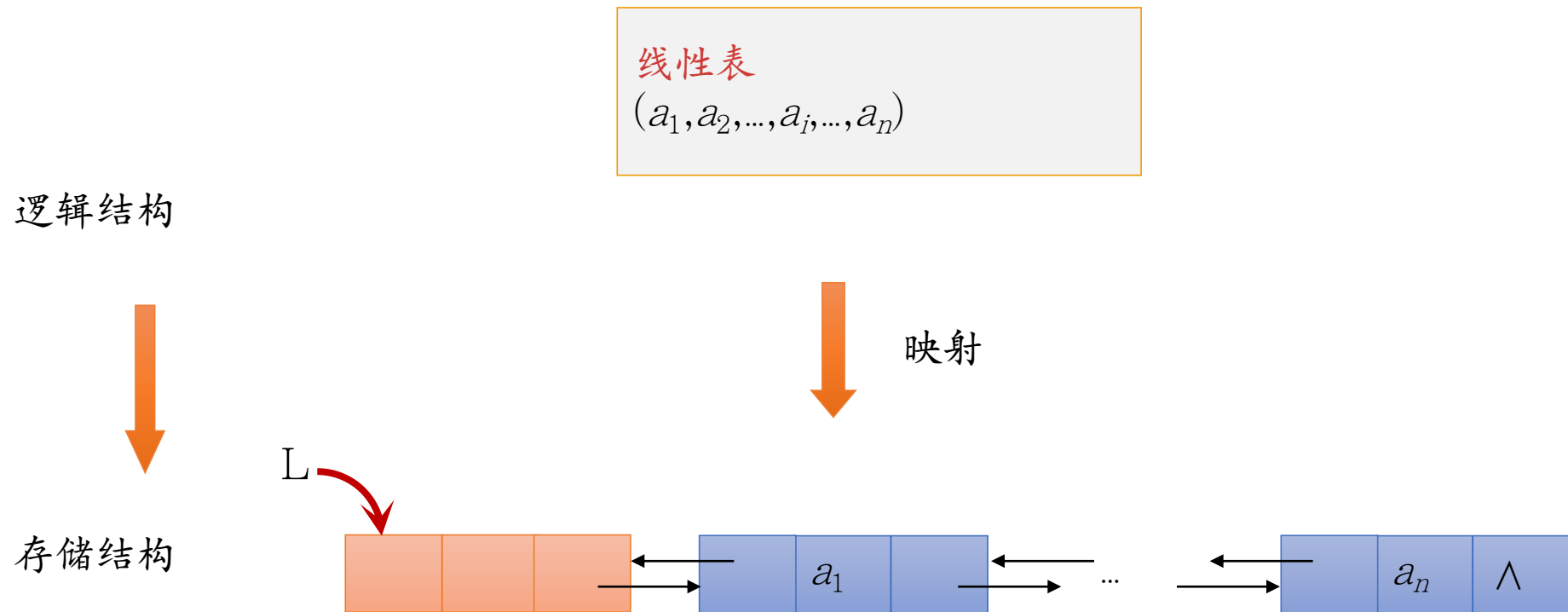
2.3.3 双链表



在线性表的链式存储结构中，每个物理结点增加一个指向后继结点的指针域和一个指向前驱结点的指针域 $\Rightarrow$ 双链表。

2.3 线性表的链式存储结构

2.3.3 双链表



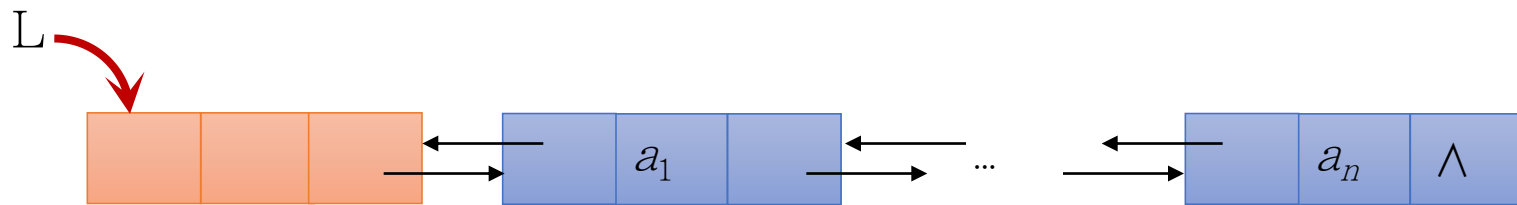
带头结点双链表示意图

2.3 线性表的链式存储结构

2.3.3 双链表

双链表的优点:

- 从任一结点出发可以快速找到其前驱结点和后继结点;
- 从任一结点出发可以访问其他结点。



2.3 线性表的链式存储结构

2.3.3 双链表

对于双链表，采用类似于单链表的类型定义，其结点类型DLinkNode声明如下：

```
typedef struct DNode      //双链表结点类型
{
    ElemType data;
    struct DNode *prior;  //指向前驱结点
    struct DNode *next;   //指向后继结点
} DLinkNode;
```



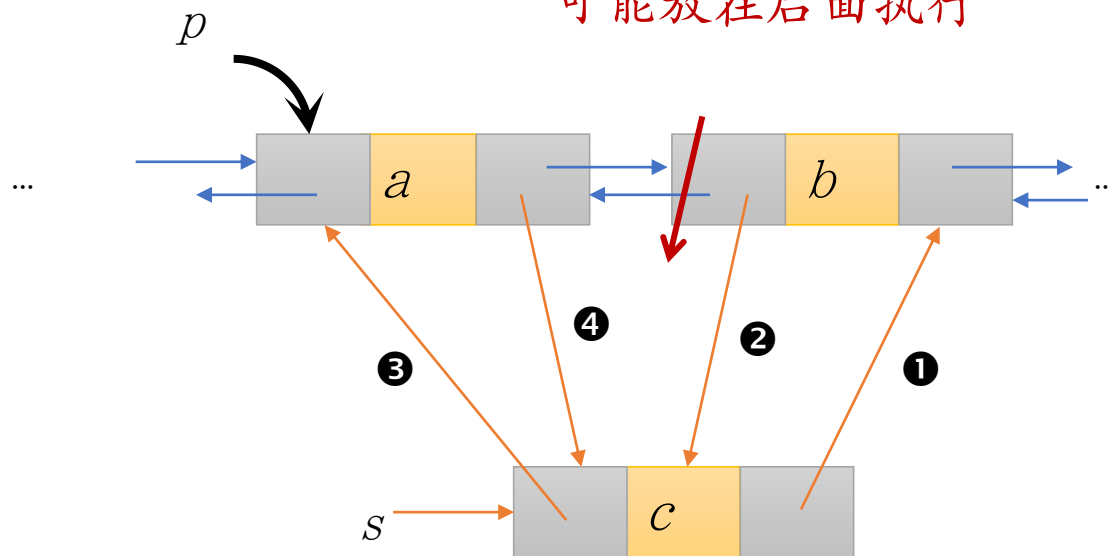
2.3 线性表的链式存储结构

2.3.3 双链表

(1) 双链表插入结点操作

在 p 结点之后插入结点 s

p 结点next的修改尽可能放在后面执行



操作语句:

- ① $s \rightarrow \text{next} = p \rightarrow \text{next}$
- ② $p \rightarrow \text{next} \rightarrow \text{prior} = s$ $p \rightarrow \text{next} \neq \text{NULL}$
- ③ $s \rightarrow \text{prior} = p$
- ④ $p \rightarrow \text{next} = s$

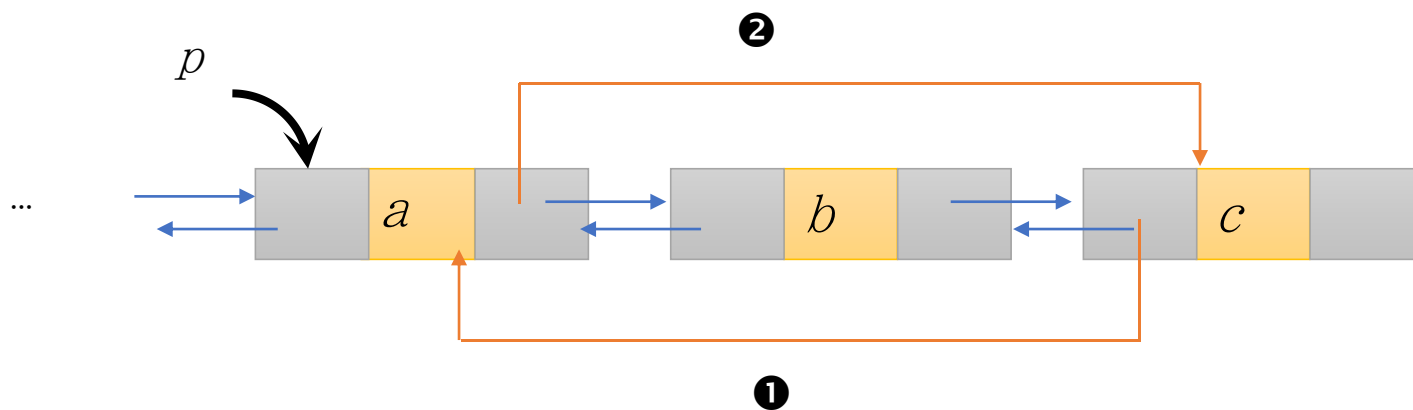
插入完毕

2.3 线性表的链式存储结构

2.3.3 双链表

(2) 双链表删除结点操作

删除 p 结点之后的一个结点



操作语句:

① $p \rightarrow \text{next} \rightarrow \text{next} \rightarrow \text{prior} = p$

② $p \rightarrow \text{next} = p \rightarrow \text{next} \rightarrow \text{next}$

删除完毕

2.3 线性表的链式存储结构

2.3.3 双链表

1、建立双链表

整体建立双链表也有两种方法：头插法和尾插法。

与单链表的建表算法相似，主要是插入和删除的不同。

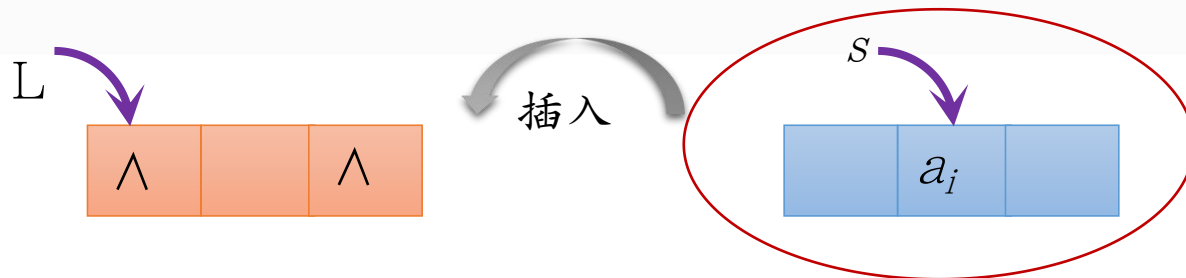


2.3 线性表的链式存储结构

2.3.3 双链表

头插法建立双链表：由含有 n 个元素的数组 a 创建带头结点的双链表 L 。

```
void CreateListF(DLinkNode *&L, ElemType a[], int n)
{ DLinkNode *s; int i;
  L=(DLinkNode *)malloc(sizeof(DLinkNode)); //创建头结点
  L->prior=L->next=NULL; //前后指针域置为NULL
  for (i=0;i<n;i++) //循环建立数据结点
  {
    s=(DLinkNode *)malloc(sizeof(DLinkNode));
    s->data=a[i]; //创建数据结点s
    s->next=L->next; //将s插入到头结点之后
    if (L->next!=NULL) //若L存在数据结点, 修改前驱指针
      L->next->prior=s;
    L->next=s;
    s->prior=L;
  } }
```



2.3 线性表的链式存储结构

2.3.3 双链表

尾插法建立双链表： 由含有 n 个元素的数组 a 创建带头结点的双链表 L 。

```
void CreateListR(DLinkNode *&L, ElemType a[], int n)
{ DLinkNode *s, *r;
  int i;
  L=(DLinkNode *)malloc(sizeof(DLinkNode)); //创建头结点
  r=L; //r始终指向尾结点, 开始时指向头结点
  for (i=0;i<n;i++) //循环建立数据结点
  { s=(DLinkNode *)malloc(sizeof(DLinkNode));
    s->data=a[i]; //创建数据结点s
    r->next=s;s->prior=r; //将s插入r之后
    r=s; //r指向尾结点
  }
  r->next=NULL; //尾结点next域置为NULL
}
```



2.3 线性表的链式存储结构

2.3.3 双链表

2、线性表基本运算在双链表中的实现

和单链表相比，双链表主要是插入和删除运算不同。

① 双链表的插入算法: `ListInsert(&L, i, e)`

```
bool ListInsert(DLinkNode *&L, int i, ElemType e)
{
    int j=0;
    DLinkNode *p=L, *s;           //p指向头结点, j设置为0
    if (i <= 0) return false;
    while (j<i-1 && p!=NULL)      //查找第i-1个结点
    {
        j++;
        p=p->next;
    }
```

查找第 $i-1$ 个结点 p

2.3 线性表的链式存储结构

2.3.3 双链表

```
if (p==NULL)                                //未找到第i-1个结点, 返回false
    return false;
else                                          //找到第i-1个结点p, 在其后插入新结点s
{
    s=(DLinkNode *)malloc(sizeof(DLinkNode));
    s->data=e;                               //创建新结点s
    s->next=p->next; //在p之后插入s结点
    if (p->next!=NULL)                       //若存在后继结点, 修改其前驱指针
        p->next->prior=s;
    s->prior=p;
    p->next=s;
    return true;
}
```

↓

新建结点s, 将其插入到p结点之后

另外解法: 在双链表中, 可以查找第*i*个结点, 并在它前面插入一个结点。

2.3 线性表的链式存储结构

2.3.3 双链表

```
while (j < i && p != NULL)
{
    j++;
    p = p->next;
}
if (p == NULL)
    return false;
s = (DLinkNode *)malloc(sizeof(DLinkNode)); // 创建新结点
if (s == NULL) return false;
s->data = e; //赋值

// 插入到 p 前面
s->prior = p->prior;
s->next = p;
p->prior->next = s;
p->prior = s;
```

2.3 线性表的链式存储结构

2.3.3 双链表

2、线性表基本运算在双链表中的实现

② 双链表的删除算法:

```
bool ListDelete(DLinkNode *&L, int i, ElemType &e)
{
    int j=0; DLinkNode *p=L, *q;    //p指向头结点, j设置为0
    if (i <= 0) return false;
    while (j<i-1 && p!=NULL)        //查找第i-1个结点
    {
        j++;
        p=p->next;
    }
```

查找第 $i-1$ 个结点 p

2.3 线性表的链式存储结构

2.3.3 双链表

```
if (p==NULL)                //未找到第i-1个结点
    return false;
else                          //找到第i-1个结点p
{
    q=p->next;                //q指向第i个结点
    if (q==NULL)              //当不存在第i个结点时返回false
        return false;
    e=q->data;
    p->next=q->next;           //从双单链表中删除q结点
    if (q->next!=NULL)         //若q结点存在后继结点
        q->next->prior=p;      //修改q结点后继结点的前驱指针
    free(q);                  //释放q结点
    return true;
}
```

↓
删除q结点并释放其空间

另外解法：在双链表中，可以查找第i个结点，并将它删除。

2.3 线性表的链式存储结构

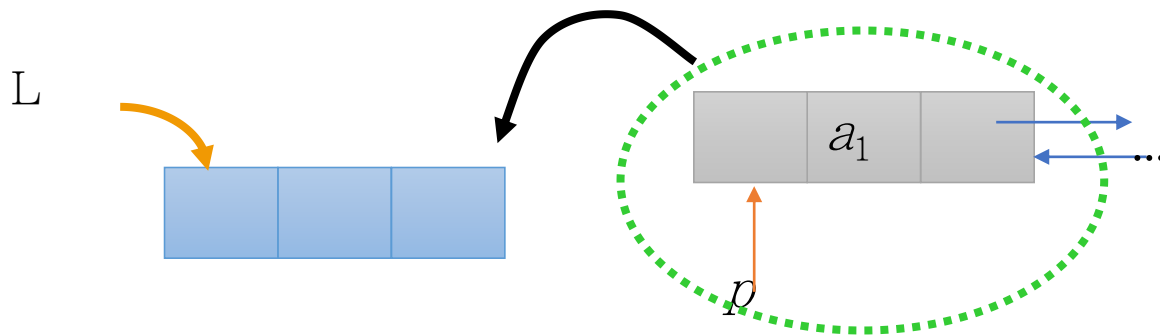
2.3.3 双链表

3、双链表的应用示例

【例2.9】 有一个带头结点的双链表L，设计一个算法将其所有元素逆置，即第1个元素变为最后一个元素，第2个元素变为倒数第2个元素，...，最后一个元素变为第1个元素。

解

采用头插法建表。



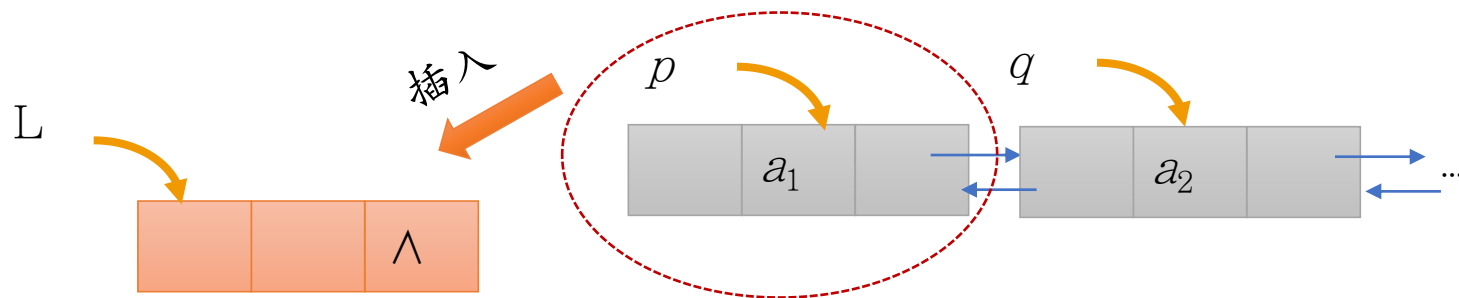
2.3 线性表的链式存储结构

2.3.3 双链表

```
void reverse(DLinkNode *&L)
{ DLinkNode *p=L->next, *q;
  L->next=NULL;
  while (p!=NULL)
  {   q=p->next;
      p->next=L->next;
      if (L->next!=NULL)
          L->next->prior=p;
      L->next=p;
      p->prior=L;
      p=q;
  }
```

//双链表结点逆置
//p指向开始结点
//构造只有头结点的双链表L
//扫描L的数据结点
//用q保存其后继结点
//采用头插法将p结点插入
//修改其前驱指针

//让p重新指向其后继结点



2.3 线性表的链式存储结构

2.3.4 循环链表

循环链表是另一种形式的链式存储结构。

- **循环单链表**：将表中尾结点的指针域改为指向表头结点，整个链表形成一个环。由此从表中任一结点出发均可找到链表中其他结点。
- **循环双链表**：形成两个环。



结点类型与非循环单链表的相同

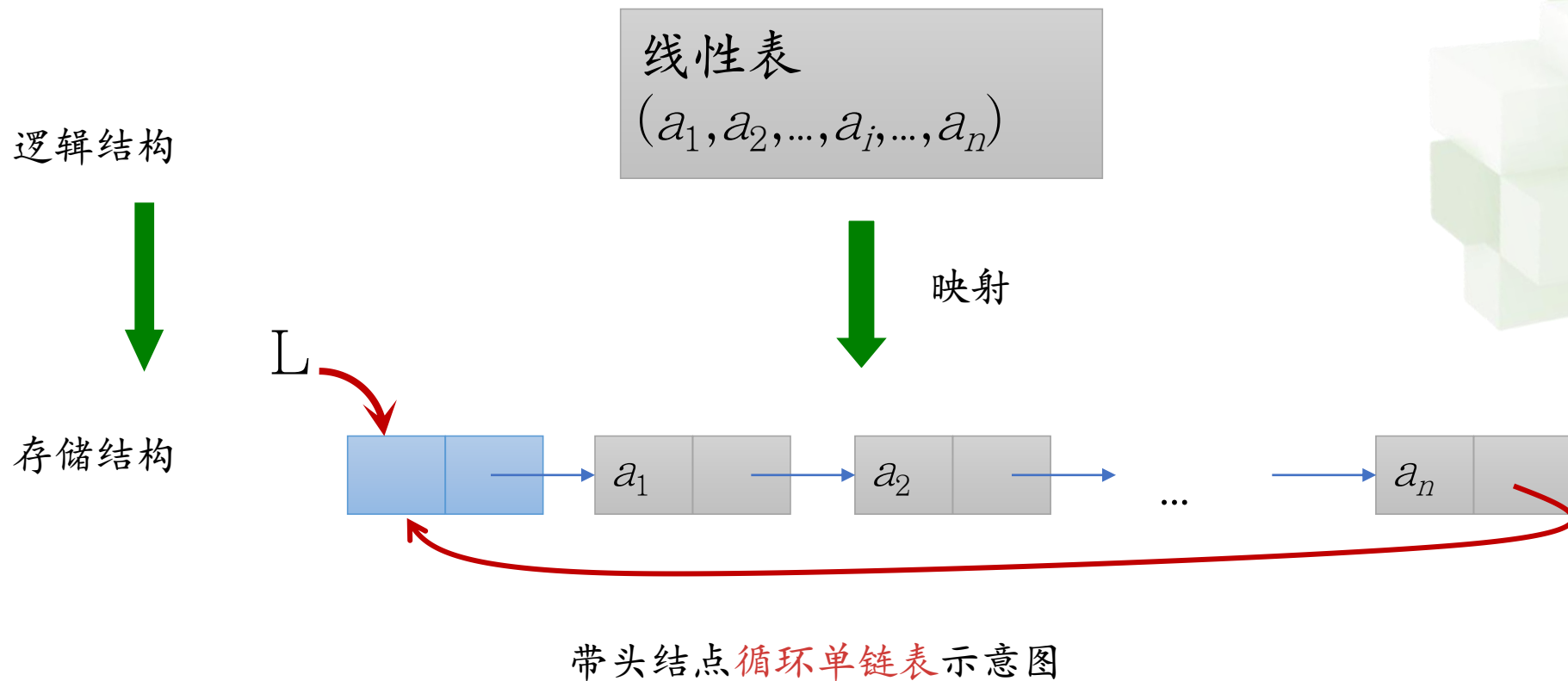


结点类型与非循环双链表的相同

2.3 线性表的链式存储结构

2.3.4 循环链表

1、循环单链表



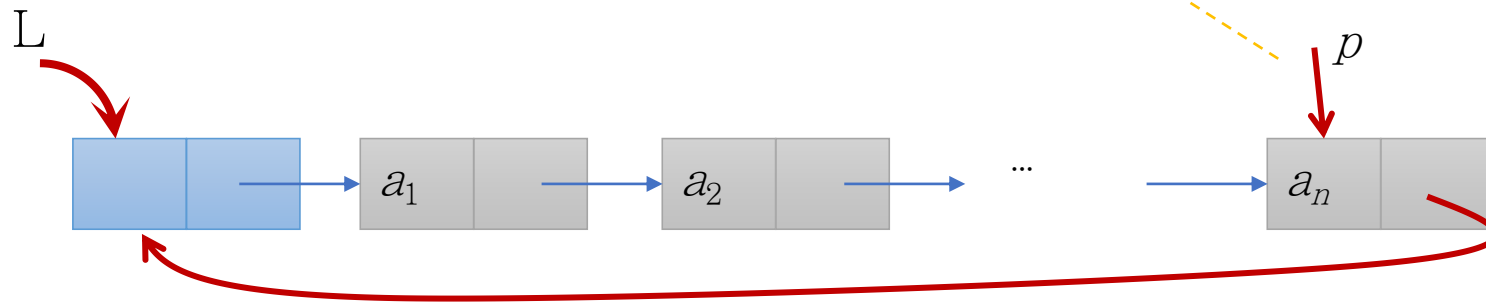
2.3 线性表的链式存储结构

2.3.4 循环链表

1、循环单链表

与非循环单链表相比，循环单链表：

- 链表中没有空指针域
- p 所指结点为尾结点的条件： $p \rightarrow \text{next} == L$



2.3 线性表的链式存储结构

2.3.4 循环链表

1、循环单链表

示例

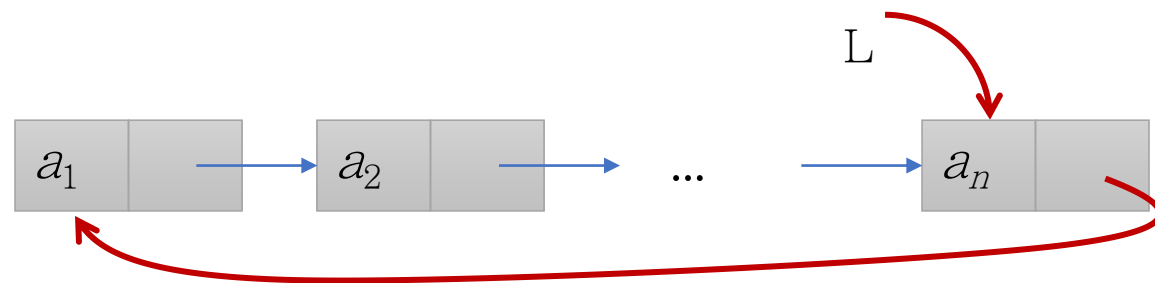
某线性表最常用的操作是在尾元素之后插入一个元素和删除第一个元素，故采用（ ）存储方式最节省运算时间。

- A. 单链表 $O(n), O(1)$
- B. 仅有头结点指针的循环单链表 $O(n), O(1)$
- C. 双链表 $O(n), O(1)$
- D. 仅有尾结点指针的循环单链表 $O(1), O(1)$



2.3 线性表的链式存储结构

2.3.4 循环链表

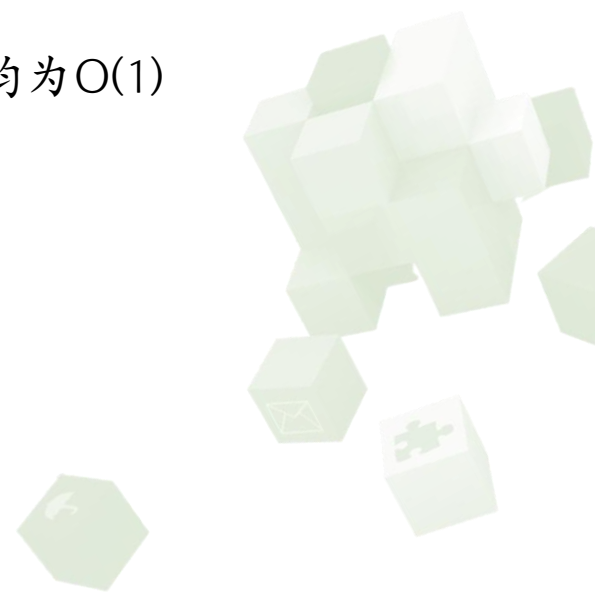


- 在尾元素之后插入一个元素
- 删除第一个元素



时间复杂度均为 $O(1)$

选择D



2.3 线性表的链式存储结构

2.3.4 循环链表

【例2.11】 有一个带头结点的循环单链表L，设计一个算法统计其data域值为x的结点个数。

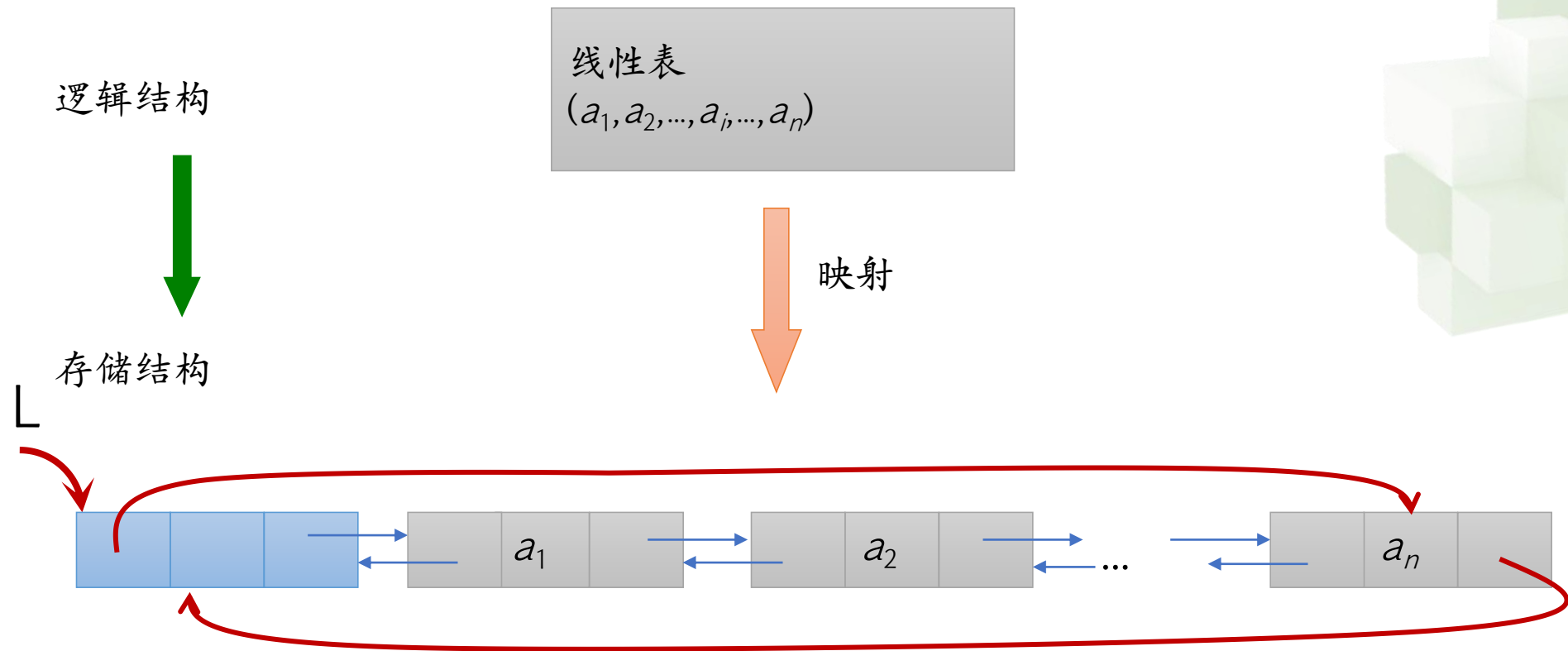
解

```
int count(LinkNode *L, ElemType x)
{
    int cnt=0;
    LinkNode *p=L->next;           //p指向首结点, cnt置为0
    while (p!=L)                   //扫描循环单链表L
    {
        if (p->data==x)             //找到值为x的结点后cnt增1
            cnt++;
        p=p->next;                  //p指向下一个结点
    }
    return cnt;                     //返回值为x的结点个数
}
```

2.3 线性表的链式存储结构

2.3.4 循环链表

2、循环双链表



带头结点循环双链表示意图

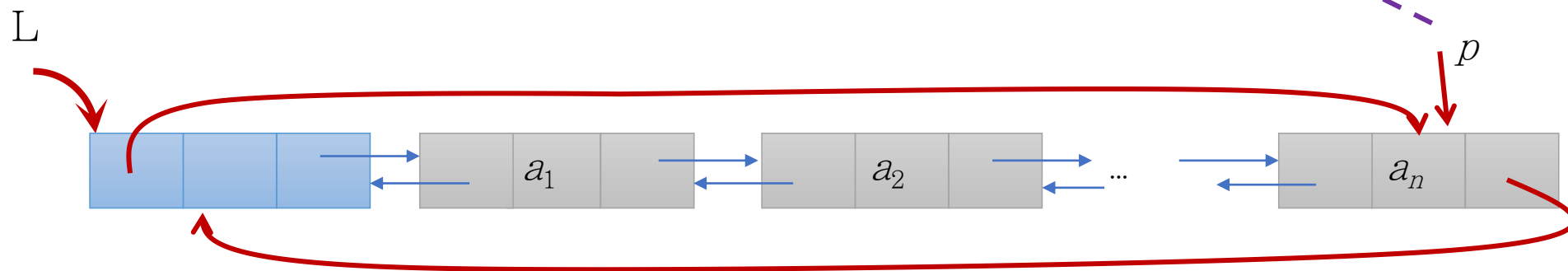
2.3 线性表的链式存储结构

2.3.4 循环链表

2、循环双链表

与非循环双链表相比，循环双链表：

- 链表中没有空指针域
- p 所指结点为尾结点的条件： $p \rightarrow \text{next} == L$
- 一步操作即 $L \rightarrow \text{prior}$ 可以找到尾结点



2.3 线性表的链式存储结构

2.3.4 循环链表

2、循环双链表

示例

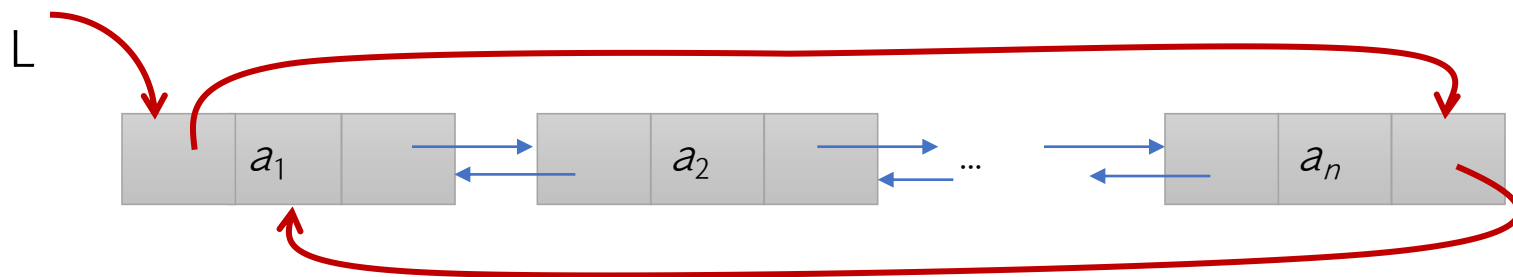
如果对含有 n ($n>1$) 个元素的线性表的运算有4种，即删除第一个元素、删除尾元素、在第一个元素前面插入新元素、在尾元素的后面插入新元素，则最好使用（ ）。

- A. 只有尾结点指针没有头结点的循环单链表 删头 $O(1)$,删尾 $O(n)$, 插头 $O(1)$,插尾 $O(1)$
- B. 只有尾结点指针没有头结点的非循环双链表 删头 $O(n)$,删尾 $O(1)$, 插头 $O(n)$,插尾 $O(1)$
- C. 只有首结点指针没有尾结点指针的循环双链表 删头 $O(1)$,删尾 $O(1)$, 插头 $O(1)$,插尾 $O(1)$
- D. 既有头指针也有尾指针的循环单链表 删头 $O(1)$,删尾 $O(n)$, 插头 $O(1)$,插尾 $O(1)$

需要找到尾指针的前驱结点

2.3 线性表的链式存储结构

2.3.4 循环链表



- 删除第一个元素
- 删除尾元素
- 在第一个元素前面插入新元素
- 在尾元素的后面插入新元素

时间复杂度均为 $O(1)$



选择C

2.3 线性表的链式存储结构

2.3.4 循环链表

【例2.12】 有一个带头结点的循环双链表L，设计一个算法删除第一个data域值为x的结点。

[illegible]

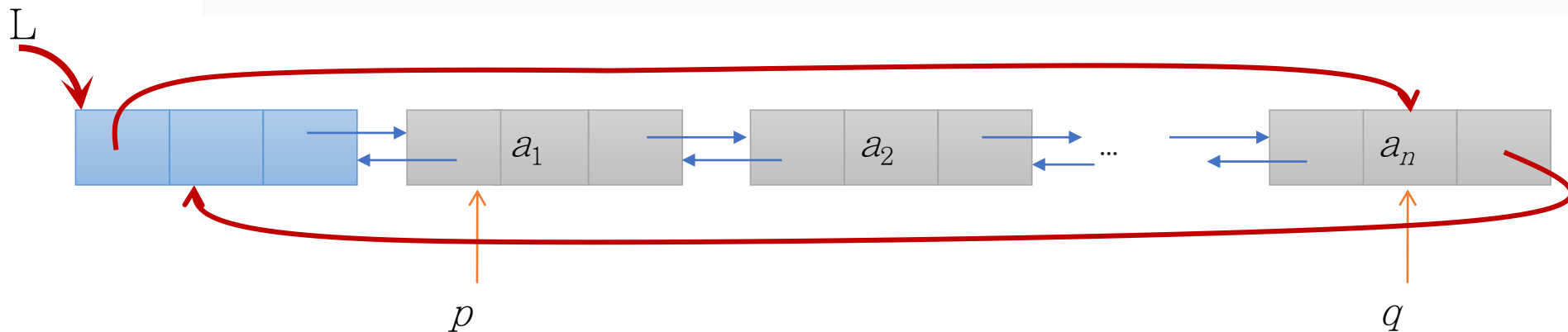
2.3 线性表的链式存储结构

2.3.4 循环链表

【例2.13】设计判断带头结点的循环双链表L（含两个以上的结点）是否对称相等的算法。

解

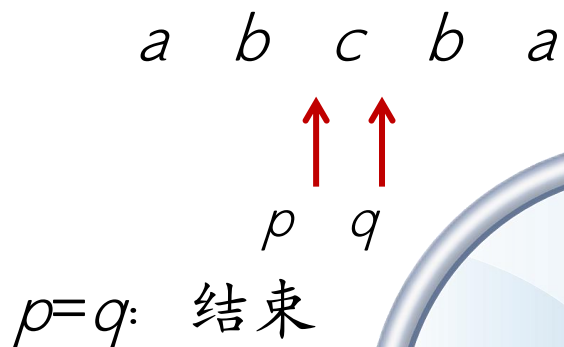
- p 从左向右扫描L， q 从右向左扫描L
- 若对应数据结点的data域不相等，则退出循环
- 否则继续比较，直到 p 与 q 相等或 p 的下一个结点为 q 为止。



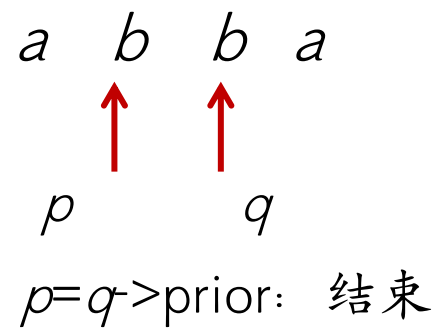
2.3 线性表的链式存储结构

2.3.4 循环链表

① 数据结点为奇数的情况：



② 数据结点为偶数的情况：



2.3 线性表的链式存储结构

2.3.4 循环链表

```
bool Symm(DLinkNode *L)
{
    bool same=true;
    DLinkNode *p=L->next;           //p指向首结点
    DLinkNode *q=L->prior;          //q指向尾结点
    while (same)
    {
        if (p->data!=q->data)
            same=false;
        else
        {
            if (p==q || p==q->prior) break;
            q=q->prior;               //q前移
            p=p->next;                //p后移
        }
    }
    return same;
}
```

第2章 线性表

提纲

CONTENTS

2.1 线性表及其逻辑结构

2.2 线性表的顺序存储结构

2.3 线性表的链式存储结构

2.4 线性表的应用

2.5 有序表

2.5 有序表

2.5.1 有序表的概念

所谓**有序表**，是指这样的**线性表**，其中所有元素以**递增**或**递减**方式有序排列。

后面讨论的有序表默认元素是以**递增方式排列**。

例如： $L = (1, 5, 8, 10, 15, 20)$ 就是一个整数有序表。



2.5 有序表

2.5.1 有序表的概念

有序表是线性表的一个子集。

有序表 $\subset$ 线性表

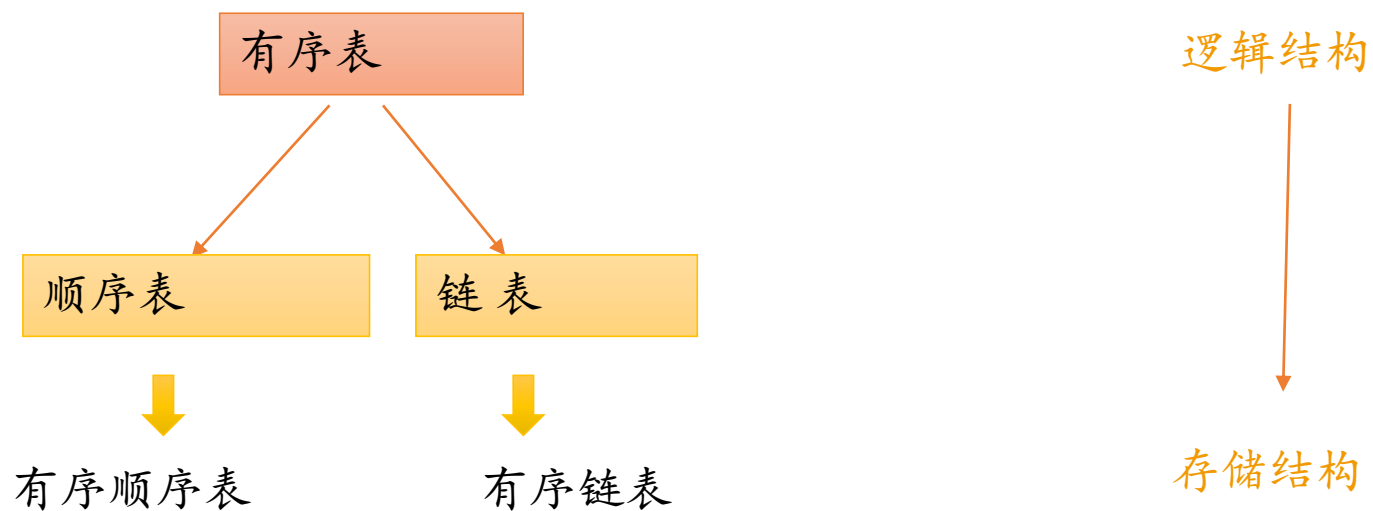
有序表和线性表中元素之间的逻辑关系相同，其区别是运算实现的不同。



2.5 有序表

2.5.2 有序表的存储结构及其基本运算算法

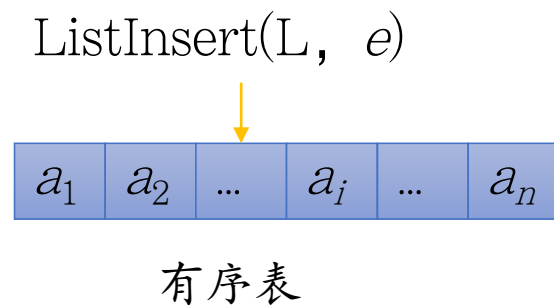
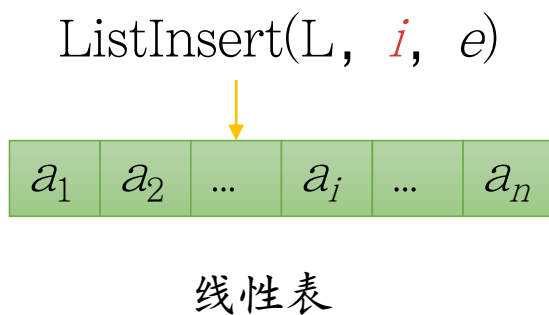
既然有序表中元素逻辑关系与线性表的相同，有序表可以采用与线性表相同的存储结构。



2.5 有序表

2.5.2 有序表的存储结构及其基本运算算法

- 在以顺序表或者链表存储有序表时，许多基本运算算法与线性顺序表或者链表的算法相同（不考虑优化）。
- 只有插入运算—ListInsert()算法有所差异。



2.5 有序表

2.5.2 有序表的存储结构及其基本运算算法

若以**顺序表**存储有序表，ListInsert()算法算法如下：

```
void ListInsert(SqList *&L, ElemType e)
{
    int i=0, j;
    while (i<L->length && L->data[i]<e)
        i++;
    for (j=L->length;j>i;j--)
        L->data[j]=L->data[j-1];
    L->data[i]=e;
    L->length++;
}
```

//查找值为e的元素
//将data[i..n]后移一个位置
//有序顺序表长度增1

2.5 有序表

2.5.2 有序表的存储结构及其基本运算算法

若以单链表存储有序表，ListInsert()的算法如下：

```
void ListInsert(LinkNode *&L, ElemType e)
{
    LinkNode *pre=L, *p;

    while (pre->next!=NULL && pre->next->data<e)
    {
        pre=pre->next;    //查找插入结点的前驱结点pre
    }

    p=(LinkNode *)malloc(sizeof(LinkNode));
    p->data=e;            //创建存放e的数据结点p
    p->next=pre->next;    //在pre结点之后插入p结点
    pre->next=p;
}
```

在pre之后插入p结点 查找插入的位置pre

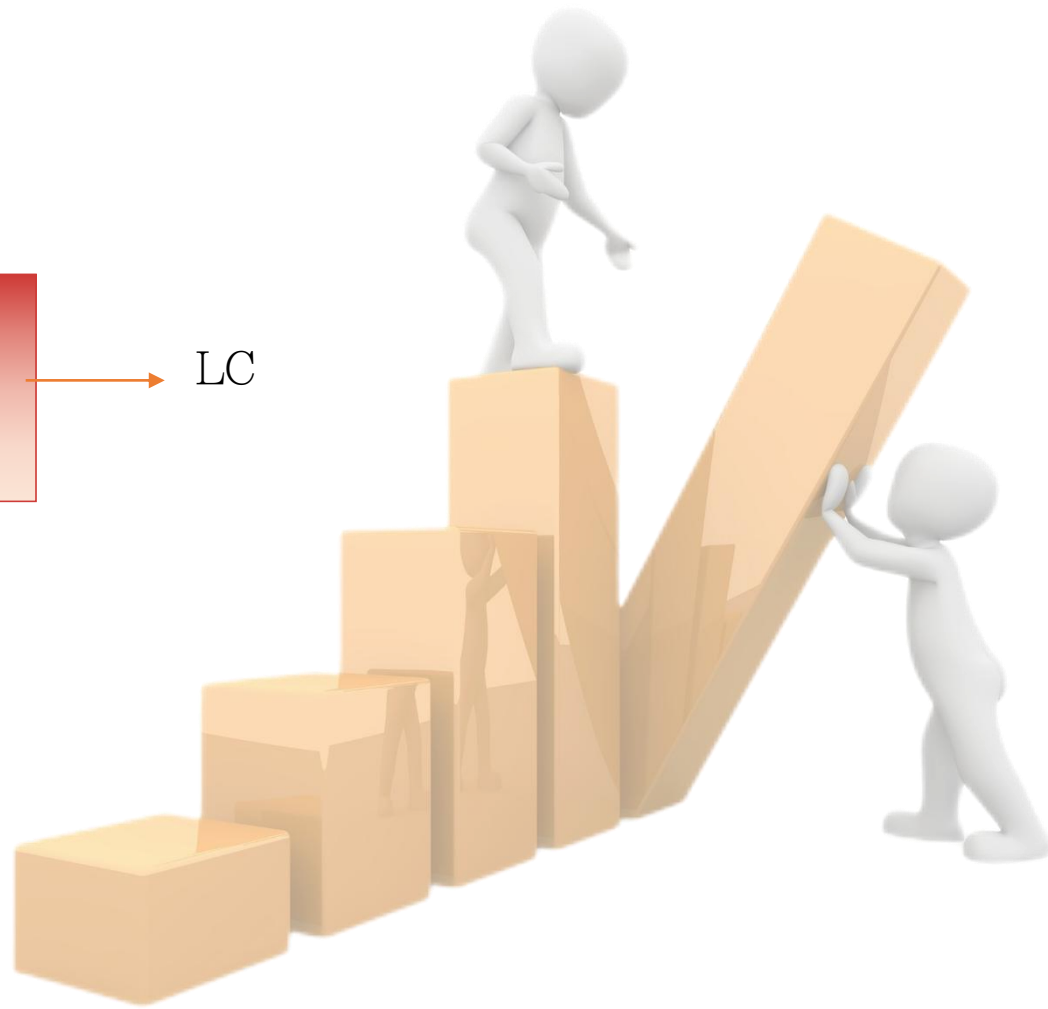
2.5 有序表

2.5.3 有序表的归并算法

【例2.14】 假设有两个有序表LA和LB。设计一个算法，将它们合并成一个有序表LC。



二路归并示意图



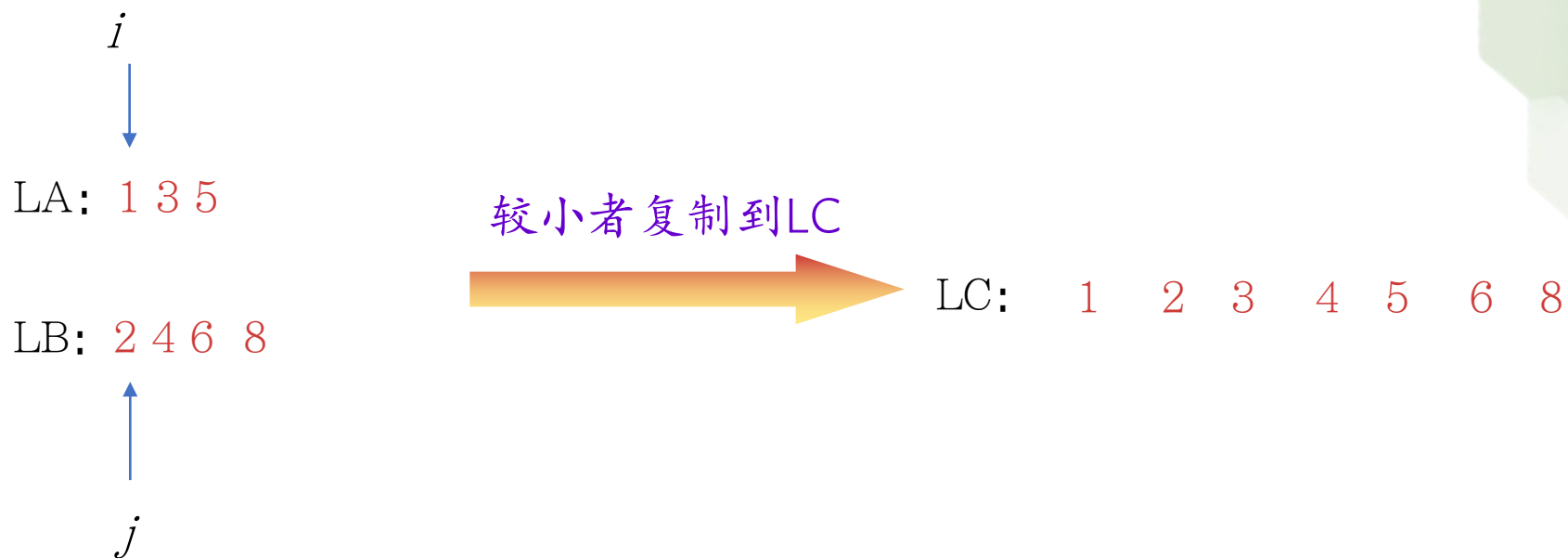
2.5 有序表

2.5.3 有序表的归并算法

顺序表二路归并示例的演示

例如, $LA = (1, 3, 5)$, $LB = (2, 4, 6, 8)$

其二路归并过程如下:



LA 、 LB 中每个元素恰好遍历一次, 时间执行次数为 $m+n$ 次。

2.5 有序表

2.5.3 有序表的归并算法

采用顺序表存放有序表时，二路归并算法如下：

```
void UnionList(SqList *LA, SqList *LB, SqList *&LC)
{ int i=0, j=0, k=0;          //i、j分别为LA、LB的下标，k为LC中元素个数
  LC=(SqList *)malloc(sizeof(SqList));      //建立有序顺序表LC
  while (i<LA->length && j<LB->length)
  { if (LA->data[i]<LB->data[j])
    { LC->data[k]=LA->data[i];
      i++;k++;
    }
    else //LA->data[i]>LB->data[j]
    { LC->data[k]=LB->data[j];
      j++;k++;
    }
  }
}
```

两个有序表均
没有遍历完

2.5 有序表

2.5.3 有序表的归并算法

```
while (i < LA->length)    //LA尚未扫描完，将其余元素插入LC中
{
    LC->data[k] = LA->data[i];
    i++; k++;
}
while (j < LB->length)    //LB尚未扫描完，将其余元素插入LC中
{
    LC->data[k] = LB->data[j];
    j++; k++;
}
LC->length = k;
}
```

本算法的时间复杂度为 $O(m+n)$ ，空间复杂度为 $O(m+n)$ 。

2.5 有序表

2.5.3 有序表的归并算法

采用单链表存放有序表时，二路归并算法如下：

```
void UnionList1(LinkNode *LA, LinkNode *LB, LinkNode *&LC)
{
    LinkNode *pa=LA->next, *pb=LB->next, *r, *s;
    LC=(LinkNode *)malloc(sizeof(LinkNode)); //创建LC的头结点
    r=LC;                                     //r始终指向LC的尾结点
    while (pa!=NULL && pb!=NULL)
    {
        if (pa->data<pb->data)
        {
            s=(LinkNode *)malloc(sizeof(LinkNode)); //复制结点
            s->data=pa->data;
            r->next=s;r=s;                          //采用尾插法将s插入到LC中
            pa=pa->next;
        }
        else
        {
            s=(LinkNode *)malloc(sizeof(LinkNode)); //复制结点
            s->data=pb->data;
            r->next=s;r=s;                          //采用尾插法将s插入到LC中
            pb=pb->next;
        }
    }
}
```

2.5 有序表

2.5.3 有序表的归并算法

```
while (pa!=NULL)
{ s=(LinkNode *)malloc(sizeof(LinkNode)); //复制结点
  s->data=pa->data;
  r->next=s;r=s; //采用尾插法将s插入到LC中
  pa=pa->next;
}
while (pb!=NULL)
{ s=(LinkNode *)malloc(sizeof(LinkNode)); //复制结点
  s->data=pb->data;
  r->next=s;r=s; //采用尾插法将s插入到LC中
  pb=pb->next;
}
r->next=NULL //尾结点的next域置空
}
```

本算法的时间复杂度为 $O(m+n)$ ，空间复杂度为 $O(m+n)$ 。

2.5 有序表

2.5.3 有序表的归并算法

- 上述两个算法的时间复杂度均为 $O(m+n)$ 。
- 主要的时间花费在元素比较上。
- 两个长度分别为 m 、 n 的有序表 A 和 B ，采用二路归并算法，最好情况下元素比较的次数（即最少的比较次数）是多少？

MIN(m , n):

如 $A=(1, 2, 3)$, $B=(5, 6, 7, 8, 9)$, 元素比较次数为3。

- 两个长度分别为 m 、 n 的有序表 A 和 B ，采用二路归并算法，最坏情况下元素比较的次数（即最多的比较次数）是多少？

$m+n-1$: 如 $A=(2, 4, 6)$, $B=(1, 3, 5, 7)$, 元素比较次数为6。

2.5 有序表

2.5.3 有序表的归并算法

- 上述两个算法的空间复杂度均为 $O(m+n)$ 。
- 如何设计空间复杂度均为 $O(1)$ 的算法？

采用有序顺序表时：

- A 与 B 归并到 C ， C 利用 A 或者 B 的空间（ A 或者 B 被破坏）。
- C 的空间在算法外分配。

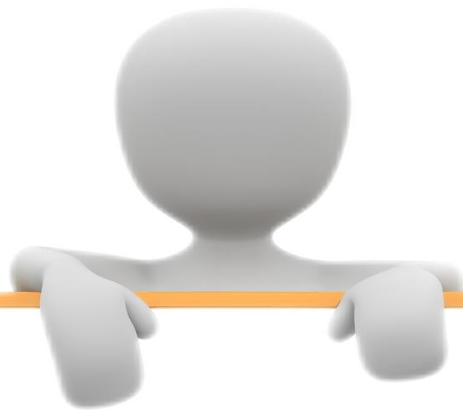
采用有序单链表时：

- A 与 B 归并到 C ， C 利用 A 和 B 的结点空间（ A 和 B 被破坏）。



2.5 有序表

2.5.4 有序表的应用



- 利用有序表元素的有序性提高查找效率
- 利用二路归并过程提高算法效率



2.5 有序表

2.5.4 有序表的应用

【例2.16】已知一个有序单链表L（允许出现值域重复的结点），设计一个高效算法删除值域重复的结点。并分析算法的时间复杂度。

解

- 由于是有序单链表，所以相同值域的结点都是相邻的。
- 用p扫描递增单链表，若p所指结点的值域等于其后继结点的值域，则删除后者。



2.5 有序表

2.5.4 有序表的应用

```
void dels(LinkNode *&L)
{
    LinkNode *p=L->next,*q;
    while (p->next!=NULL)
    {
        if (p->data==p->next->data) //找到重复值的结点
        {
            q=p->next; //q指向这个重复值的结点
            p->next=q->next; //删除q结点
            free(q);
        }
        else //不是重复结点,p指针下移
            p=p->next;
    }
}
```

算法的时间复杂度为 $O(n)$

第2章 小结

一、顺序表和单链表

1、线性表两类存储结构的比较

I. 顺序表

II. 链表



第2章 小结

一、顺序表和单链表

1、线性表两类存储结构的比较

① 顺序表

优点

- 存储密度大：无须为表示线性表中元素之间的逻辑关系而增加额外的存储空间。
- 具有随机存取特性。

缺点

- 插入和删除操作需要移动大量元素。
- 初始空间大小分配难以掌握。

第2章 小结

一、顺序表和单链表

1、线性表两类存储结构的比较

② 链表

优点

- 由于采用结点的动态分配方式，具有良好的适应性。
- 插入和删除操作只需修改相关指针域，不需要移动元素。

缺点

- 存储密度小：为表示线性表中元素之间的逻辑关系而需要增加额外的存储空间（指针域）。
- 不具有随机存取特性。

第2章 小结

一、顺序表和单链表

2、选择适合的数据结构

线性表的算法设计

一般算法如何设计？

- 数据的存储结构 — 顺序表 or 链表？
- 算法的处理过程 — 用C/C++语言描述。



第2章 小结

一、顺序表和单链表

2、选择适合的数据结构

示例

假设一个学生年级有若干个班，每个班有唯一的班号（如1班、2班等），一个班有若干学生（人数可能从10到几百），每个学生信息包括学号和姓名（所有学生的班号和学号均唯一），一个班的学生记录的排列是无序的。其中最频繁的操作如下：

- ① 添加一个某班某学号和姓名的学生记录。
- ② 删除某班某学号的学生记录。
- ③ 查找某班某学号的学生记录。

第2章 小结

二、其他链表

- 双链表
- 循环链表
- 有序表



第2章 小结

二、其他链表

1、双链表

- ① 每个结点有指向前、后相邻结点的指针域。



通常，存储密度低于单链表



第2章 小结

二、其他链表

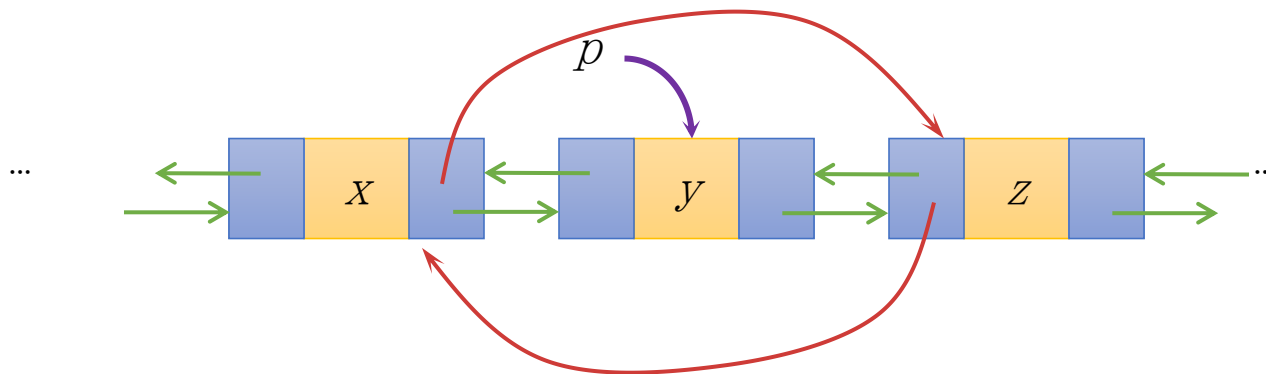
1、双链表

② 特点：方便查找一个结点的前、后相邻结点。

- 已知某个结点的地址，删除它的时间为 $O(1)$ 。

删除过程：

$p \rightarrow \text{prior} \rightarrow \text{next} = p \rightarrow \text{next}$



$p \rightarrow \text{next} \rightarrow \text{prior} = p \rightarrow \text{prior}$



修改p结点前驱结点的next指针和p结点后继结点的prior指针。

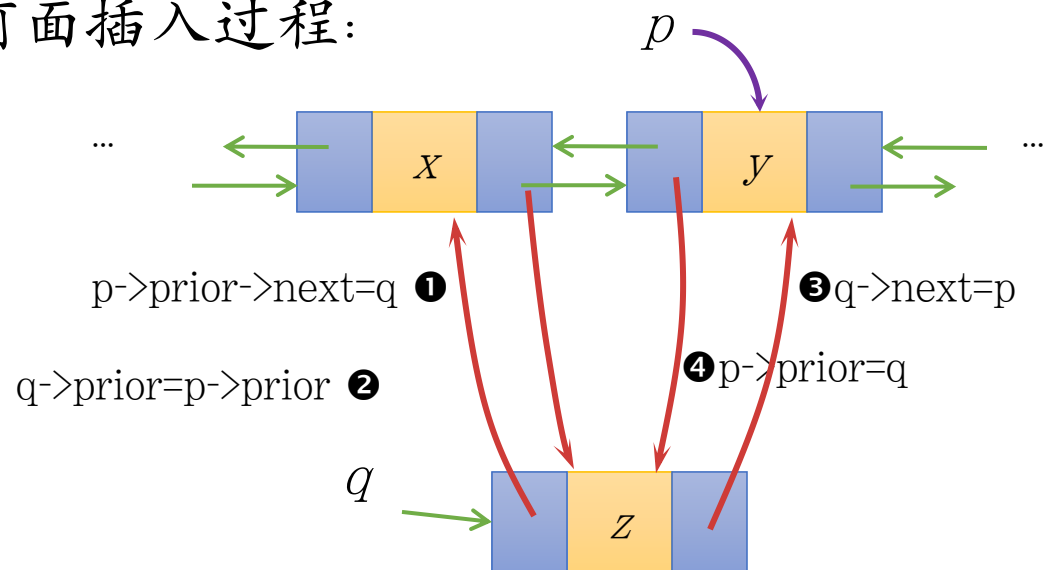
第2章 小结

二、其他链表

1、双链表

- 在某个结点的前、后插入一个结点的时间为 $O(1)$ 。

前面插入过程:



通常修改 $p \rightarrow \text{prior}$ 在最后进行!

第2章 小结

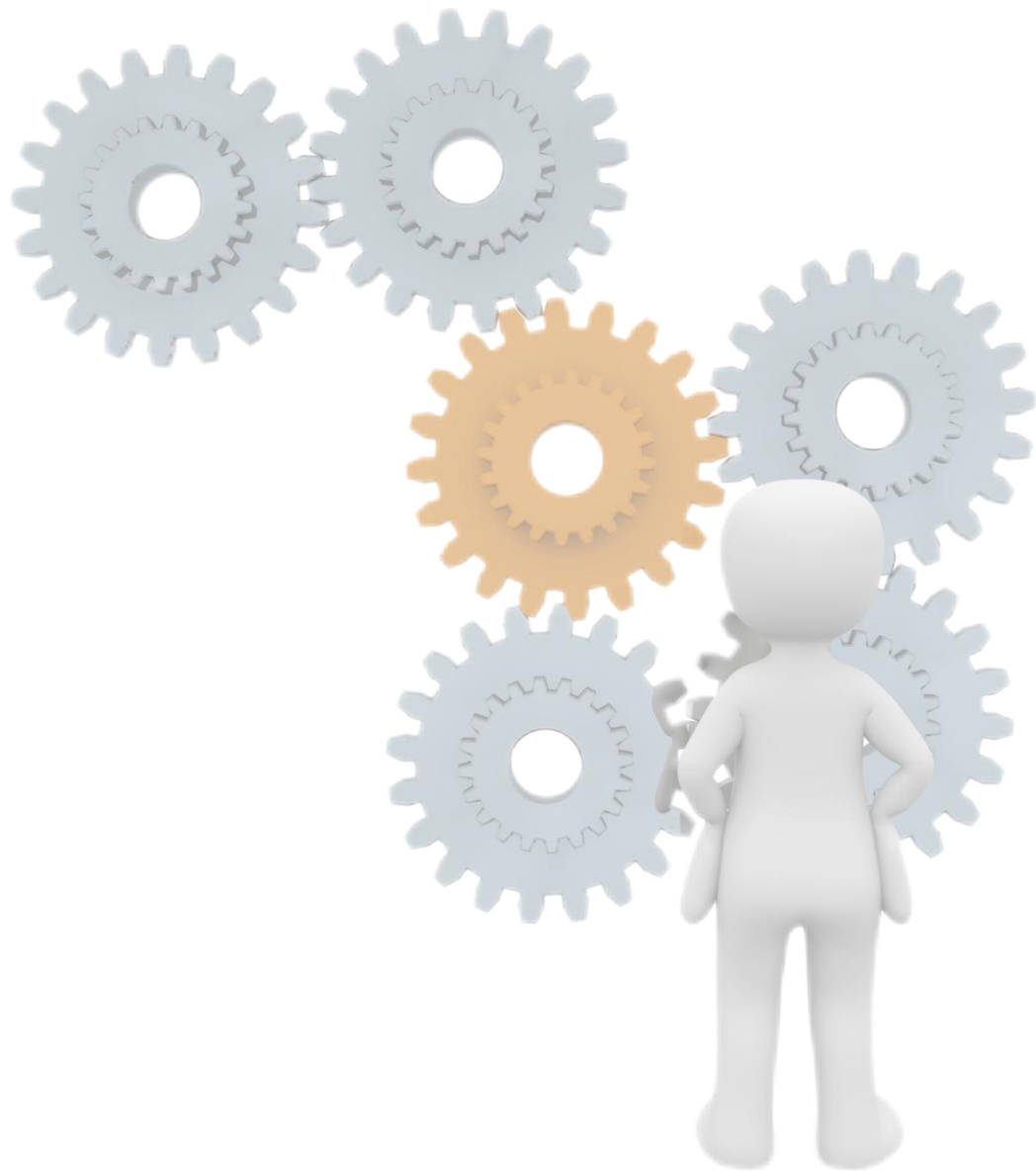
二、其他链表

2、循环链表

- ① 循环单链表：构成一个环。



可以循环查找



第2章 小结

二、其他链表

2、循环链表

② 循环双链表：构成两个环。



- 可以循环查找
- 可以通过头结点快速找到尾结点



删除尾结点、在尾结点前后插入一个结点
的时间均为 $O(1)$ 。

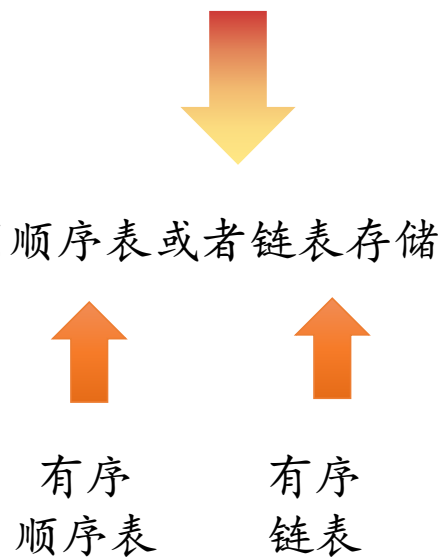


第2章 小结

二、其他链表

3、有序表

① 从逻辑结构看，有序表是线性表的一个子集。



第2章 小结

二、其他链表

3、有序表

② 利用有序表的有序特性可以提高相关算法的效率

示例

假设一个有序表采用顺序表存储。设计一个高效算法删除重复的元素。

例如: $L=(1, 1, 1, 2, 2, 3)$

↓ 本算法

$L=(1, 2, 3)$



第2章 小结

二、其他链表

3、有序表

③ 利用二路归并思路可以提高相关算法的效率

示例

假设两个递增有序表采用单链表ha和hb存储（假设同一个单链表中不存在重复的元素）。
设计一个高效算法求它们的公共元素，将结果存放在单链表hc中。

例如：ha=(1, 2, 3), hb=(2, 3, 4)

↓ 本算法

hc=(2, 3)

